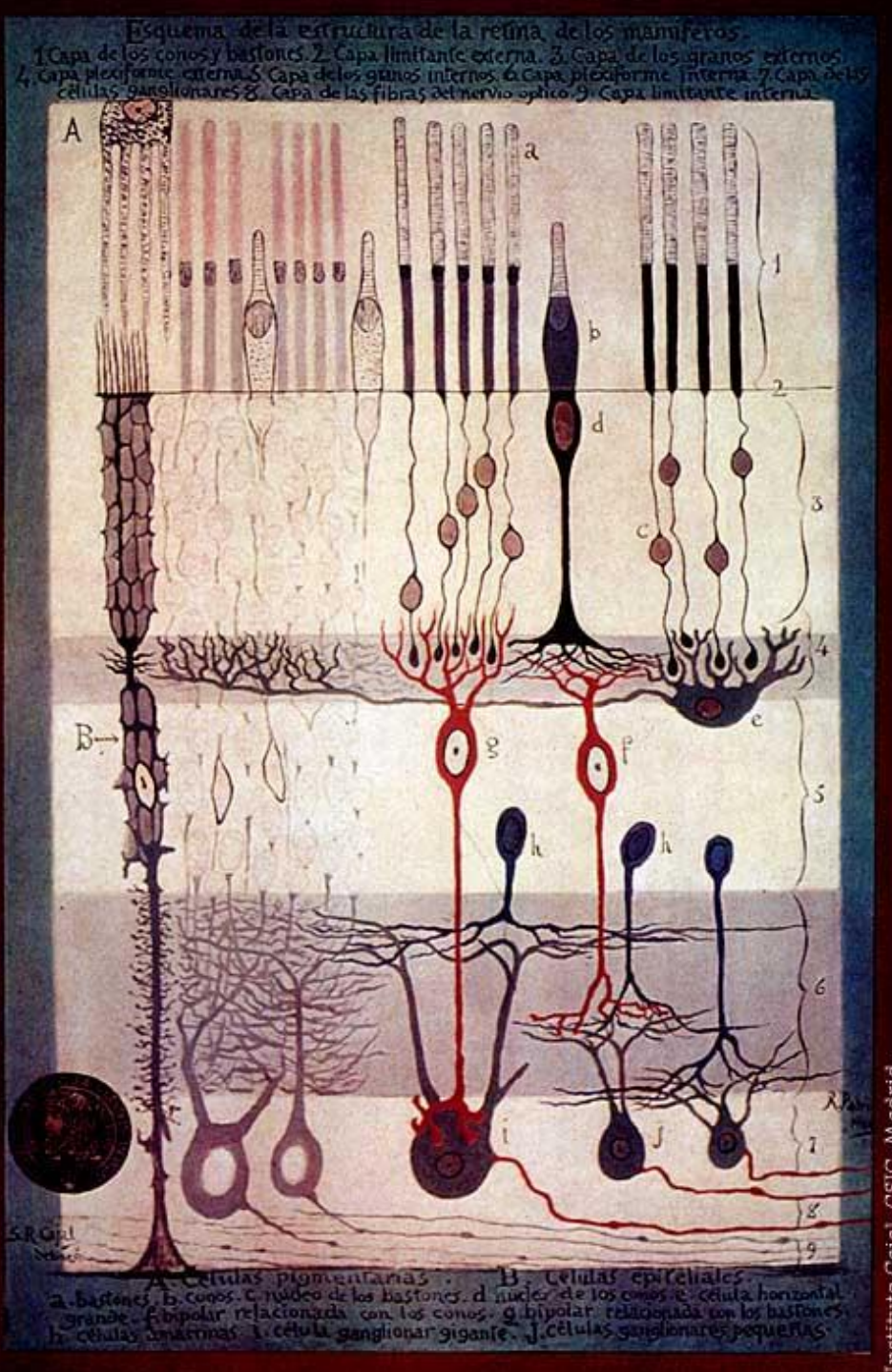




Convolutional Neural Network

draft slide, do not print

Computer Vision and Artificial Intelligence

Dr Varun Ojha

varun.ojha@ncl.ac.uk

ojhavk.github.io

School of Computing
Newcastle University

Drawing of mammalian retina by
Santiago Ramón y Cajal

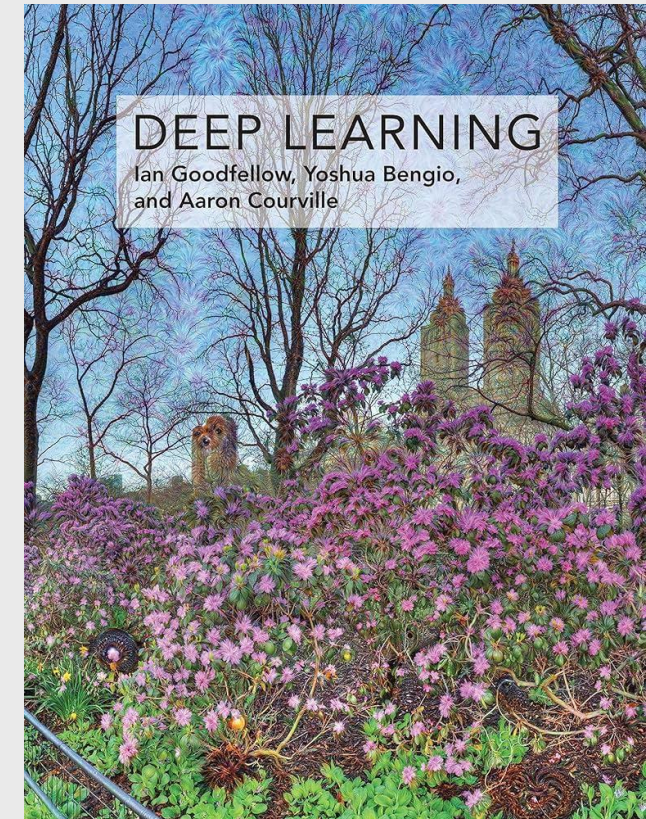
Learning objectives (Convolutional Nets)

By the end of this week, you will be able to:

- Learn the concepts of convolutional neural networks (CNNs or ConvNets)
- Design various convolutional architectures
- Understand computer vision tasks and models:
 - image classification
 - image segmentation
 - object detection
 - Image generation
- Apply and evaluate a ConvNet on image classification task.

Content of this week (CNNs)

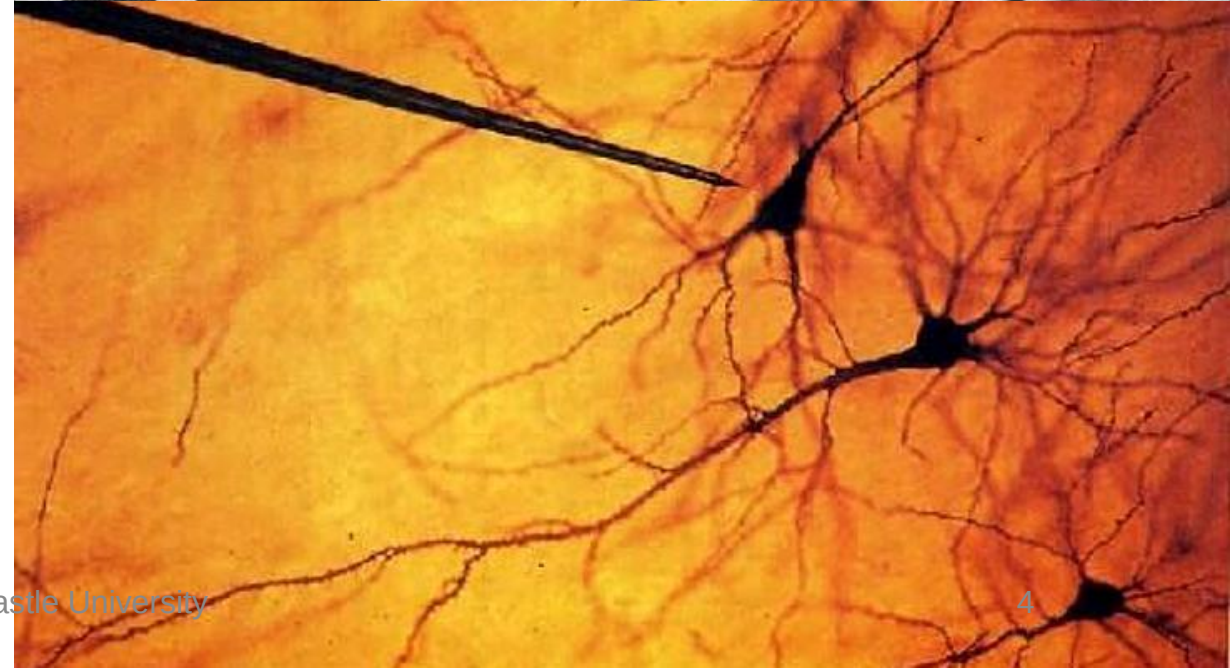
- **Part 1: Introduction**
- **Part 2: Design of Convolutional Nets**
 - Image Data
 - Components of ConvNets
 - Regularisation in ConvNets / DNNs
 - ConvNet Architectures
- **Part 3: Convolutional Neural Nets Applications and Models**
 - Image Segmentation Models
 - Object Detection Models
 - Generative Models Concept
- **Part 4: Practical Exercise (CNN)**



Goodfellow et al (2017) Deep Learning, MIT Press
<https://www.deeplearningbook.org/>

Part 1

Introduction



(Top) David Hubel (left) and Torsten Wiesel (right)
(Bottom) Image of a tungsten electrode (invented by David Hubel) in the brain, with individual neurons stained in black, using the Golgi method. The cell body, axon, and dendrites of neurons are visible.

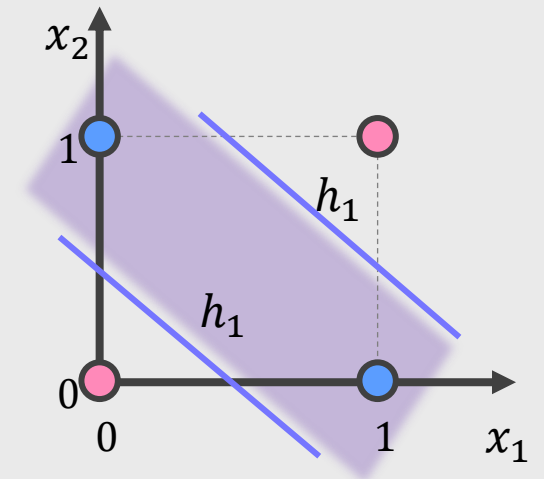
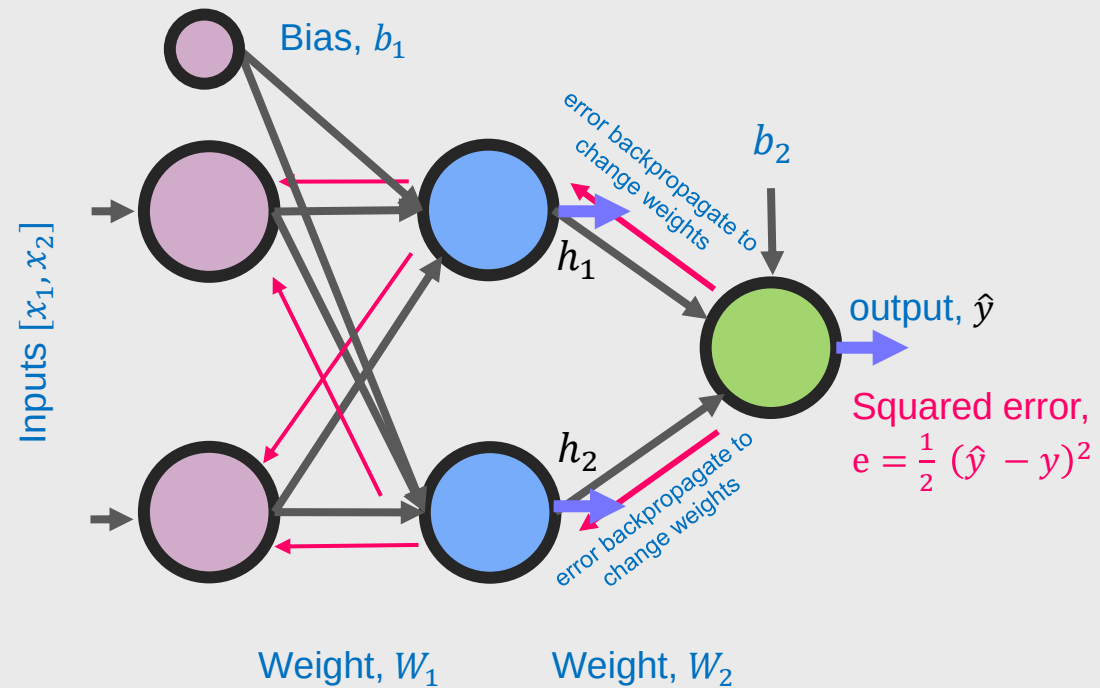
Image source: <https://explorebiology.org/collections/cell-biology/how-neurons-are-organized-in-the-brain-to-produce-thought>
Dr Varun Ojha, Newcastle University

Deep Neural Networks

Error backpropagation can solve non-linearly separable patterns

XOR Problem

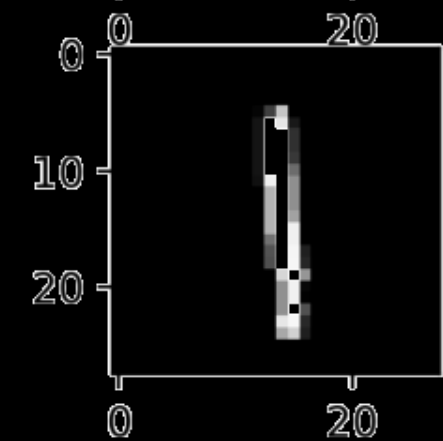
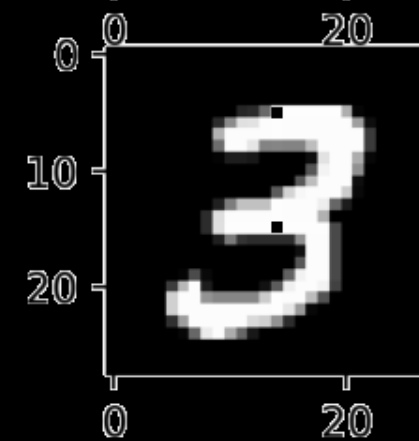
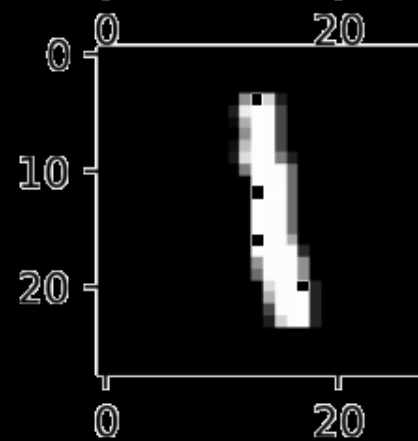
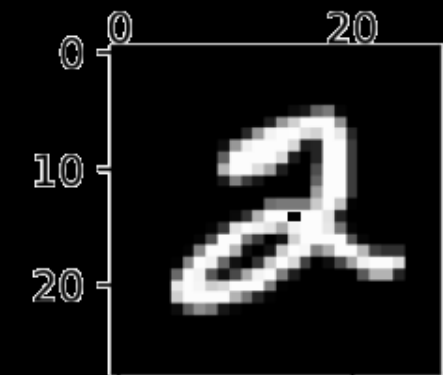
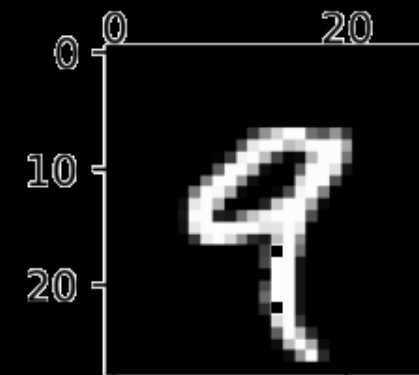
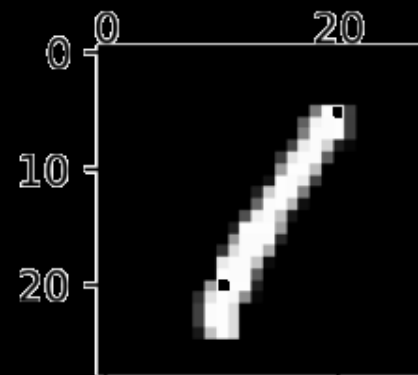
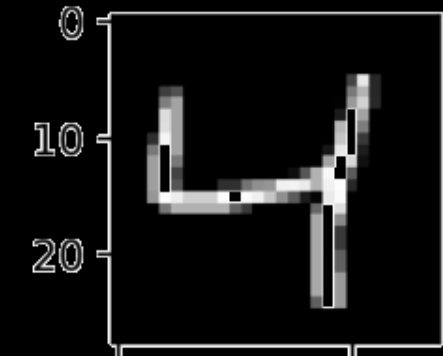
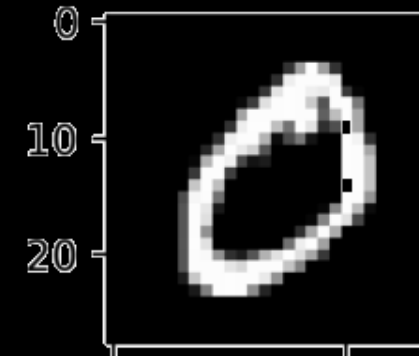
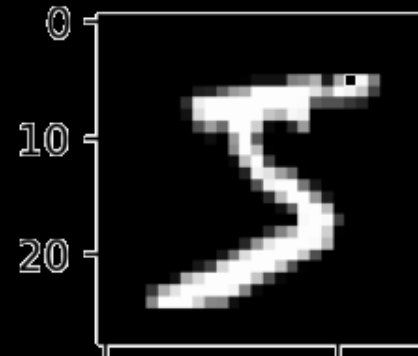
	x_1	x_2	$y = f(\mathbf{x})$
1:	0	0	0
2:	0	1	1
3:	1	0	1
4:	1	1	0



Sec 8.1.1 to 8.1.3, *Deep Learning* by C Bishop

Computer Vision and Pattern Recognition

Dataset is a set of
labelled images



Computer Vision

Dataset is a set of labelled images.

Images can more complex images than just character recognition and the task can vary e.g.

classifying one image against other or

recognising objects such rock, water, statue, cloud, sun etc.



Data: 2D

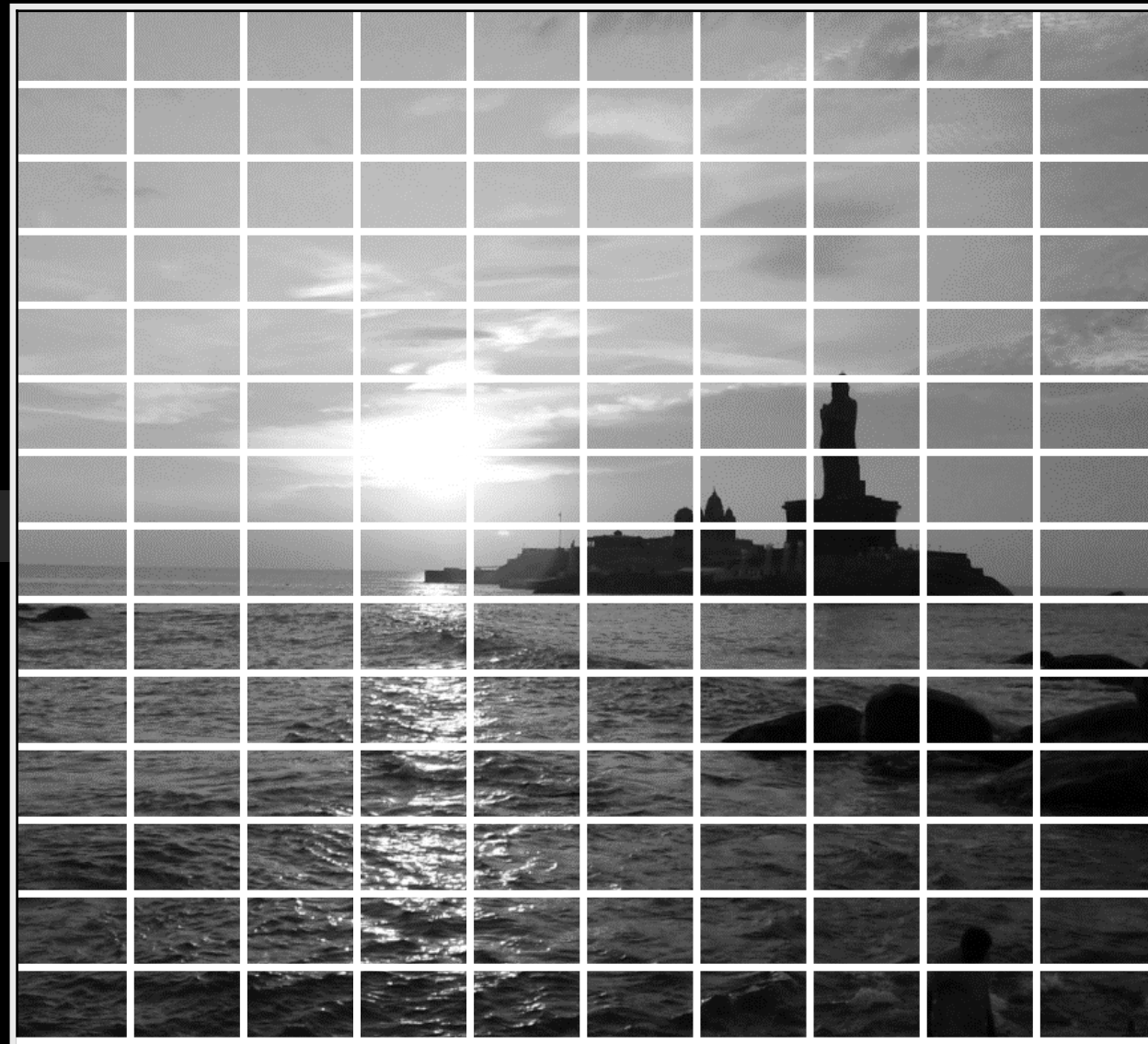
Image can be gray scale

$$I = \begin{bmatrix} p_{11} & \cdots & p_{1,W} \\ \vdots & \ddots & \vdots \\ p_{H,1} & \cdots & p_{H,W} \end{bmatrix}$$

Height (H)

For Height = 256, Width = 256

$$I = \begin{bmatrix} p_{11} & \cdots & p_{1,256} \\ \vdots & \ddots & \vdots \\ p_{256,1} & \cdots & p_{256,256} \end{bmatrix}$$



Width (W)

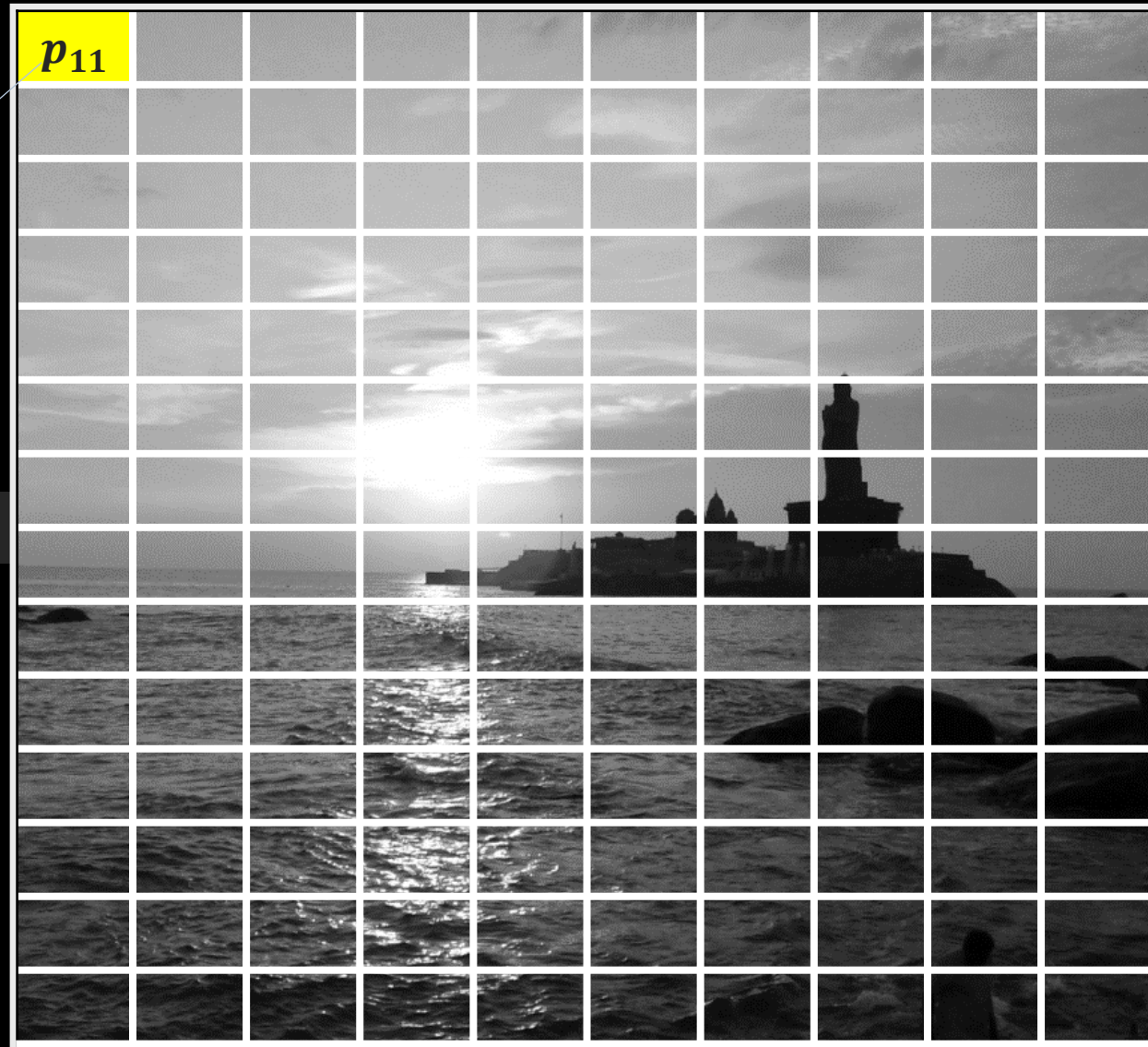
Data: 2D

Image can be gray scale

$$p_{ij} \in \{0, 1, 2, \dots, 255\}$$

For $Height = 256, Width = 256$

$$I = \begin{bmatrix} p_{11} & \cdots & p_{1,256} \\ \vdots & \ddots & \vdots \\ p_{256,1} & \cdots & p_{256,256} \end{bmatrix}$$



Height (H)

Width (W)

Data

Image can be color scale

11:31
AM



Data: 3D

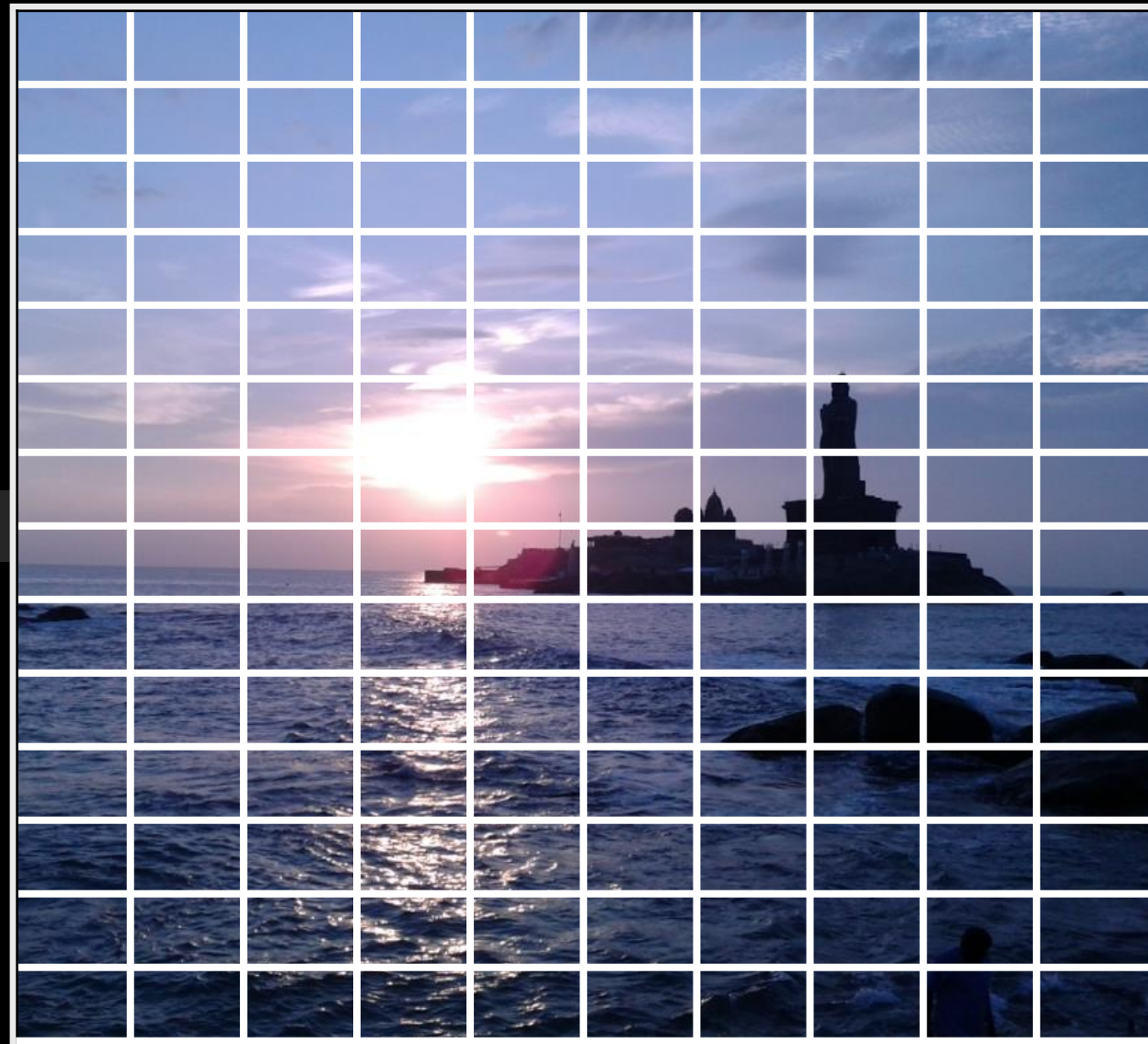
Image can be color scale

$$I_{RED} = \begin{bmatrix} p_{11} & \cdots & p_{1,W} \\ \vdots & \ddots & \vdots \\ p_{H,1} & \cdots & p_{H,W} \end{bmatrix}$$

Height (H)

$$I_{Green} = \begin{bmatrix} p_{11} & \cdots & p_{1,W} \\ \vdots & \ddots & \vdots \\ p_{H,1} & \cdots & p_{H,W} \end{bmatrix}$$

$$I_{Blue} = \begin{bmatrix} p_{11} & \cdots & p_{1,W} \\ \vdots & \ddots & \vdots \\ p_{H,1} & \cdots & p_{H,W} \end{bmatrix}$$



Width (W)

Data

Image can be color scale

$$I_{RED} = \begin{bmatrix} p_{11} & \cdots & p_{1,W} \\ \vdots & \ddots & \vdots \\ p_{H,1} & \cdots & p_{H,W} \end{bmatrix}$$

$$I_{Green} = \begin{bmatrix} p_{11} & \cdots & p_{1,W} \\ \vdots & \ddots & \vdots \\ p_{H,1} & \cdots & p_{H,W} \end{bmatrix}$$

$$I_{Blue} = \begin{bmatrix} p_{11} & \cdots & p_{1,W} \\ \vdots & \ddots & \vdots \\ p_{H,1} & \cdots & p_{H,W} \end{bmatrix}$$

Channel /Depth (D)

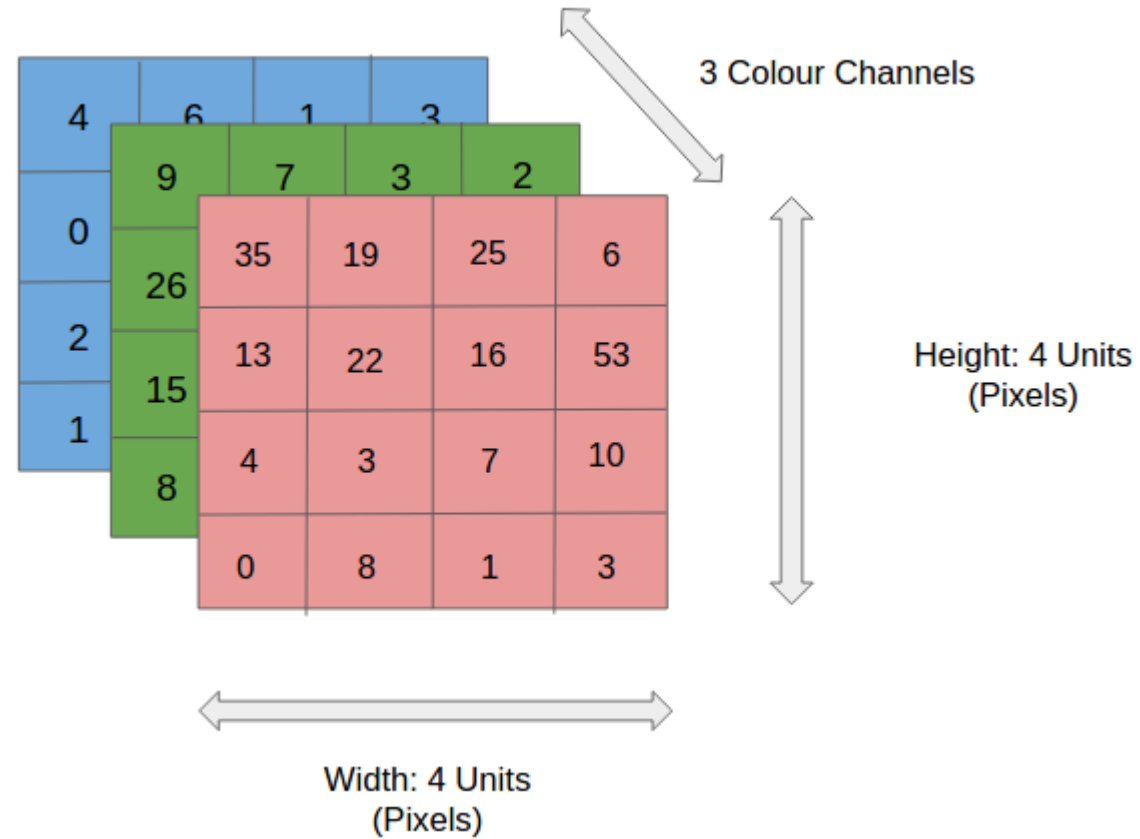
Height (H)

Width (W)



11:31
AM

Computer Vision data



Part 2

Design of Convolutional Nets

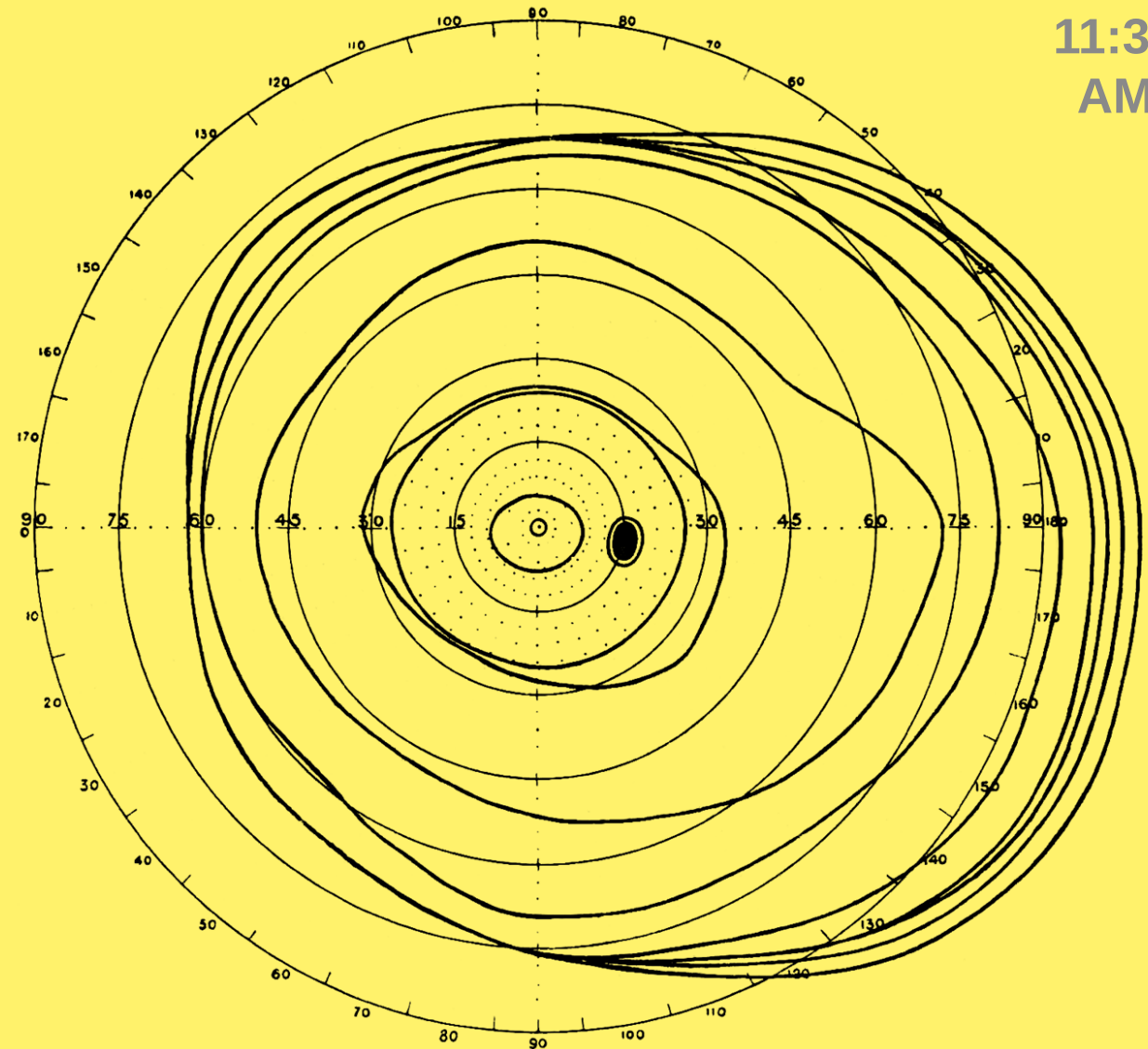
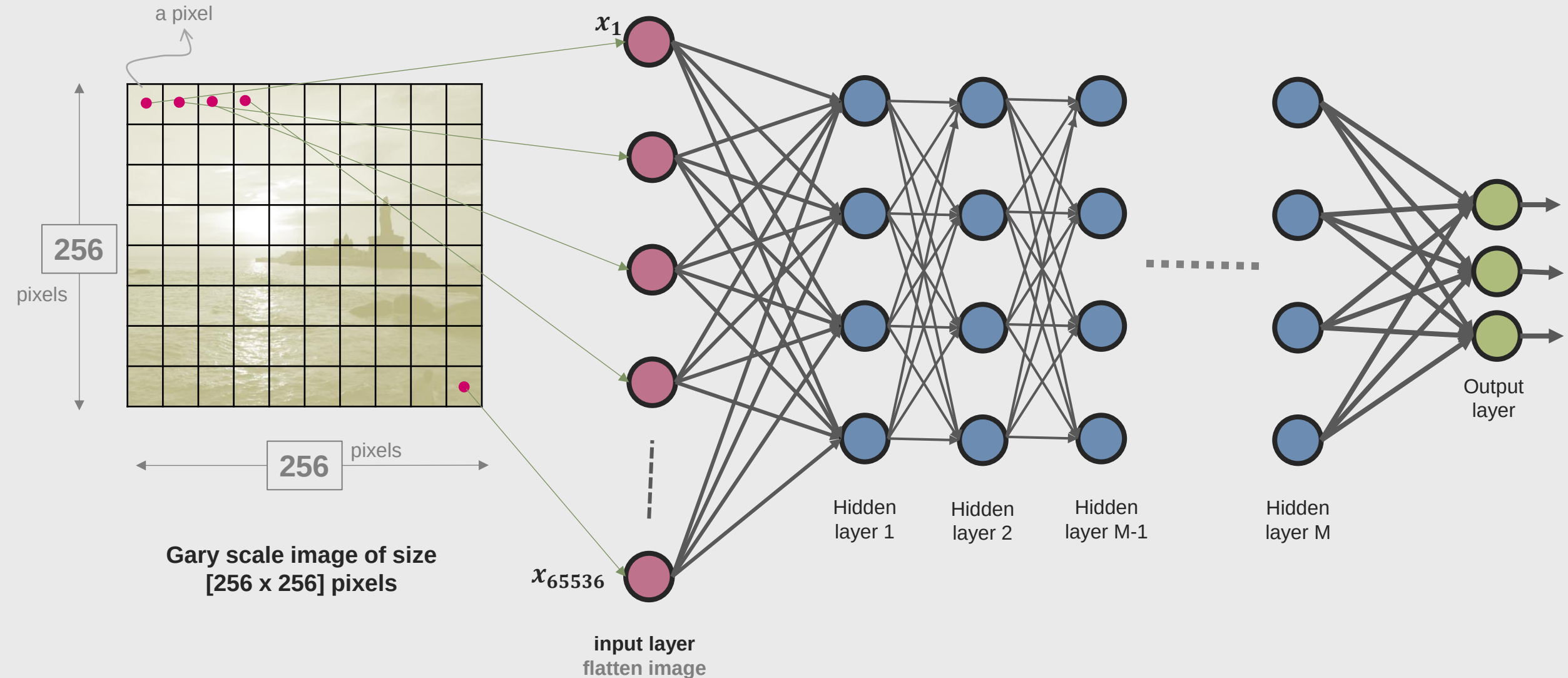


FIG. 1.—CHART OF THE VISUAL FIELD OF THE RIGHT EYE, SHOWING THE ISOPTERS FROM THE PERIPHERY INWARDS TO THE CENTRE FOR $\frac{160}{1000}$, $\frac{80}{1000}$, $\frac{40}{1000}$, $\frac{20}{1000}$, $\frac{10}{1000}$, $\frac{5}{1000}$, $\frac{2.5}{1000}$, $\frac{1.5}{1000}$, $\frac{1}{1000}$, AND $\frac{0.63}{1000}$.

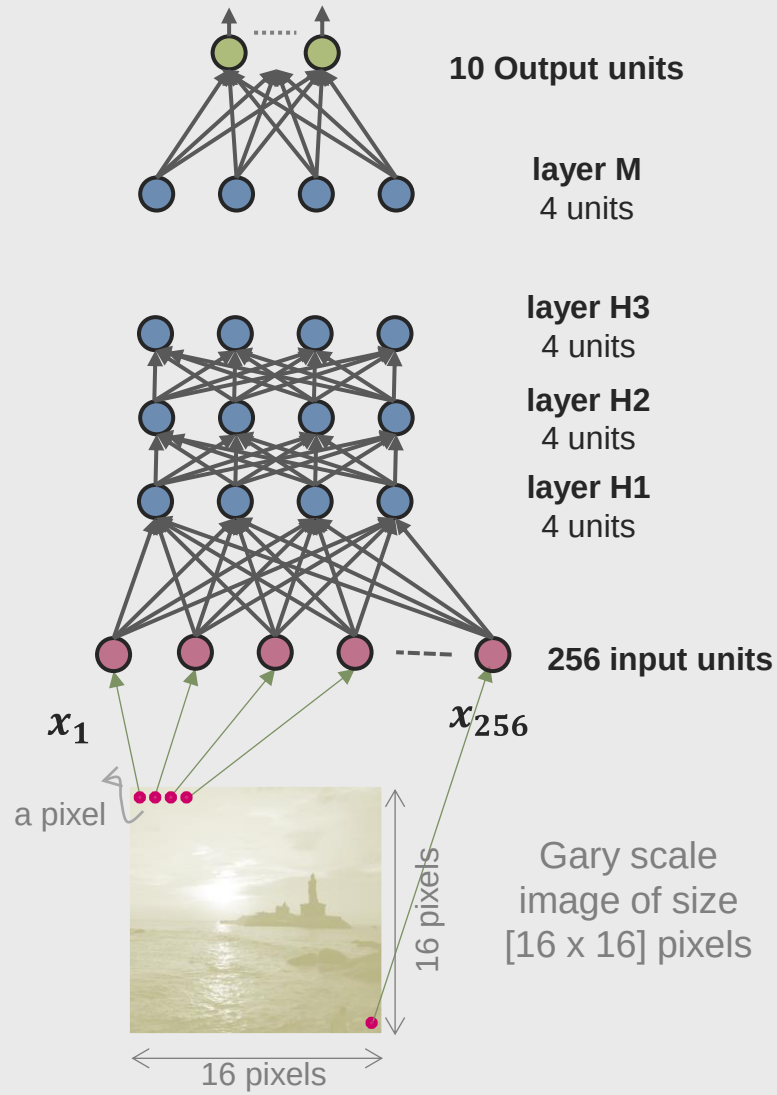
The numerator of each fraction represents the diameter of the test object in millimetres, the denominator the distance from the patient. It will be seen that as the test object is reduced in size the field becomes smaller, slowly at the periphery, but rapidly towards the centre. On the nasal side the larger test objects all give the same extent of field, so that the nasal edge of the field is perpendicular, the temporal edge being steeply sloping. The geometrical centre of the field is about 20° to the outer side of the visual axis or physiological centre.

Computer Vision using Deep Neural Networks

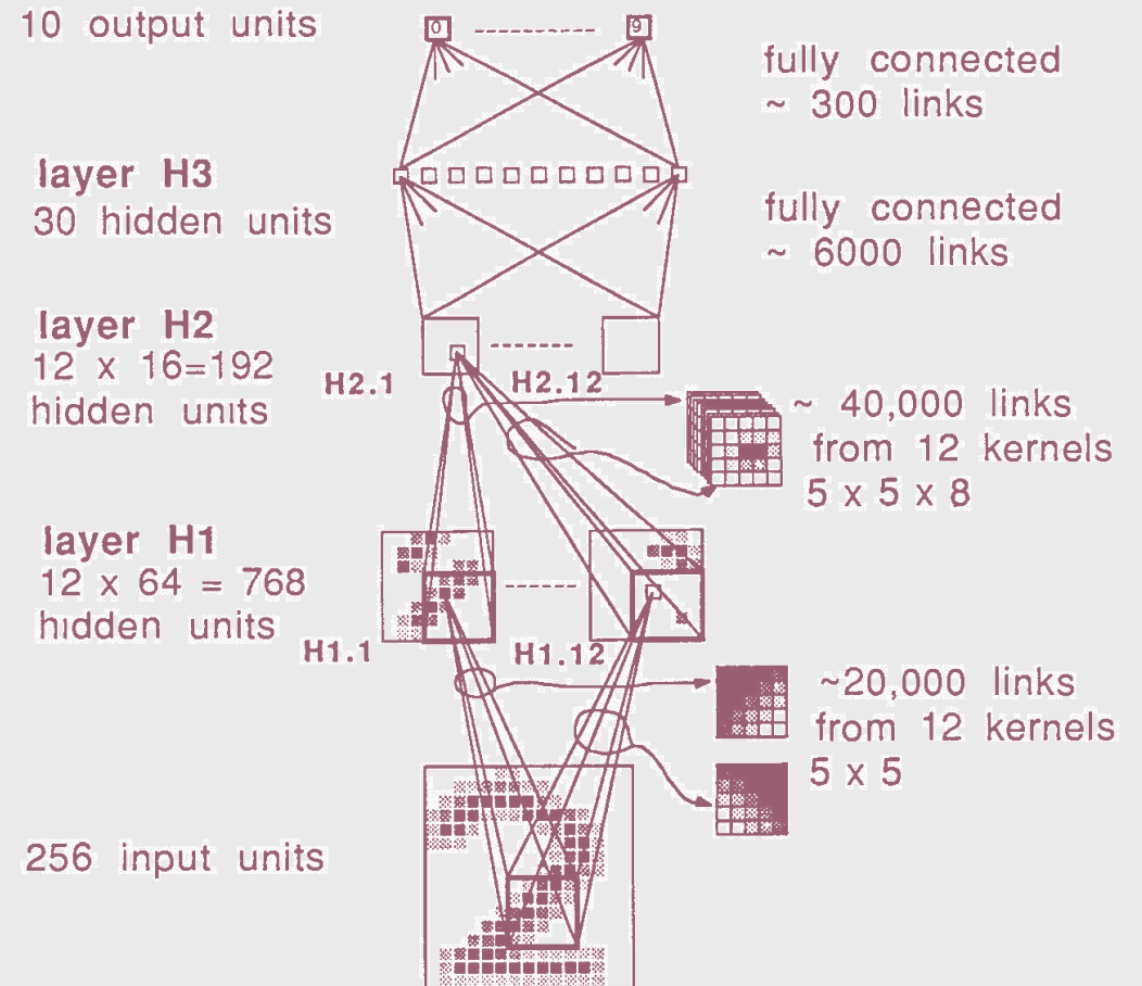


Computer Vision using Convolutional Neural Networks

11:31
AM



Deep Neural Network (DNN)

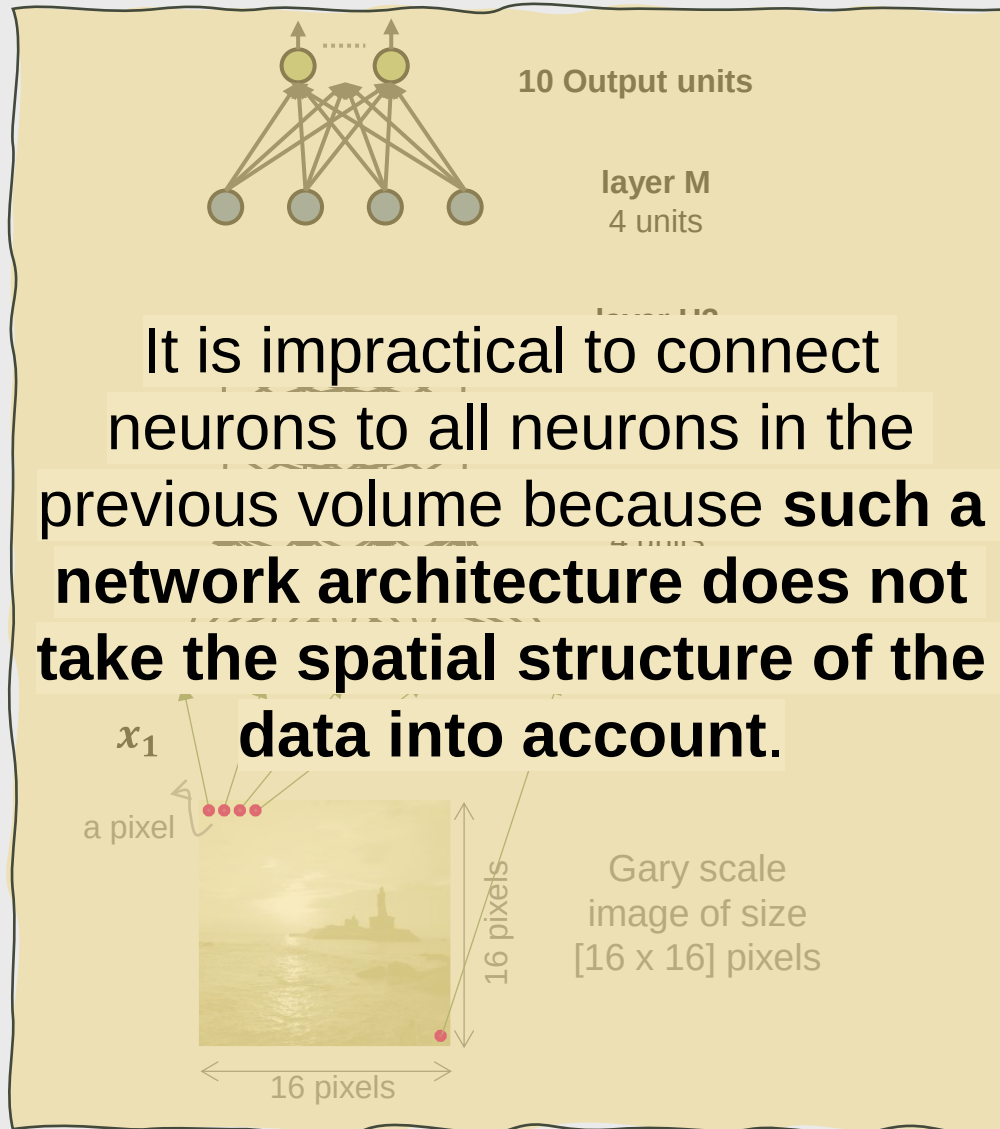


Convolutional Neural Network (CNN)

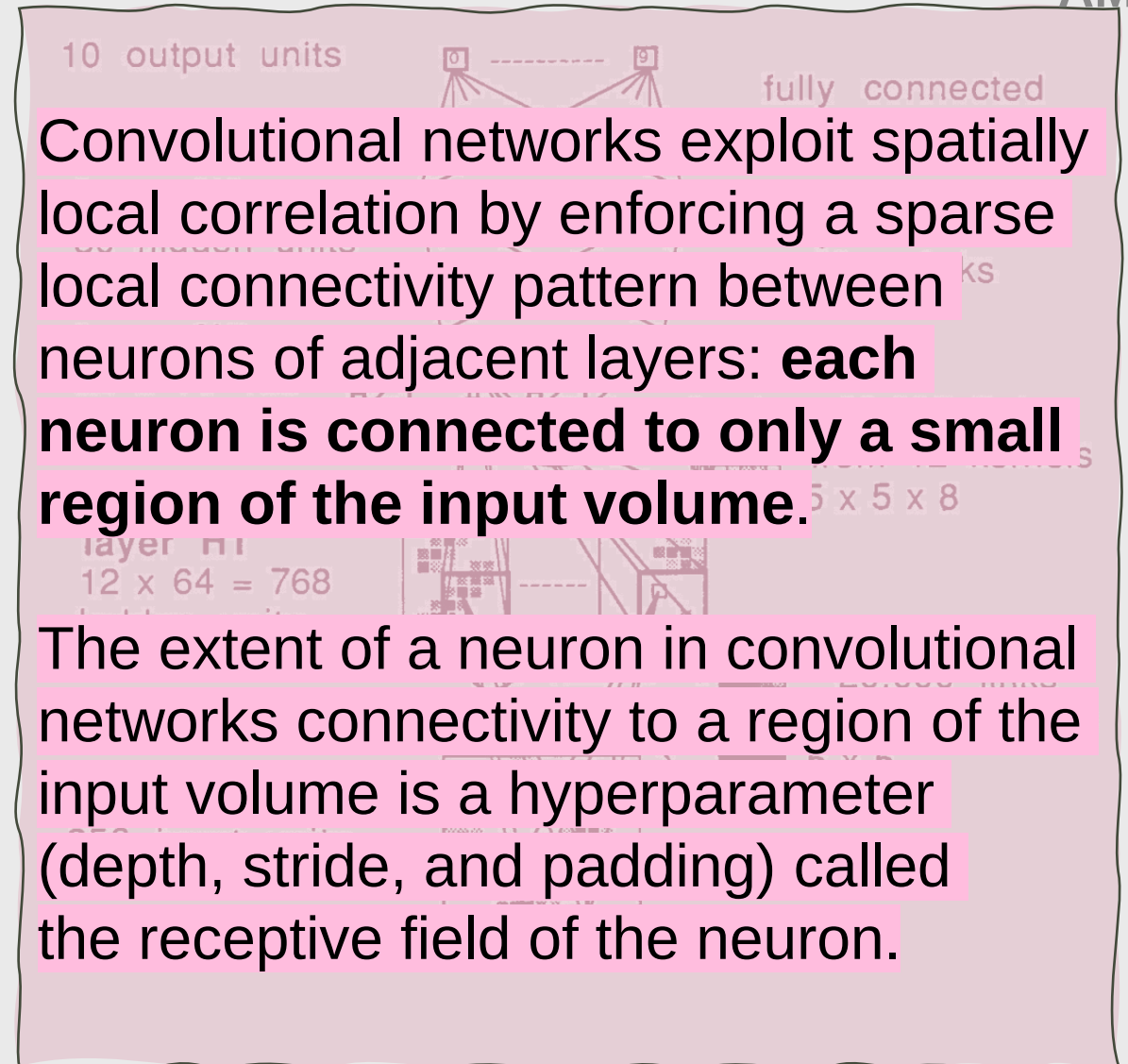
CNN developed by Y LeCun, 1989

Computer Vision using Convolutional Neural Networks

11:31
AM



Deep Neural Network (DNN)



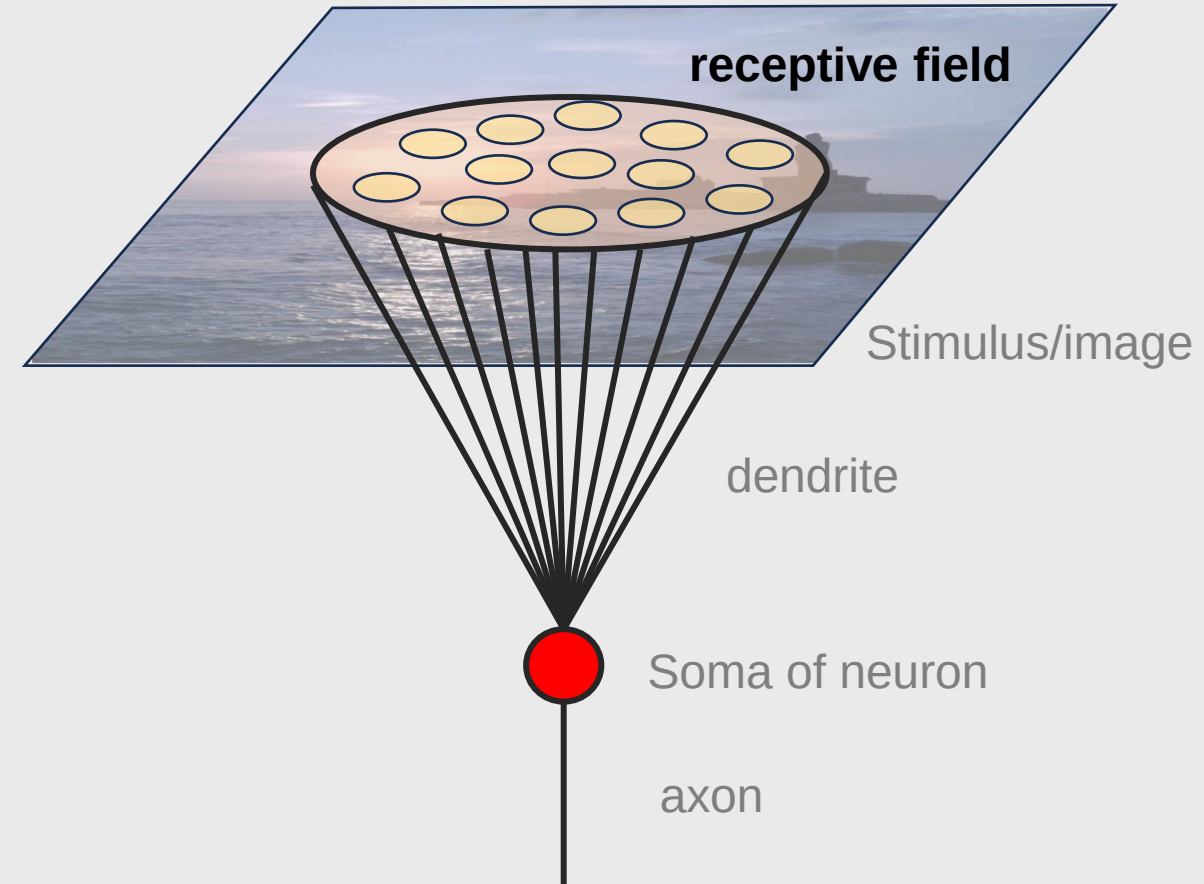
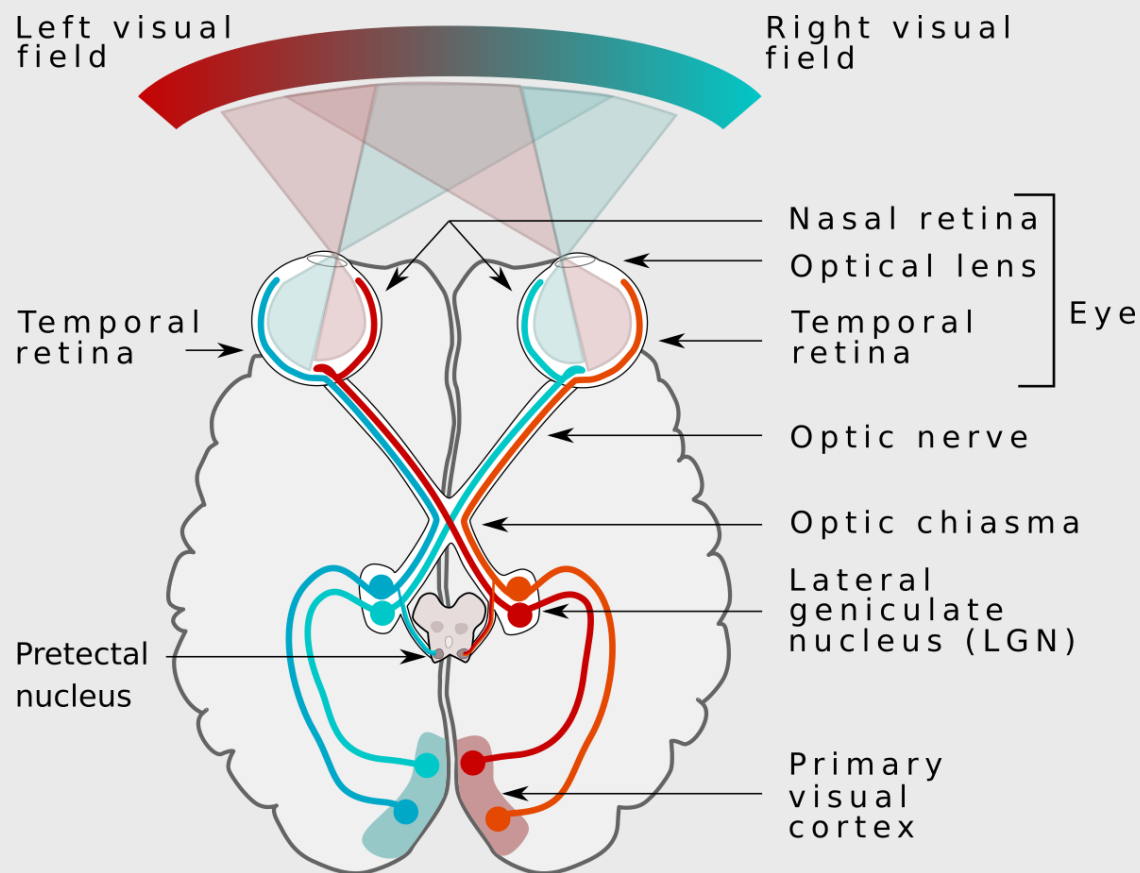
Convolutional Neural Network (CNN)

CNN developed by Y LeCun, 1989

Receptive Field

11:31
AM

The receptive field (RF) of a neuron (biological neuron) refers to the attributes of a visual stimulus that generates a response in that cell. Thompson et al (2017). Activity-dependent development of visual receptive fields. Current Opinion Neurobiology



Receptive Field Analogy in Convolutional Nets ^{11:31 AM}

Convolutional operation with depth, stride, and padding is an operation to processes the visual stimuli. Each convolutional neuron processes data only for its receptive field.

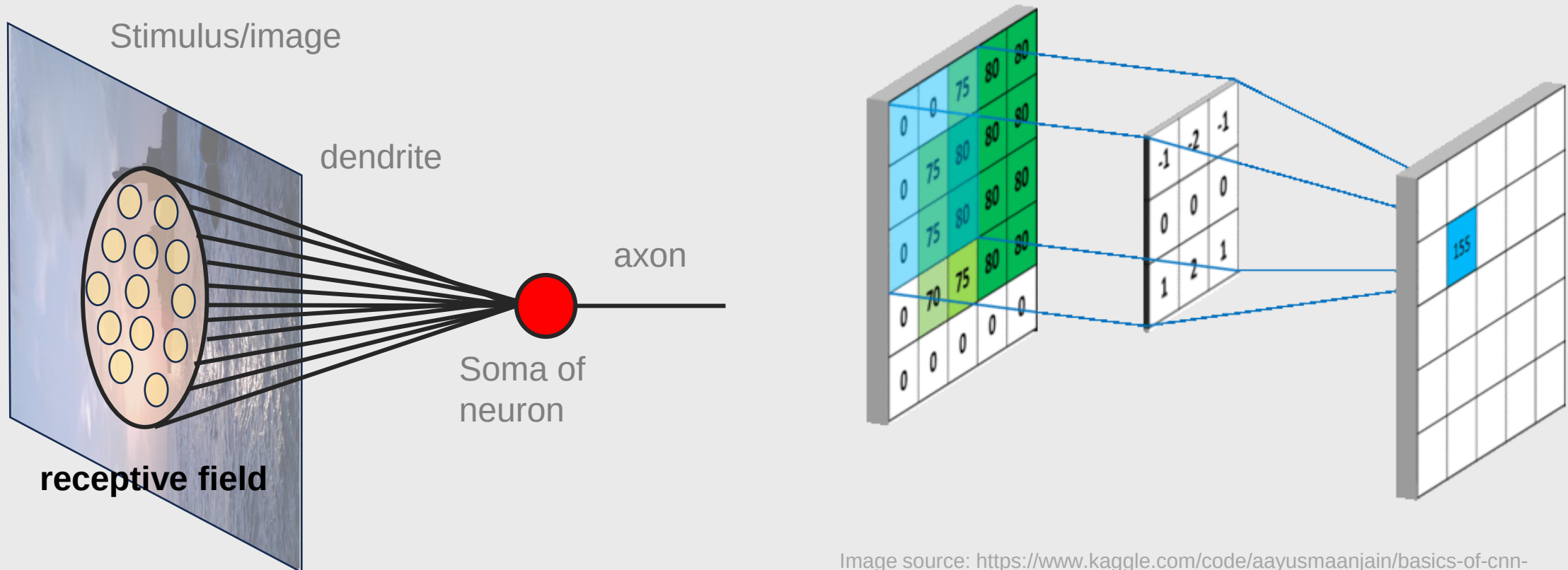
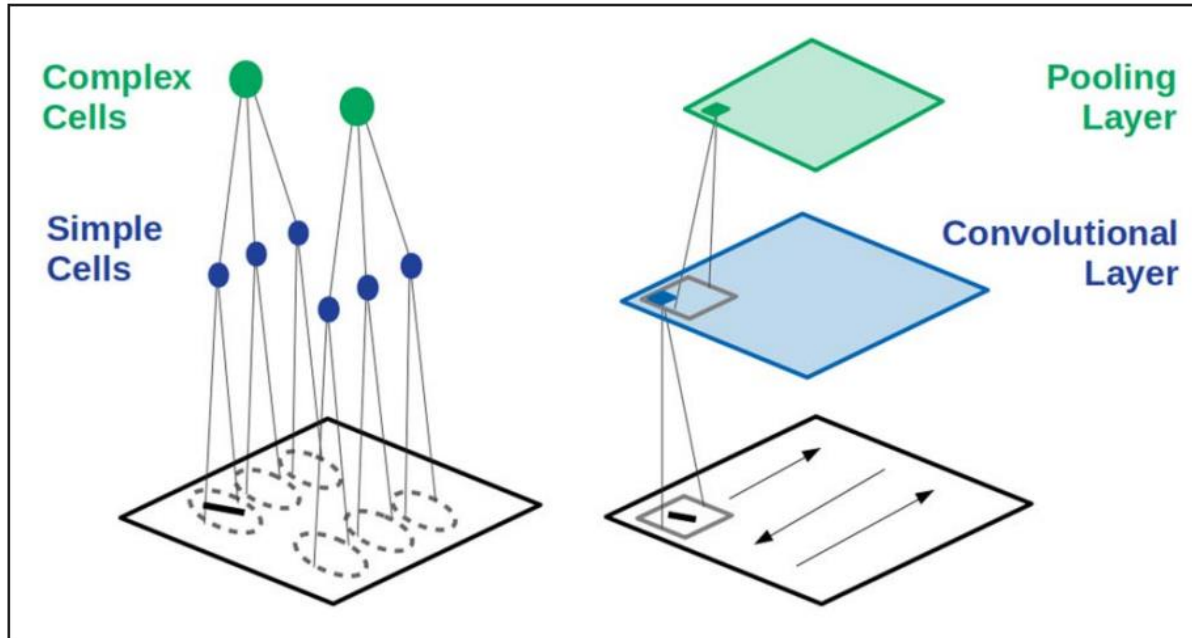


Image source: <https://www.kaggle.com/code/aayusmaanjan/basics-of-cnn-with-pytorch-lightning>

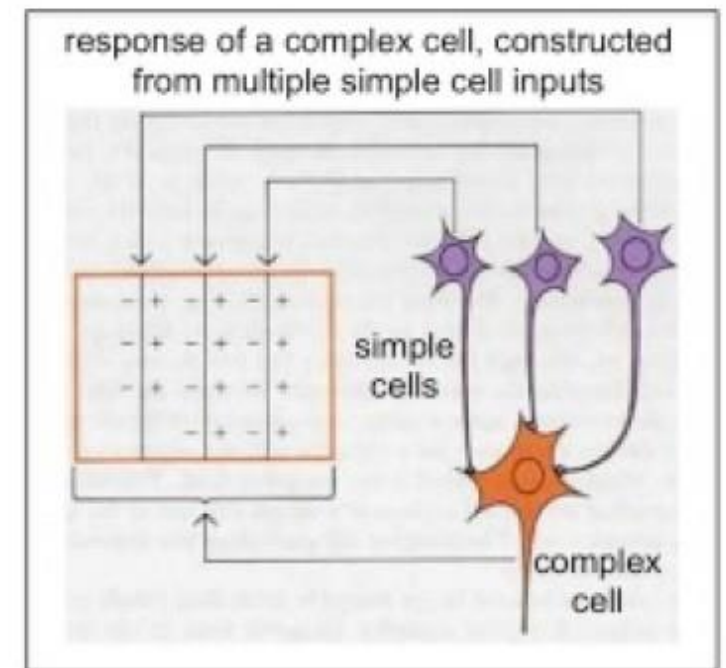
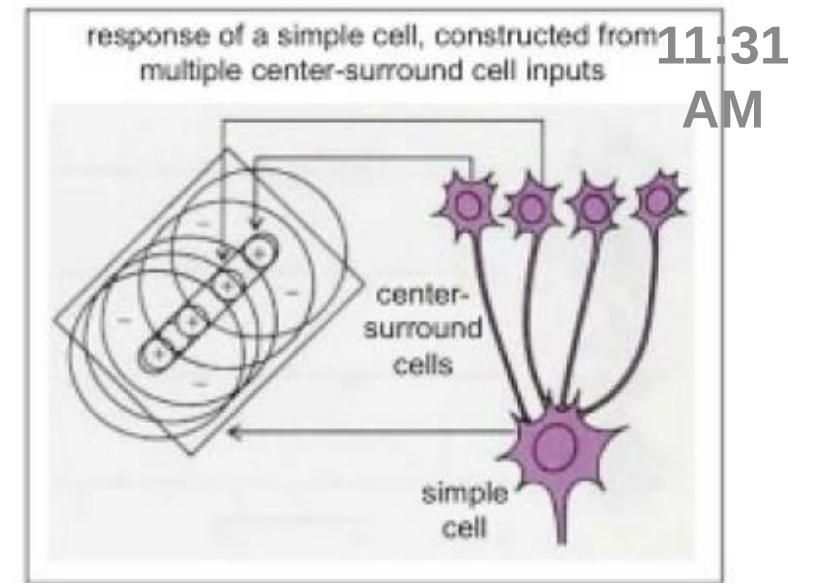
Biological Inspiration of Convolutional Neural Network



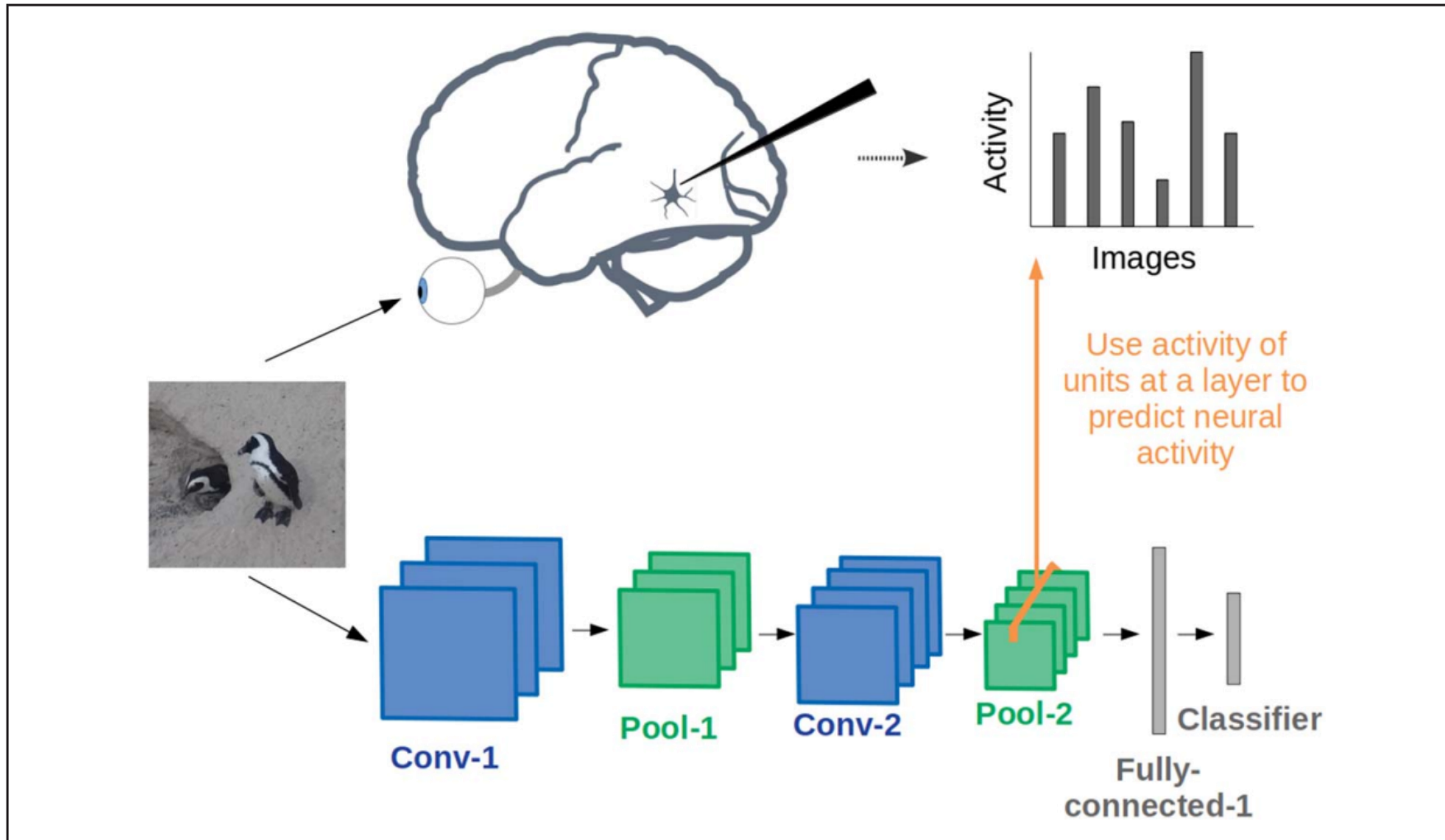
Lindsay (2019) Journal of Cognitive Neuroscience

Hubel and Wiesel (1962) discovered that simple cells (left, blue) have preferred locations in the image (dashed ovals) wherein they respond most strongly to bars of particular orientation. Complex cells (green) receive input from many simple cells and thus have more **spatially invariant responses**. These operations are replicated in a CNN (right).

Dr Varun Ojha, Newcastle University



CNN and an Animal response to an Image



Modern CNNs start by convolving a set of filters with an input image and rectifying the outputs, leading to “feature maps” akin to the planes of S-cells in the Noncognition. Max pooling is then applied, creating complex cell-like responses. After several iterations of this pattern, non-convolutional fully connected layers are added, and the last layer contains as many units as number of categories in the task to output a category label for the image

Biol. Cybernetics 36, 193–202 (1980)

**Biological
Cybernetics**

© by Springer-Verlag 1980

Neocognitron: A Self-organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position

Kunihiko Fukushima

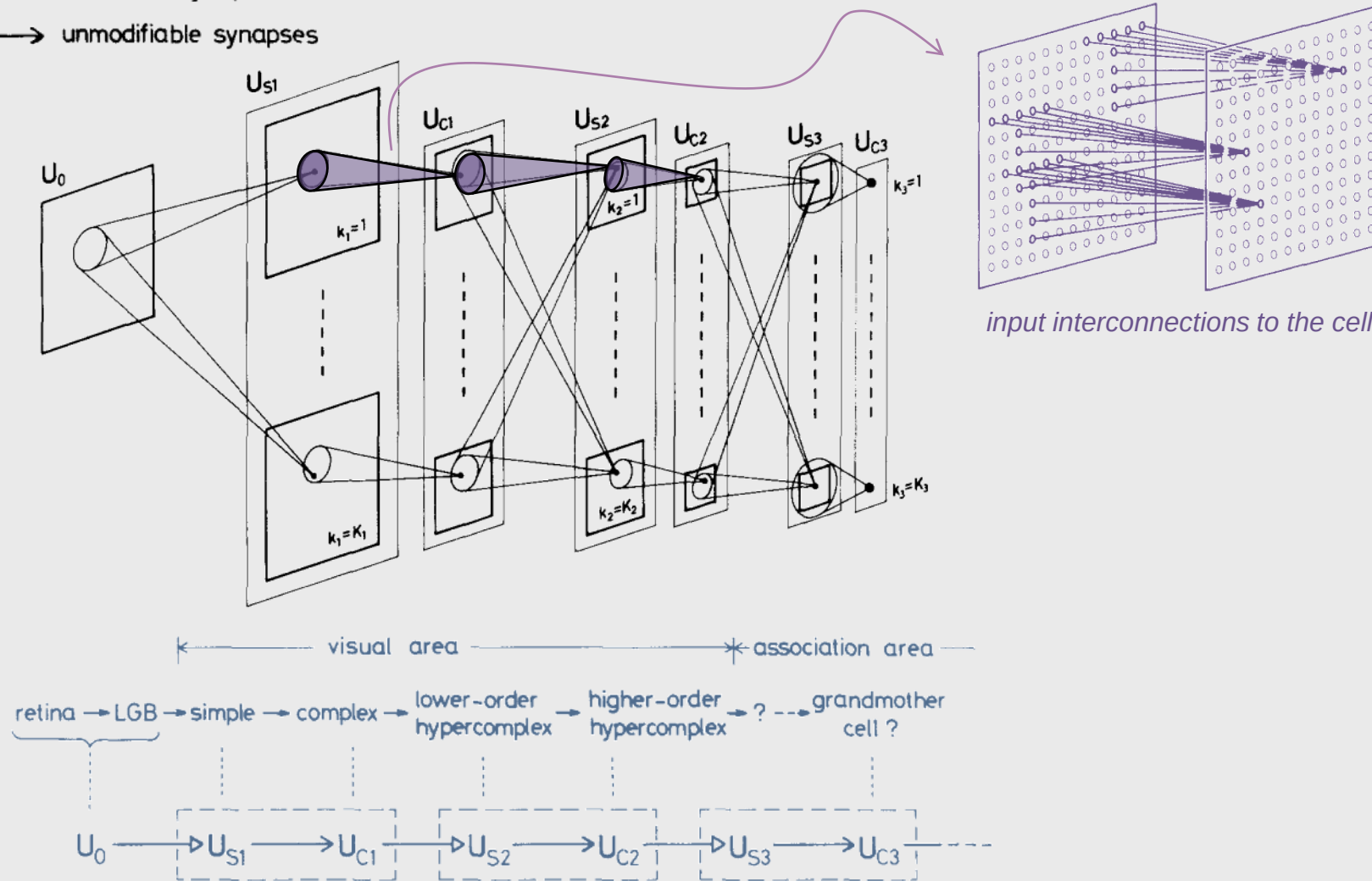
NHK Broadcasting Science Research Laboratories, Kinuta, Setagaya, Tokyo, Japan

Convolutional Layer: Neocognitron

11:31
AM

→ modifiable synapses

→ unmodifiable synapses



Input: two-dimensional photoreceptors array of cells, like retina

S-layer (U_{sl}):
Simple cell layer

C-layer (U_{cl}):
Complex cell-layer

Neural network of the Neocognitron

Kunihiko Fukushima (1980)

Convolutional Nets is Translation-equivariant

11:31
AM

Convolutional Neural Network (advantages over deep neural nets)

Convolutional neural network are Spatial invariance

(although not perfectly spatially invariant as shown by Islam, Jia and Bruse 2020)

Spatial invariance is the property of a system to produce the same output even if the input is shifted, moved, rotated, or translated.

translation-equivariant, meaning if an input shifts, the output will shift by the same amount. Convolutional and pooling layers are translational equivariant



(a) translation



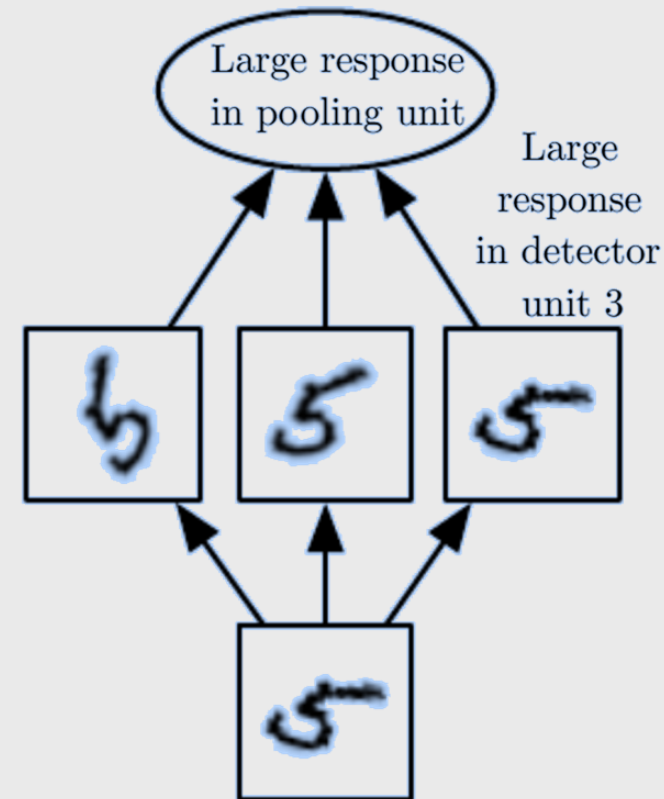
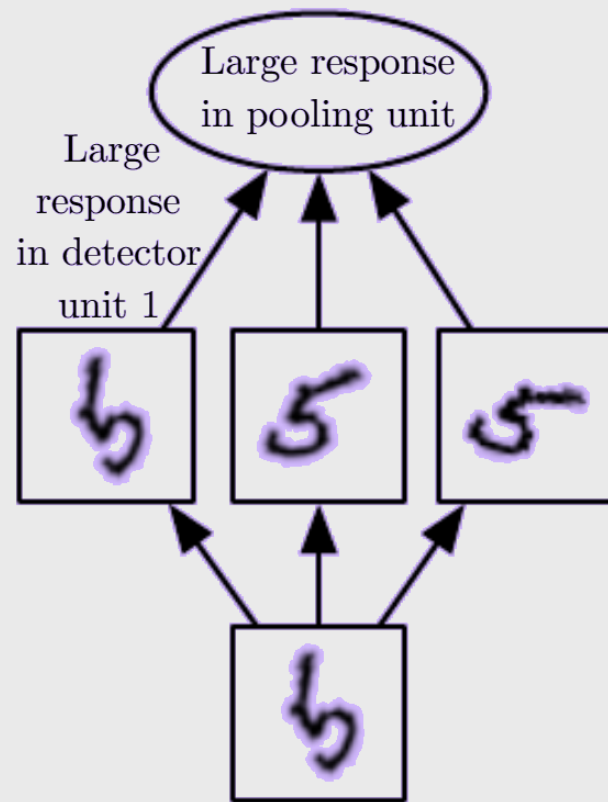
(b) scale/aspect ratio



(c) scale/rotation

Convolutional Nets is Translation invariance

CNNs can recognize a feature even if it appears in different locations in an image or images are rotated, translated, shifted.

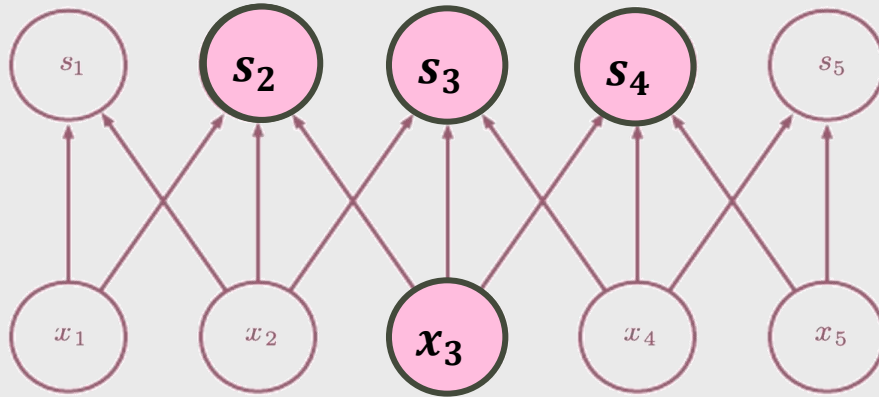


Sparsity

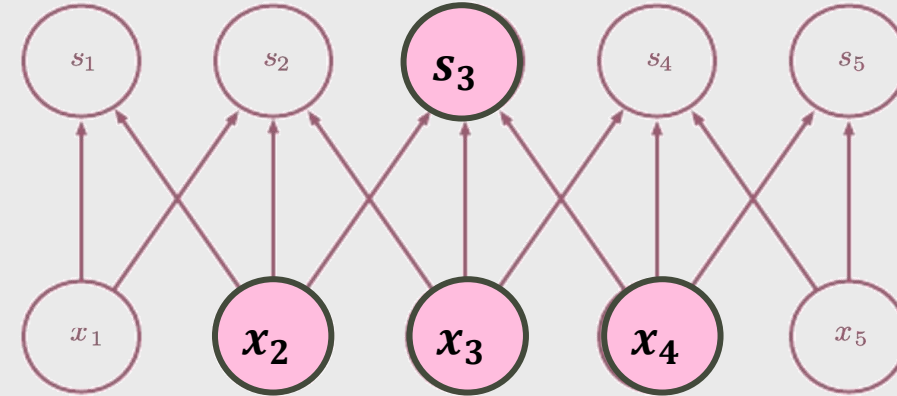
11:31
AM

Only three outputs are affected by x
 S is formed by convolution with a
kernel of width 3

CONVOLUTION

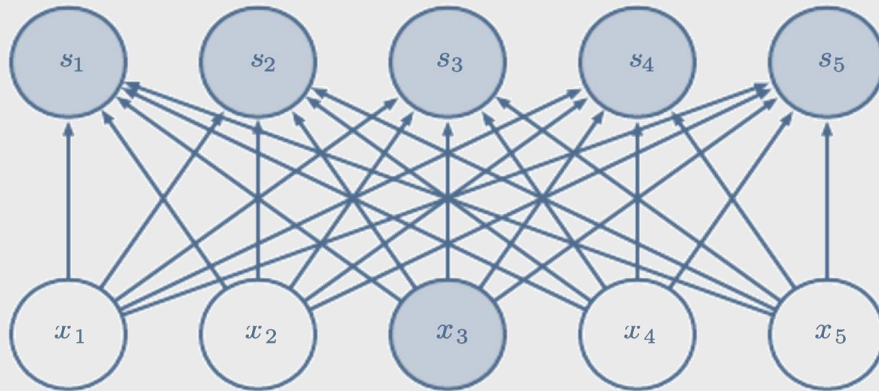


Only three input are affected by x
 S is formed by convolution with a
kernel of width 3

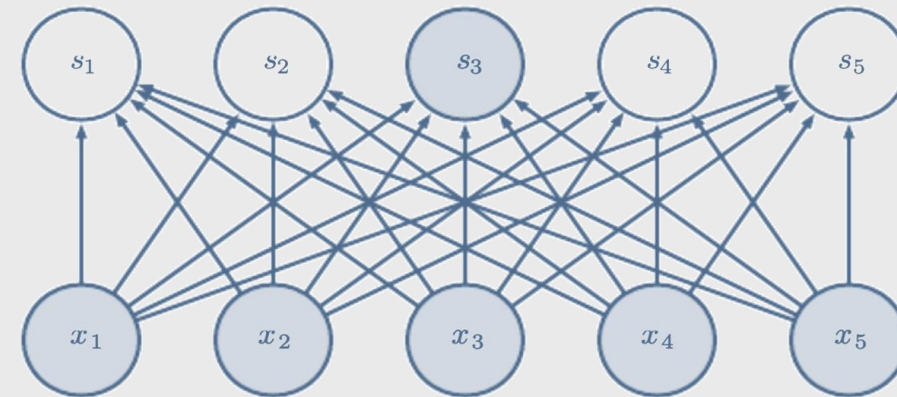


Memory and
Time efficiency

Fully Connected



All the output affect s_3

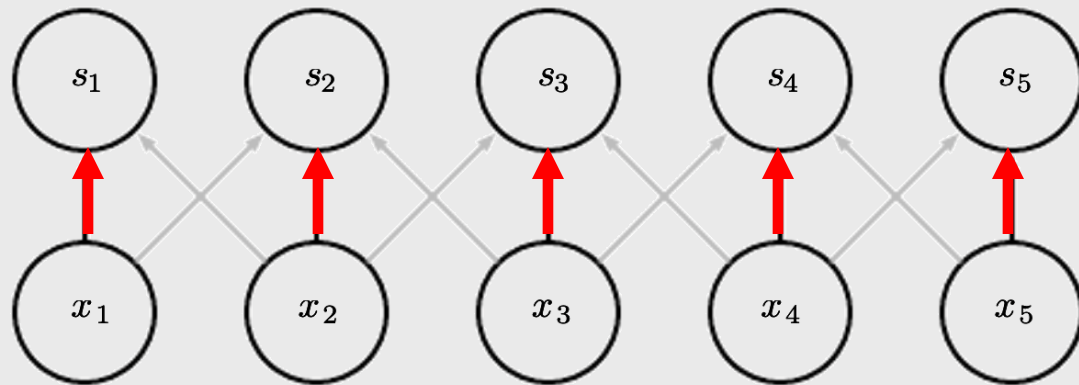


All the inputs affect s_3

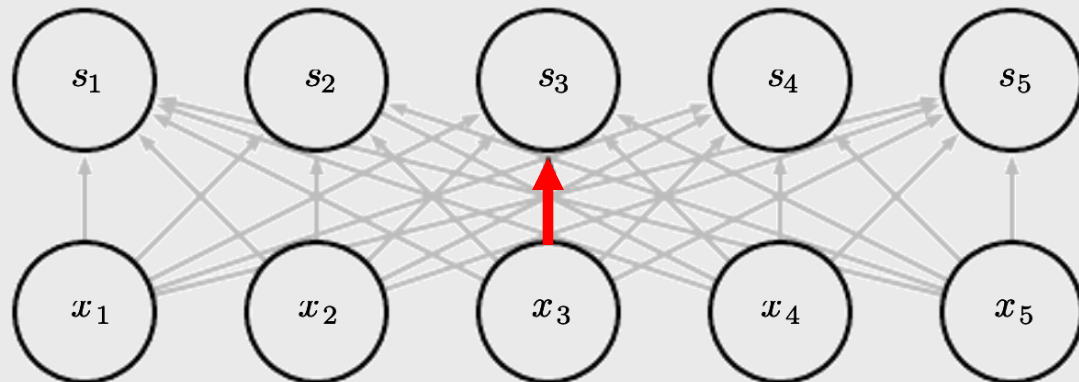
Memory and
Time intensive

Parameter Sharing (↗) and Tied weights (↑)

Parameter sharing refers to using the same parameter for more than one function in a model. In CNNs, each filter (a small weight matrix) has a set of weights (parameters) associated with it, and these weights are shared across all the neurons that use that filter

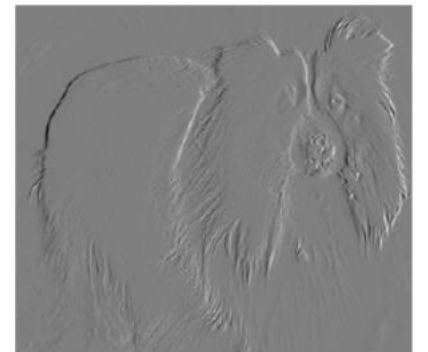


The black arrows indicate uses of the central element of a 3-element kernel in a convolutional model. Because of parameter sharing, this single parameter is used at all input locations



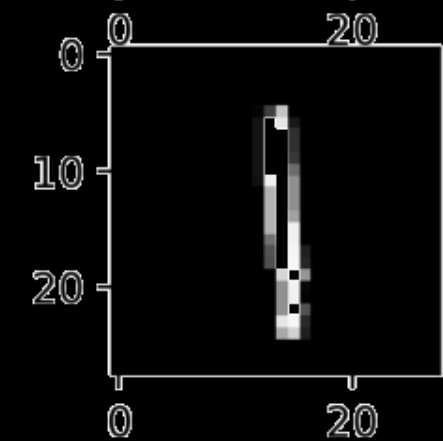
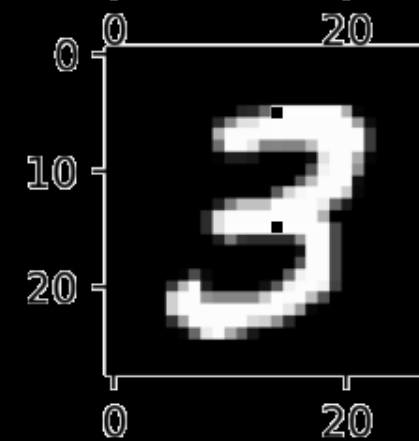
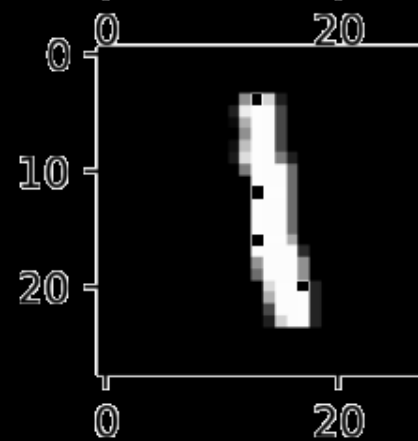
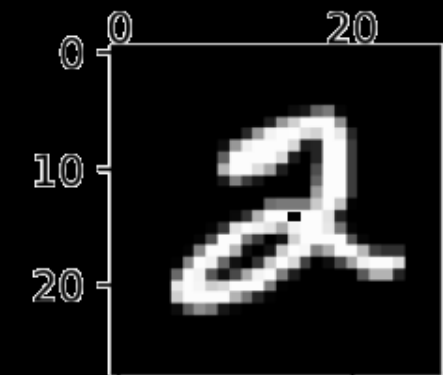
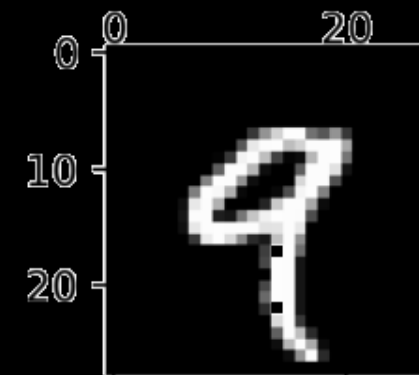
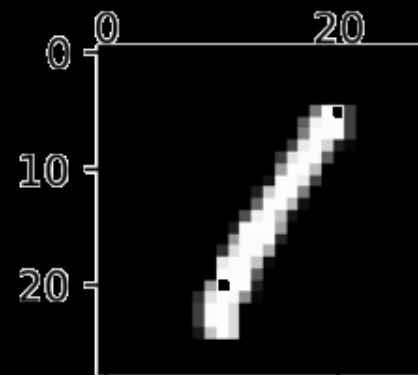
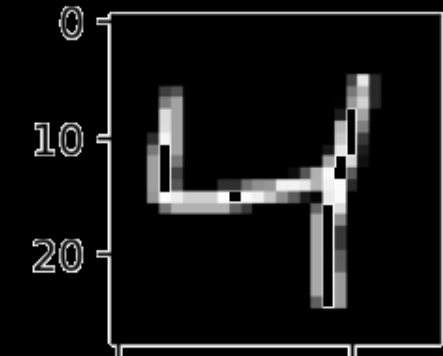
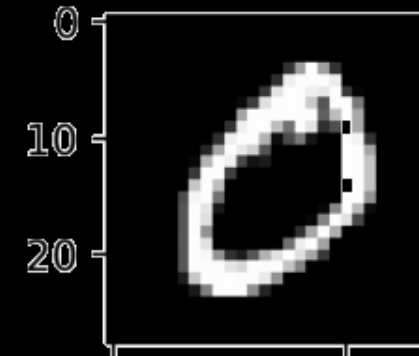
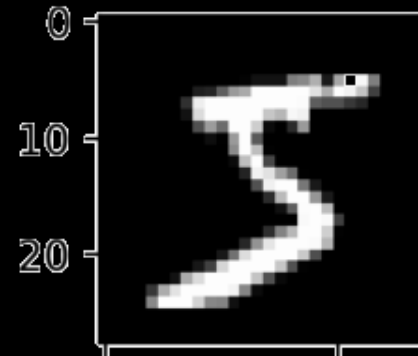
The single black arrow indicates the use of the central element of the weight matrix in a fully connected model. This model has no parameter sharing, so the parameter is used only once

Edge Detection



Computer Vision and Pattern Recognition

Dataset is a set of
labelled images



Communicated by Dana Ballard

Backpropagation Applied to Handwritten Zip Code Recognition

Y. LeCun
B. Boser
J. S. Denker
D. Henderson
R. E. Howard
W. Hubbard
L. D. Jackel

AT&T Bell Laboratories Holmdel, NJ 07733 USA

The ability of learning networks to generalize can be greatly enhanced by providing constraints from the task domain. This paper demonstrates how such constraints can be integrated into a backpropagation network through the architecture of the network. This approach has been successfully applied to the recognition of handwritten zip code digits provided by the U.S. Postal Service. A single network learns the entire recognition operation, going from the normalized image of the character to the final classification.

Neural Computation 1, 541-551 (1989) © 1989 Massachusetts Institute of Technology

80322-4129 80206

40004 14310

37872 05453

3302 75216

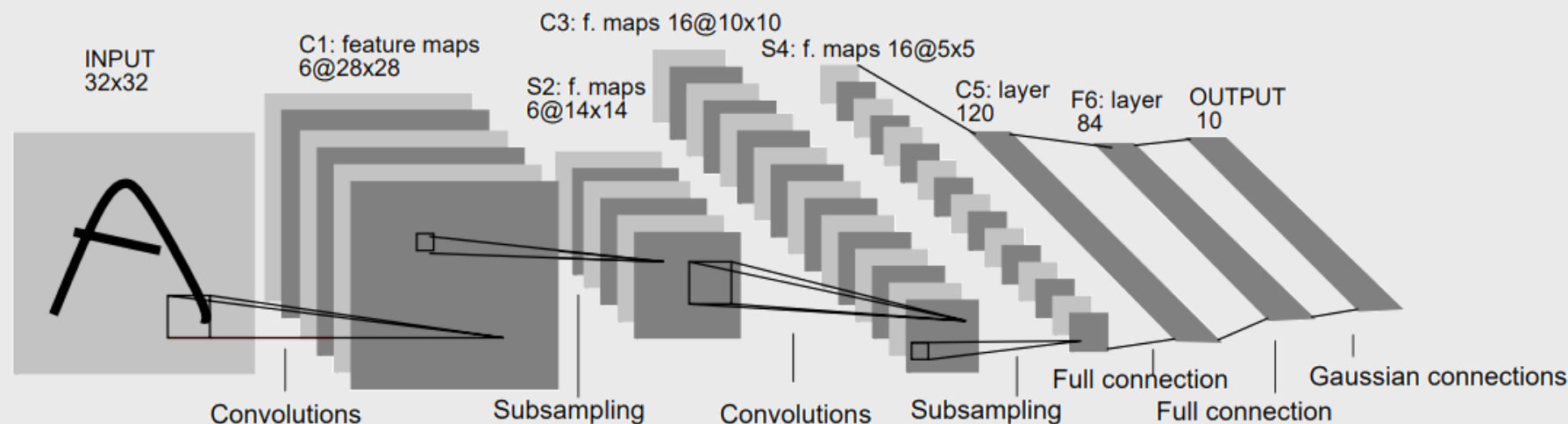
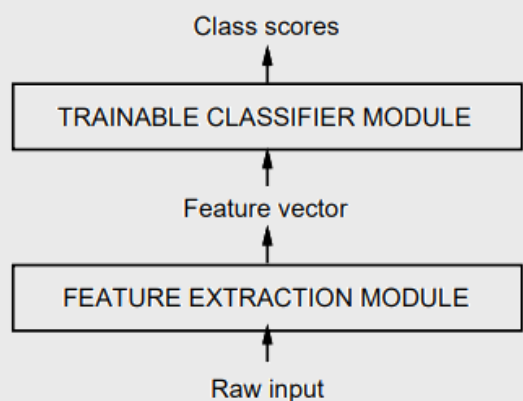
35460 44209

1011915485726803226414186
6359720299299722510046701
3084114591010615406103631
1064111030475262009979966
8912056708557131427955460
6018750187112993089970984
0109707597331972015519055
1075318255182814358090943
1787521655460354603546055
18255108503047520439401

Figure 1 Examples of original zip codes (top) and normalized digits from the testing set (bottom).

Gradient-Based Learning Applied to Document Recognition

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner



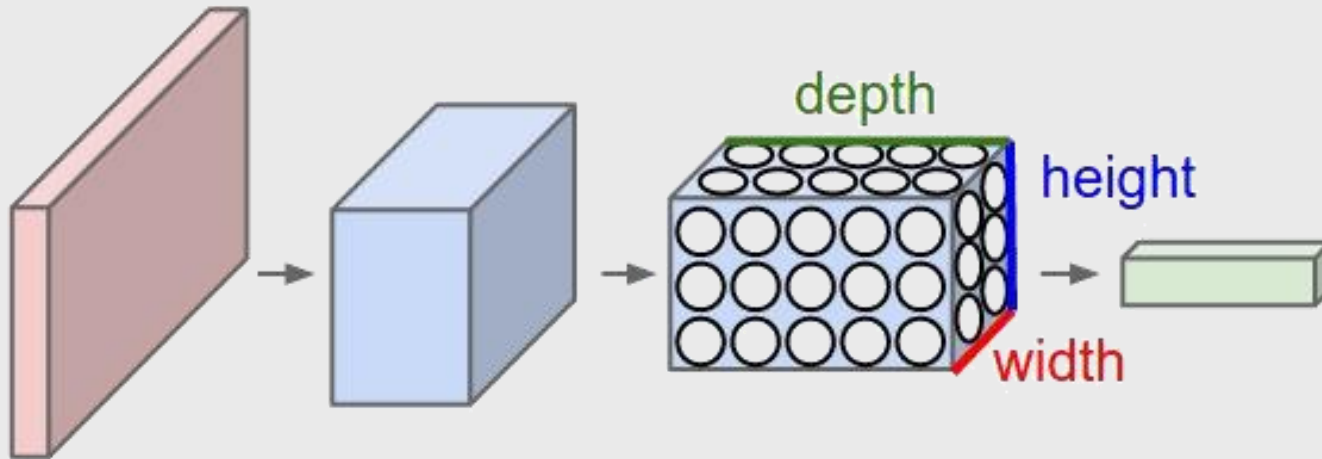
Traditional pattern
recognition.

It requires manual feature
extraction steps

Architecture of **LeNet**
a Convolutional Neural Network

CNN are End-to-End model eliminating need for
manual feature extraction

Convolutional Neural Network (ConvNet)



A **ConvNet** arranges its neurons in three dimensions (**width, height, depth**).

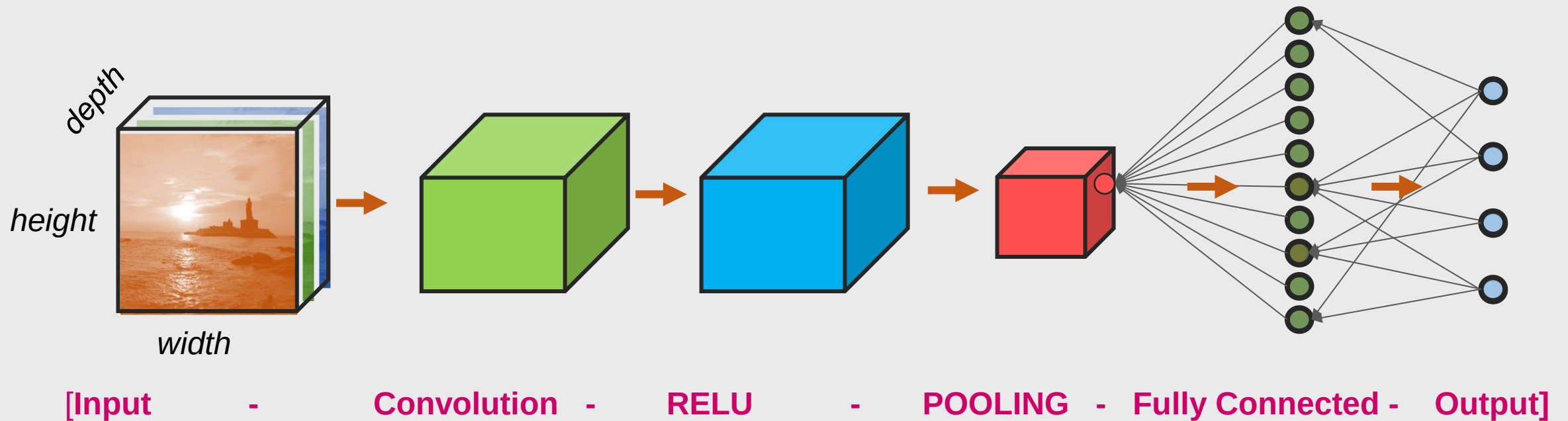
Every layer of a **ConvNet** transforms the 3D input volume to a 3D output volume of neuron activations.

In this example, the red input layer holds the image, so its width and height would be the dimensions of the image, and the depth would be 3 (Red, Green, Blue channels)

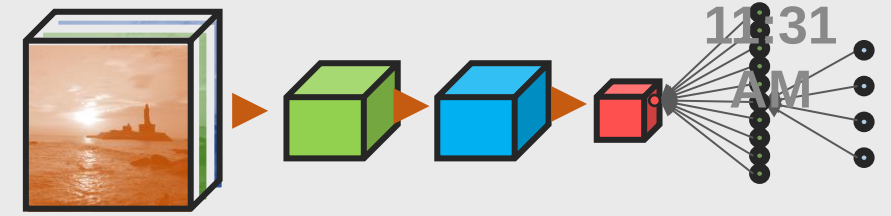
ConvNet/ CNN

Architecture: A Simple ConvNet / CNN

[INPUT - CONV - RELU - POOL - FC]



ConvNet/ CNN

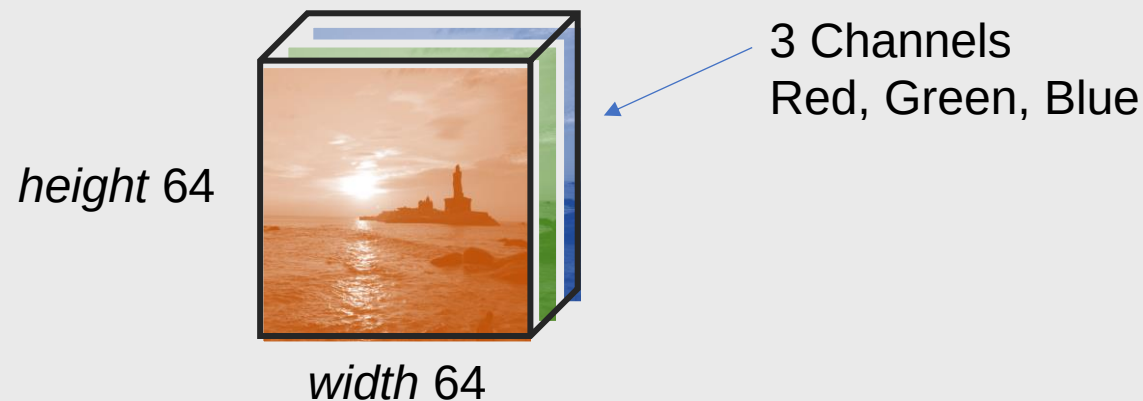


Architecture: A Simple ConvNet / CNN

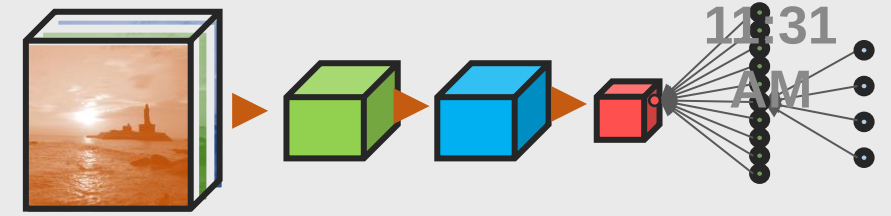
[**INPUT** - CONV - RELU - POOL - FC]

INPUT [64x64x3] holds the raw pixel values of the image.

Image *width* 64, *height* 64, and with *three* colour channels R,G,B.



ConvNet/ CNN



Architecture: A Simple ConvNet / CNN

[INPUT - CONV - RELU - POOL - FC]

CONV layer computes the output of neurons that are connected to local regions in the input, each computing a Hadamard product between their weights and a small region they are connected to in the input volume.

E.g. The Convolution of INPUT [64x64x3] may result in volume [32x32x12] if we decided to use 12 filters

ConvNet (read)

Convolution Layer

- CONV layer's parameters consist of a set of **learnable filters**.
- Every filter is small spatially (along width and height) but extends through the full depth of the input volume.
- A typical filter on a first layer of a ConvNet might have size **5x5x3** (i.e. 5 pixels width and height, and 3 because images have depth 3, the colour channels)

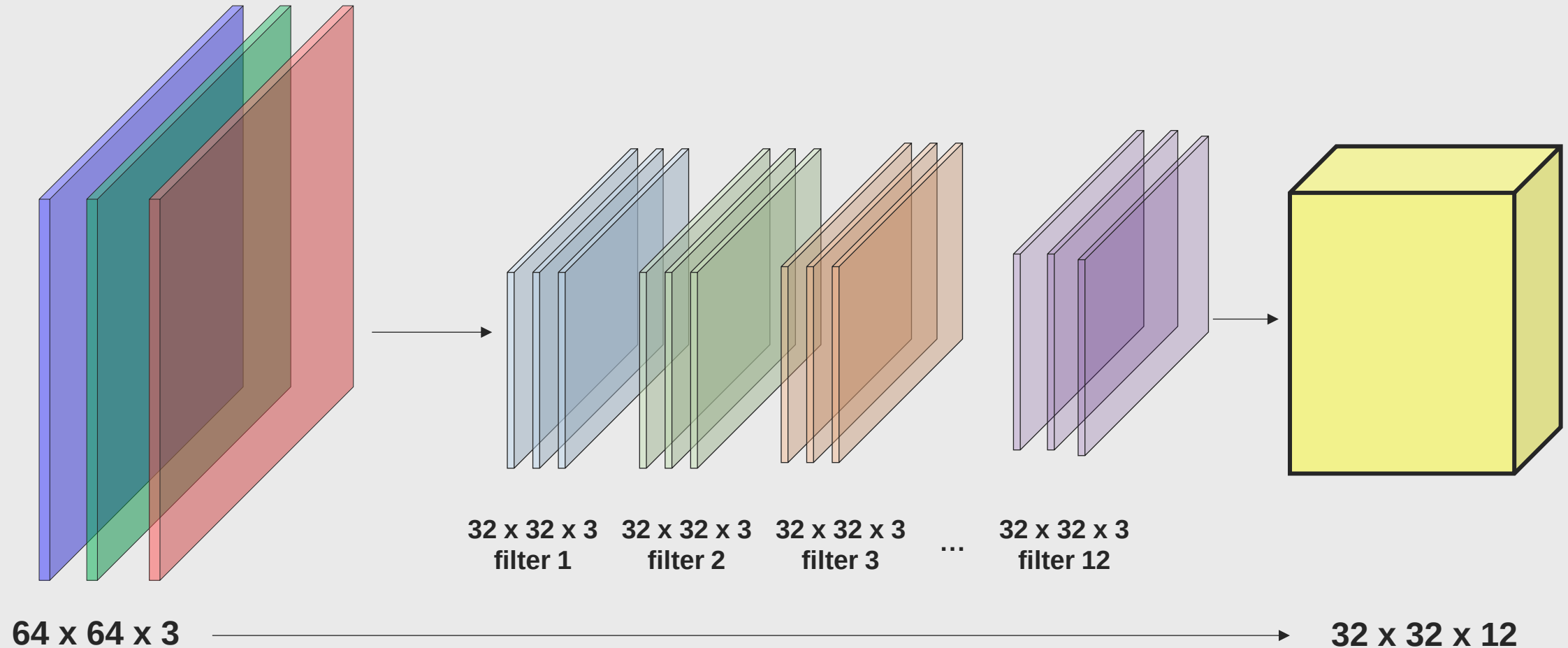
ConvNet (read)

Convolution Layer

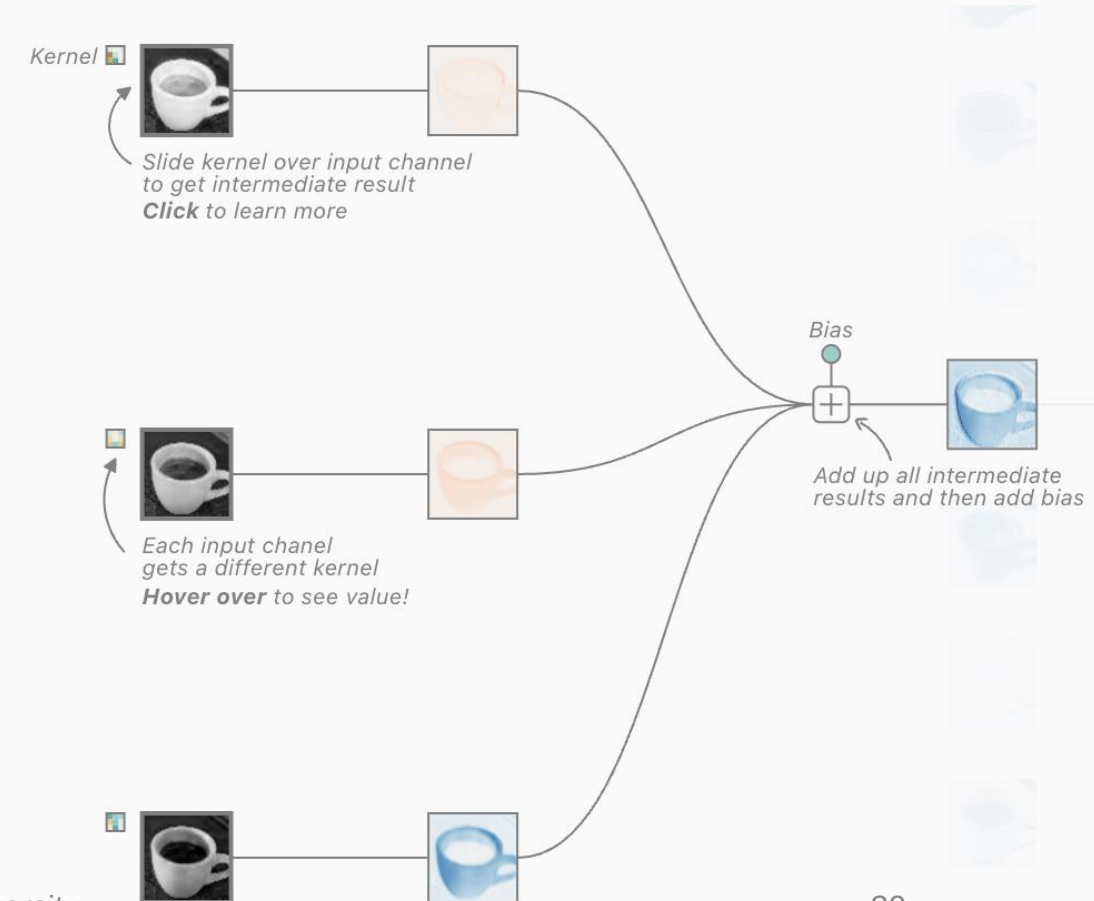
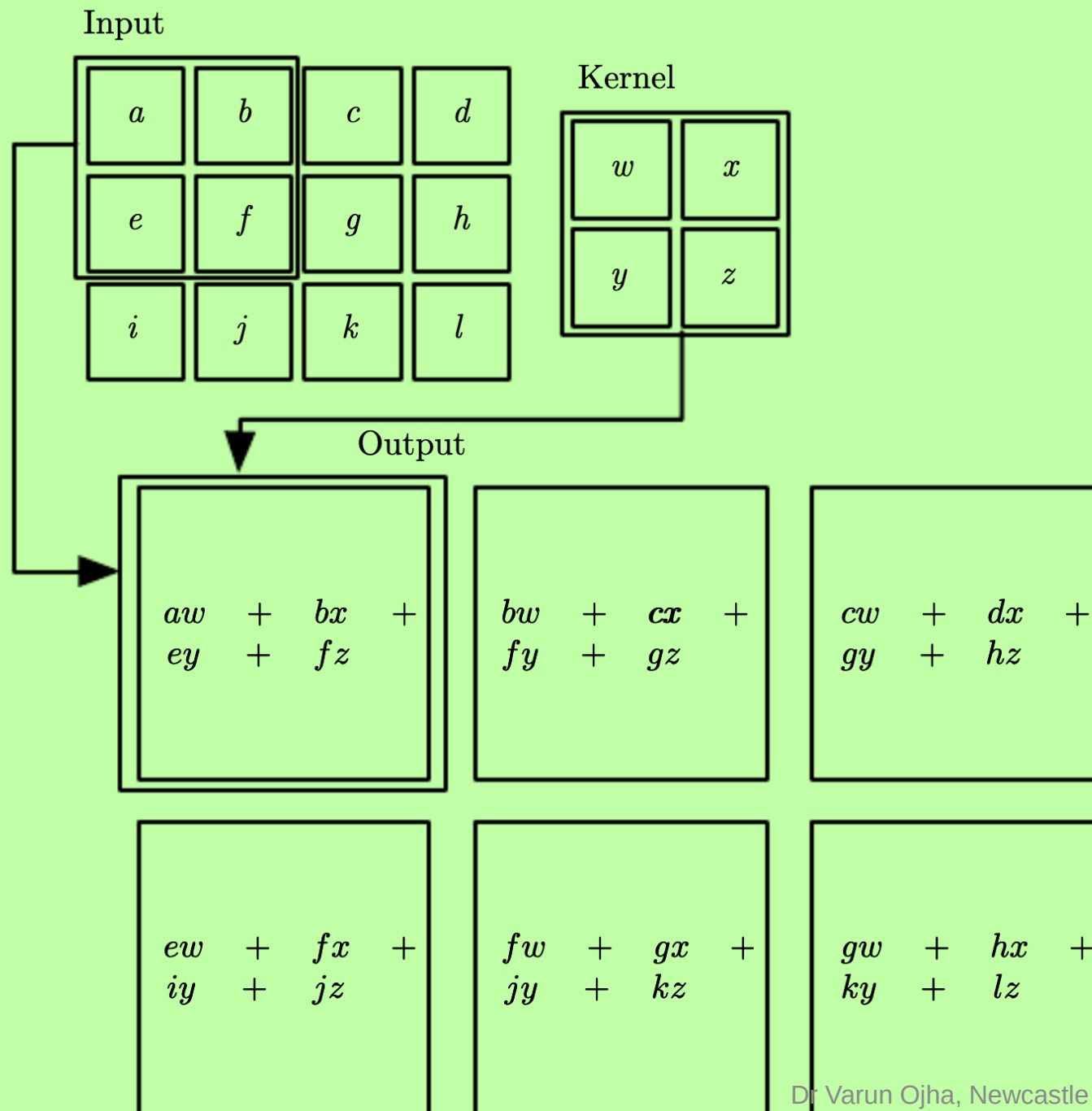
- Forward pass: we slide (**convolve**) each filter across the width and height of the input volume and compute dot products between the entries of the filter and the input at any position.
- When we slide the **filter** over the width and height of the input volume, we will produce a **2-dimensional activation map** that gives the responses of that filter at every spatial position
- We can have a set of filters (e.g., 12)

ConvNet

Convolution Layer



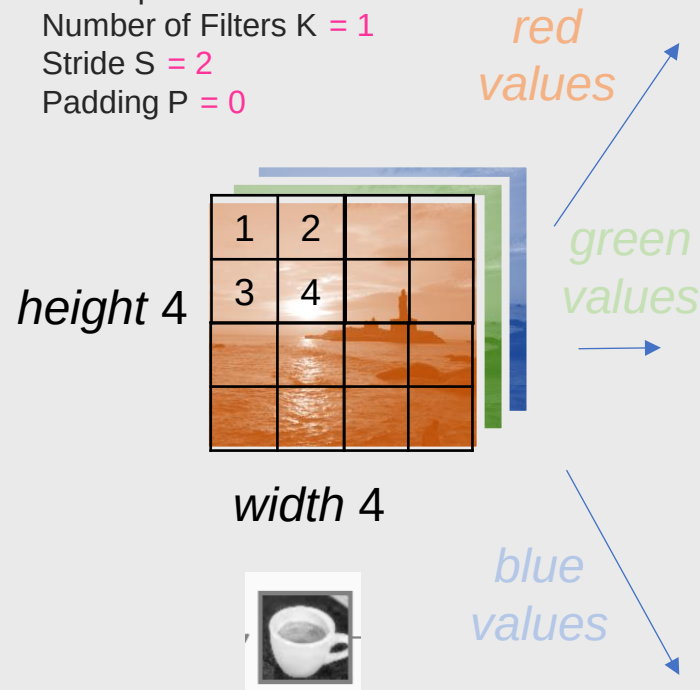
ConvNet Convolution Layer



Simple Convolution Example

Input Volume and Convolution

Image Width $W_1 = 4$
 Image Height $H_1 = 4$
 Image Depth $D_1 = 3$
 Filter spatial dimension $F = 2$
 Number of Filters $K = 1$
 Stride $S = 2$
 Padding $P = 0$



receptive field

1	2
3	4

image x_r

2	1
1	2

image x_g

2	5
4	3

image x_b

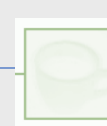
convolutional filter w_0

1	0
0	1

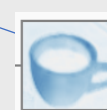
$x_r w$



$x_g w$



$x_b w$



Weighted input
Hadamard product

$1*1$ =1	$2*0$ =0
$3*0$ =0	$4*1$ =4

Output Volume

$$W_2 = (W_1 - F + 2P) / S + 1 = 2$$

$$H_2 = (H_1 - F + 2P) / S + 1 = 2$$

$$D_2 = K = 1$$

Convolved image

$$\sum x w + b$$

18	



5
 $\sum x_r w$

3
 $\sum x_g w$

9
 $\sum x_b w$

1 b bias

Note that each 2D map performs element-wise multiplication instead a dot product as may be advised in most text. **However**, if you flatten the tensors (i.e., make them vectors), you can do a dot product. Note that is only 1 filter but we may have many filters.

ConvNet Convolution Layer

Input volume of size $W_1 \times H_1 \times D_1$

Requires four hyperparameters:

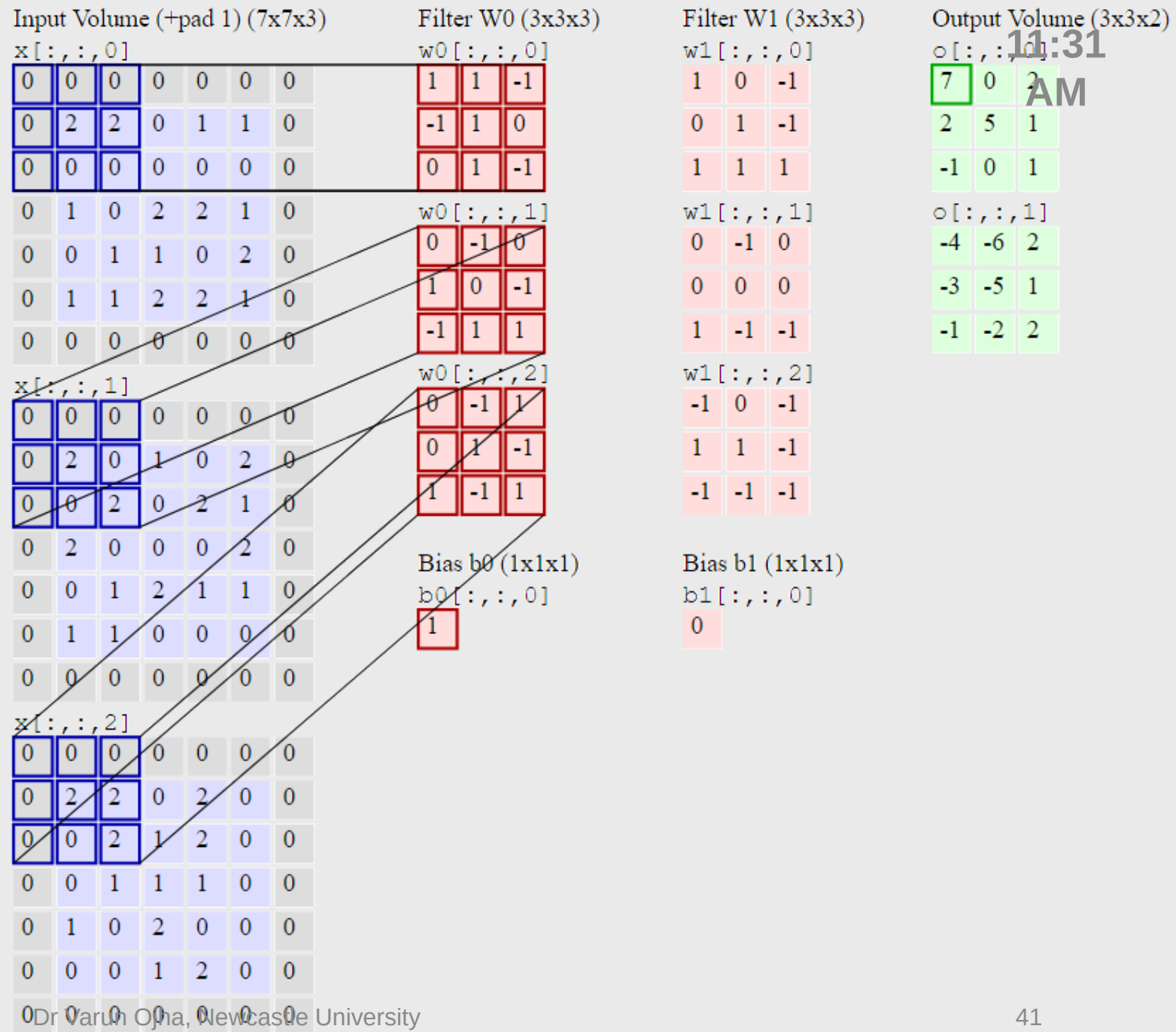
- Number of filters **K**,
- their spatial extent **F**,
- the stride **S**,
- the amount of zero padding **P**.

Output volume of size $W_2 \times H_2 \times D_2$

where:

- $W_2 = (W_1 - F + 2P)/S + 1$
- $H_2 = (H_1 - F + 2P)/S + 1$
- (i.e. width and height are computed equally by symmetry)
- $D_2 = K$

Source: <http://cs231n.github.io/convolutional-networks/>



11:31 AM

Requires four hyperparameters:

- Number of filters **K = 2**,
- their spatial extent **F = 3**,
- the stride **S = 2**,
- the amount of zero padding **P = 1**.

where:

- $W_2 = (5 - 3 + 2 \cdot 1) / 2 + 1$
- $H_2 = (5 - 3 + 2 \cdot 1) / 2 + 1$
- (i.e. width and height are computed equally by symmetry)
- $D_2 = 2$



ConvNet Convolution Layer

Input volume of size $5 \times 5 \times 3$

Requires four hyperparameters:

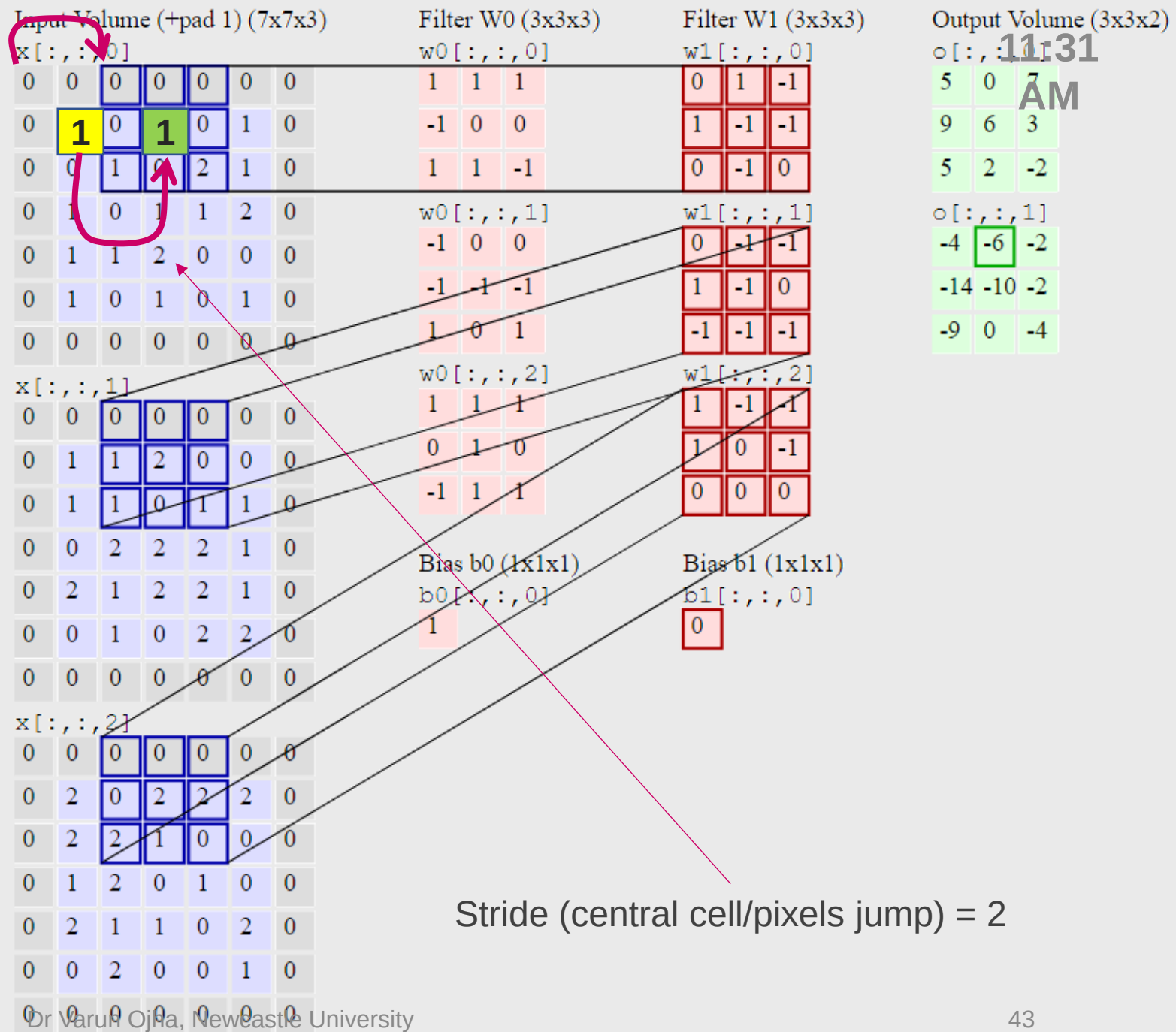
- Number of filters $K = 2$,
- their spatial extent $F = 3$,
- the stride $S = 2$, (moved 2 pixels)
- the amount of zero padding $P = 1$.

Output volume of size $W_2 \times H_2 \times D_2$

where:

- $W_2 = 3$
- $H_2 = 3$
- (i.e. width and height are computed equally by symmetry)
- $D_2 = 2$

Source: <http://cs231n.github.io/convolutional-networks/>



Effect of Learned Convolutional Filter on Images^{AM}

This is most likely scenario as we do not know exactly what will be the filter weights after training. Hence CNN is a Blackbox

These weights/values of 3x3 kernels/filters are learned during training
(filter values and outputs are mere representative)

Identity	Sharpen	Blur	Laplacian	Gaussian
0	0	1/5	0	1/8
0	-1	1/5	1	2/8
0	7	1/5	-5	4/8
0	-1	1/5	1	2/8
0	0	1/5	0	1/8



Original



Identity



Sharpen



Blur

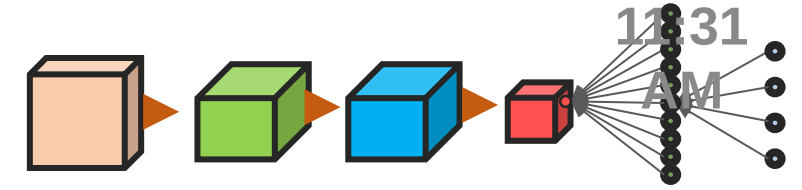


Laplacian



Gaussian

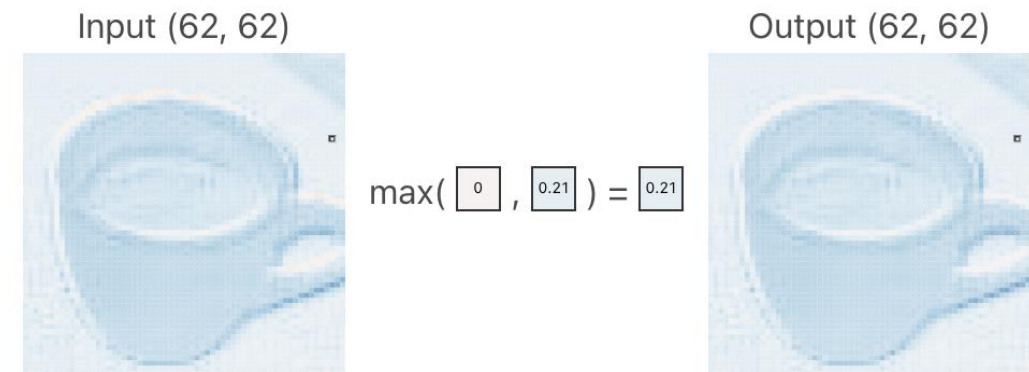
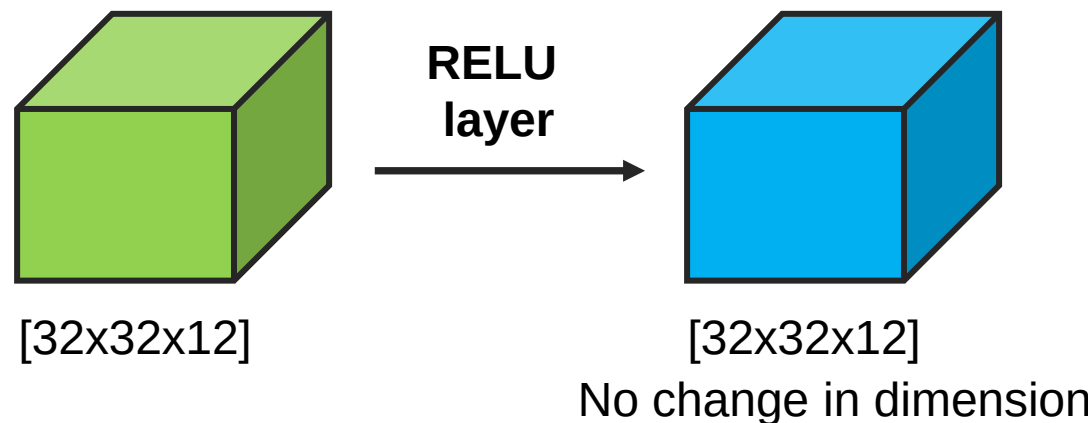
ConvNet/ CNN



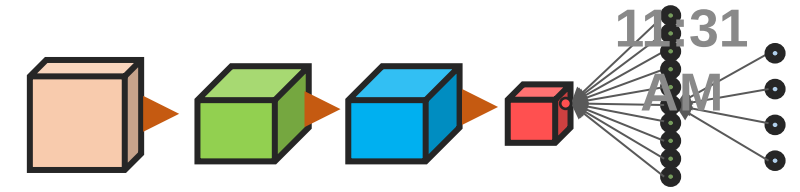
Architecture: A Simple ConvNet / CNN

[INPUT - CONV - **RELU** - POOL - FC]

RELU layer will apply an elementwise activation function, such as the $\max(0, x)$ thresholding at zero. This leaves the size of the volume unchanged ($[32 \times 32 \times 12]$).



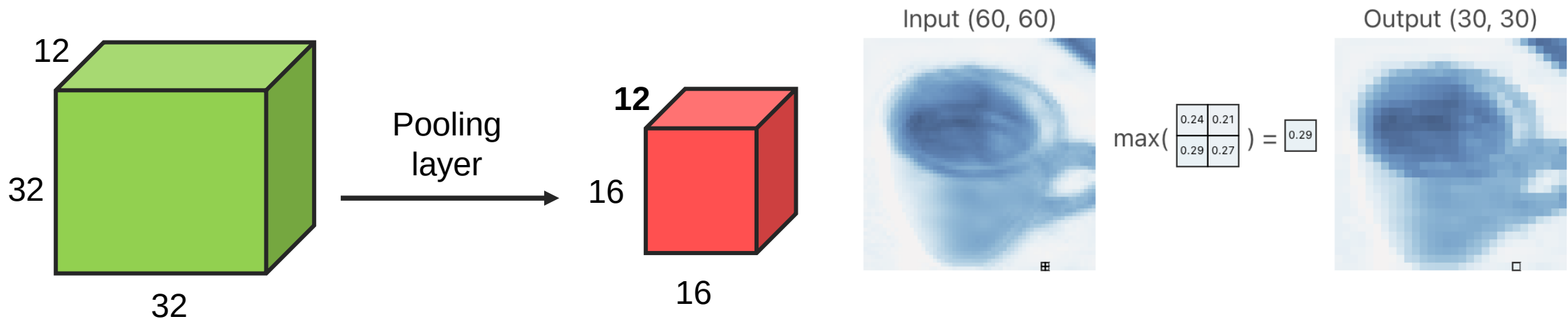
ConvNet/ CNN



Architecture: A Simple ConvNet / CNN

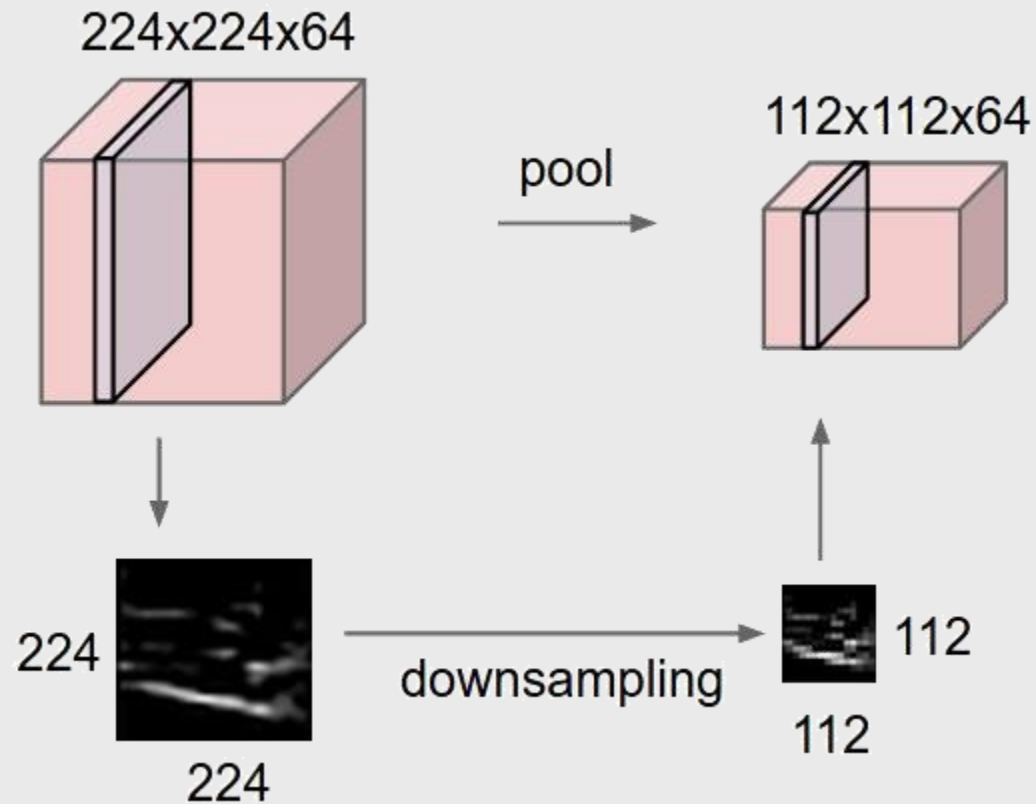
[INPUT - CONV - RELU - **POOL** - FC]

POOL layer will perform a **down sampling** operation along the spatial dimensions (width, height), resulting in volume such as [16x16x12]



ConvNet

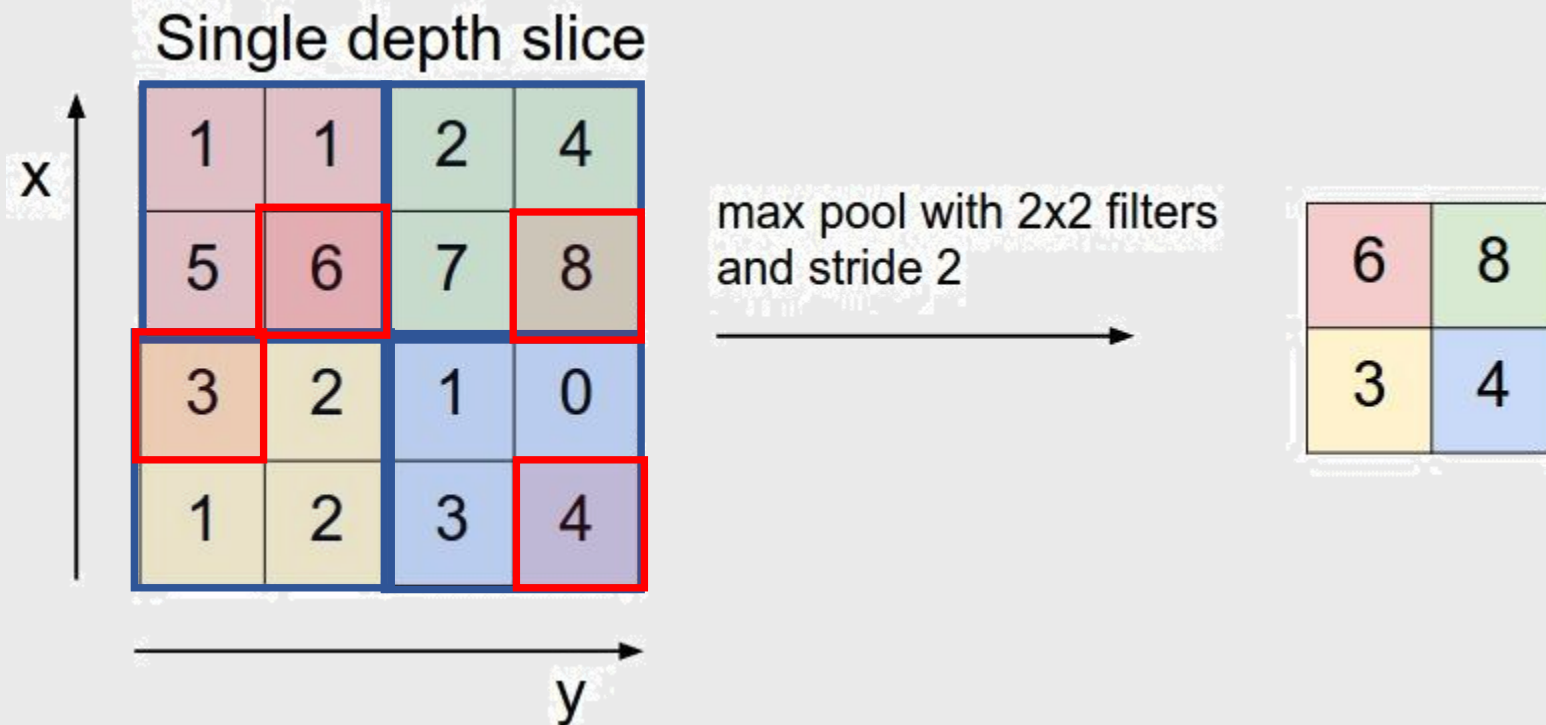
Pooling Layer



Pooling layer **down samples** the volume spatially, independently in each depth slice of the input volume

ConvNet

Pooling Layer



Max Pooling
layer

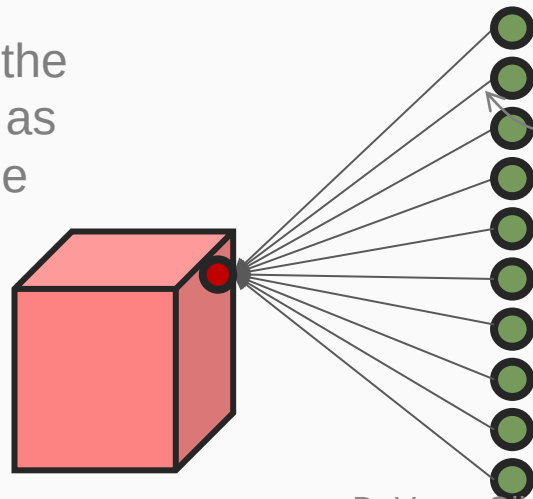
ConvNet/ CNN

Architecture: ConvNet / CNN

[INPUT - CONV - RELU - POOL - FC]

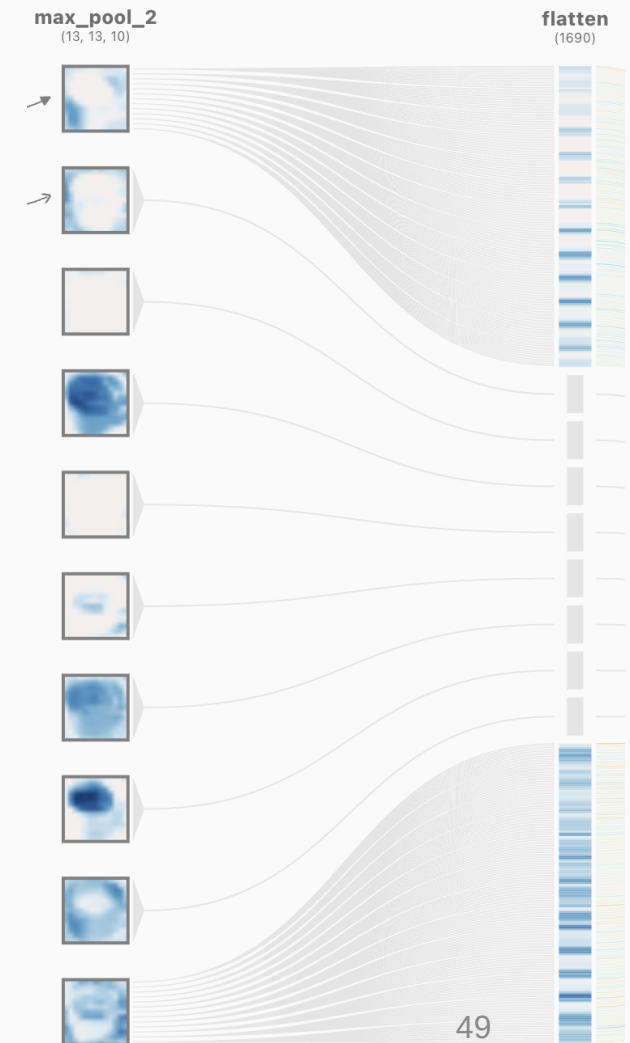
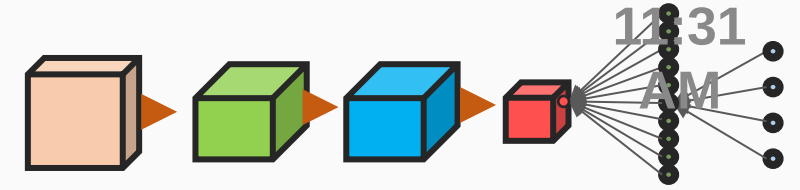
FC (i.e. fully-connected) layer may compute the class scores, resulting in volume of size $[1 \times 1 \times 10]$, where each of the 10 neurone correspond to a class score, such as among the 10 categories.

A single neuron of the volume. There are as many neuron as the volume size



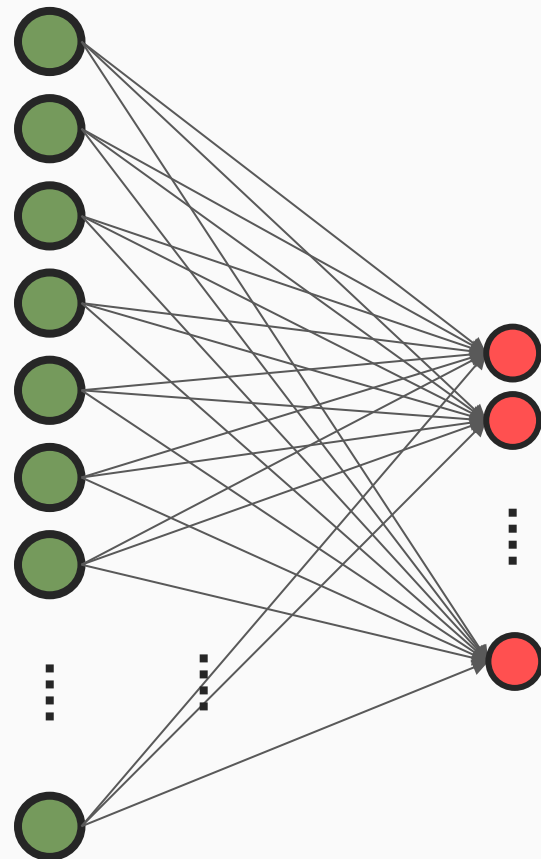
An FC layer is also a linear layer. It can have as many neurons/nodes as a user defines them to be

Dr Varun Ojha, Newcastle University



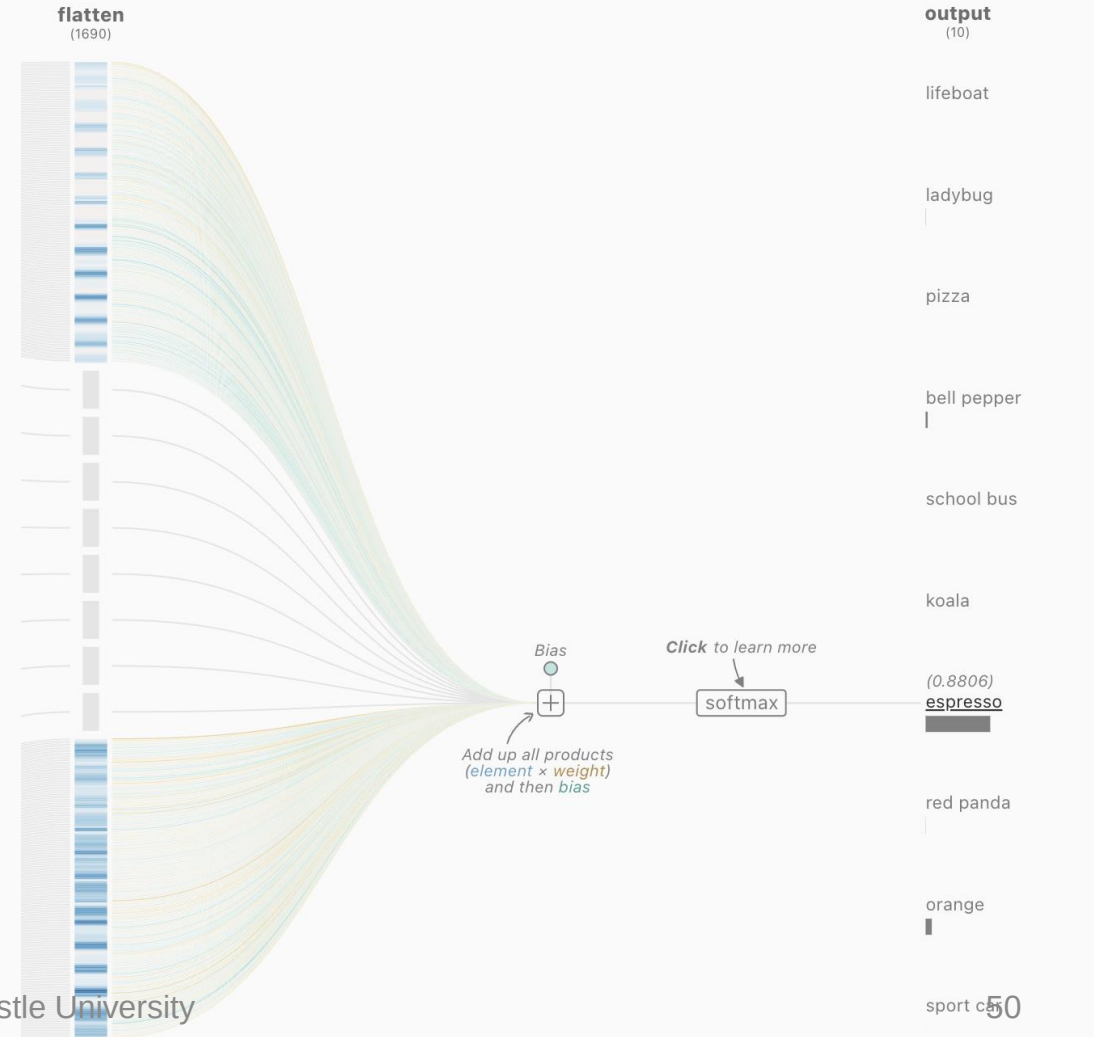
ConvNet

Fully Connected (Dense) Layer



Classes
(e.g. 10 for
MNIST
dataset)

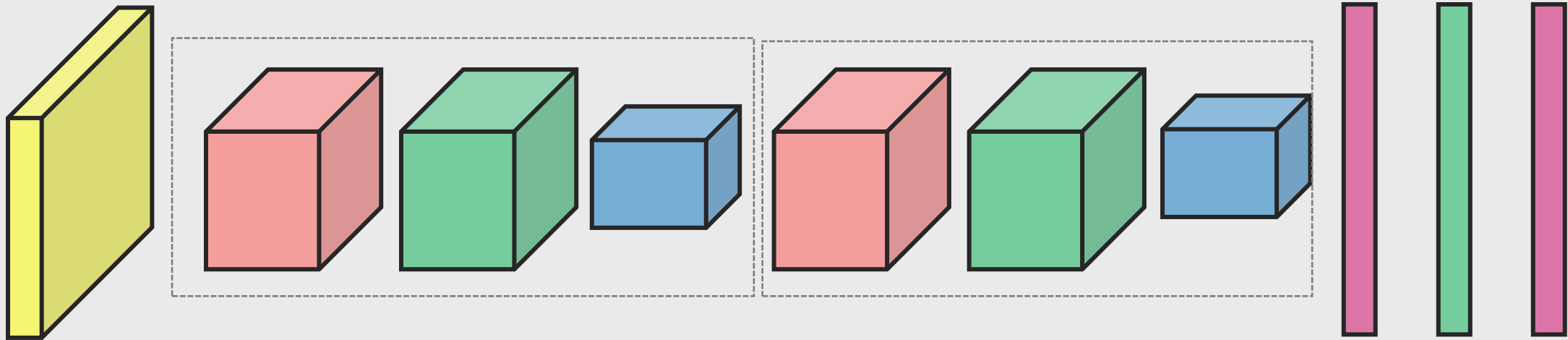
This
typically
outputs
SoftMax



ConvNet

ConvNet Architecture

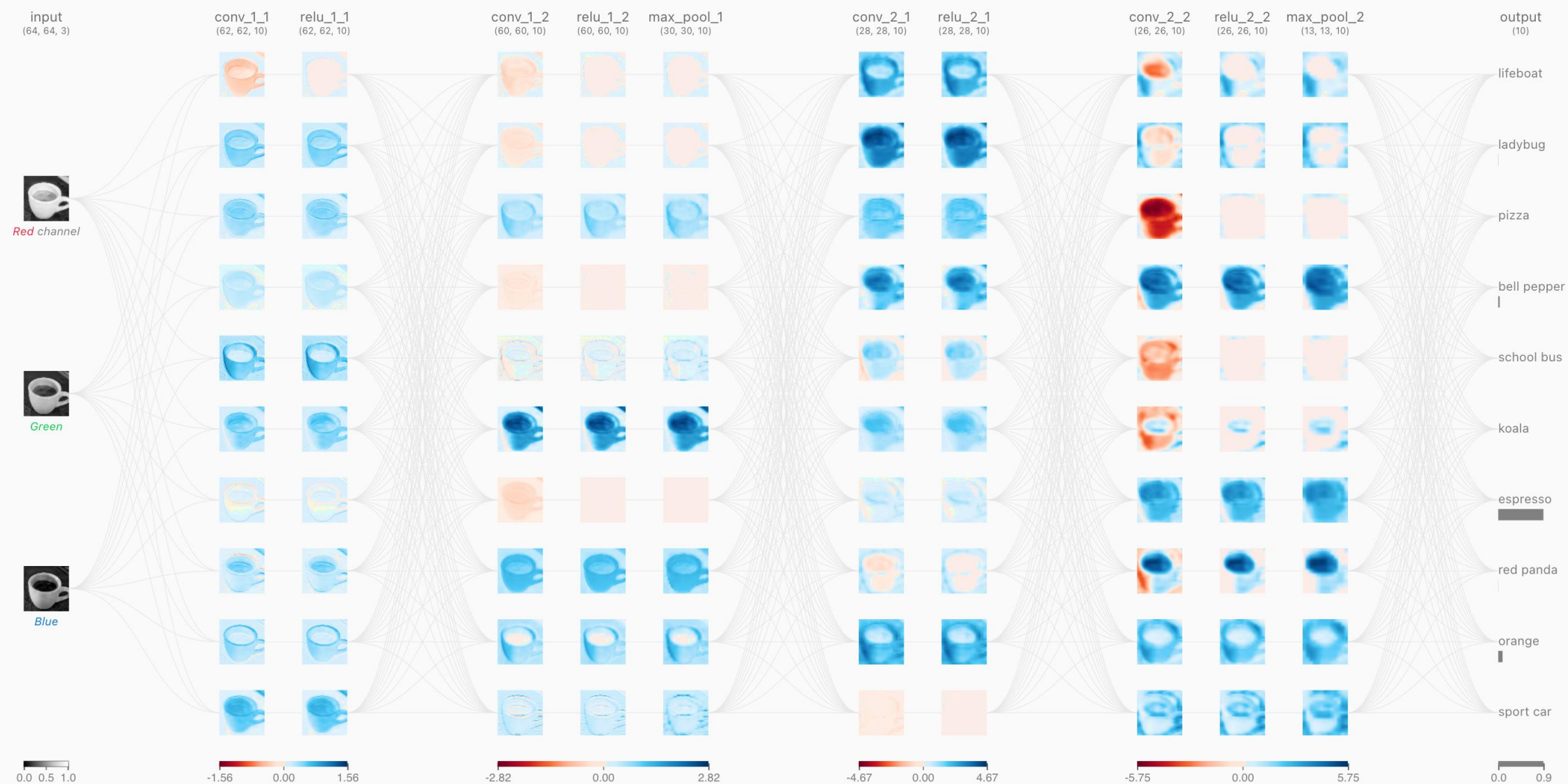
INPUT $\rightarrow$ [CONV $\rightarrow$ RELU $\rightarrow$ POOL] * 2 $\rightarrow$ FC $\rightarrow$ RELU $\rightarrow$ FC]



A very good source: <http://cs231n.github.io/convolutional-networks/>

Visualizing CNN

11:31
AM



Try yourself: <https://poloclub.github.io/cnn-explainer/>

Dr Varun Ojha, Newcastle University

ConvNet on Image Classification

Live demo <http://cs231n.stanford.edu/>

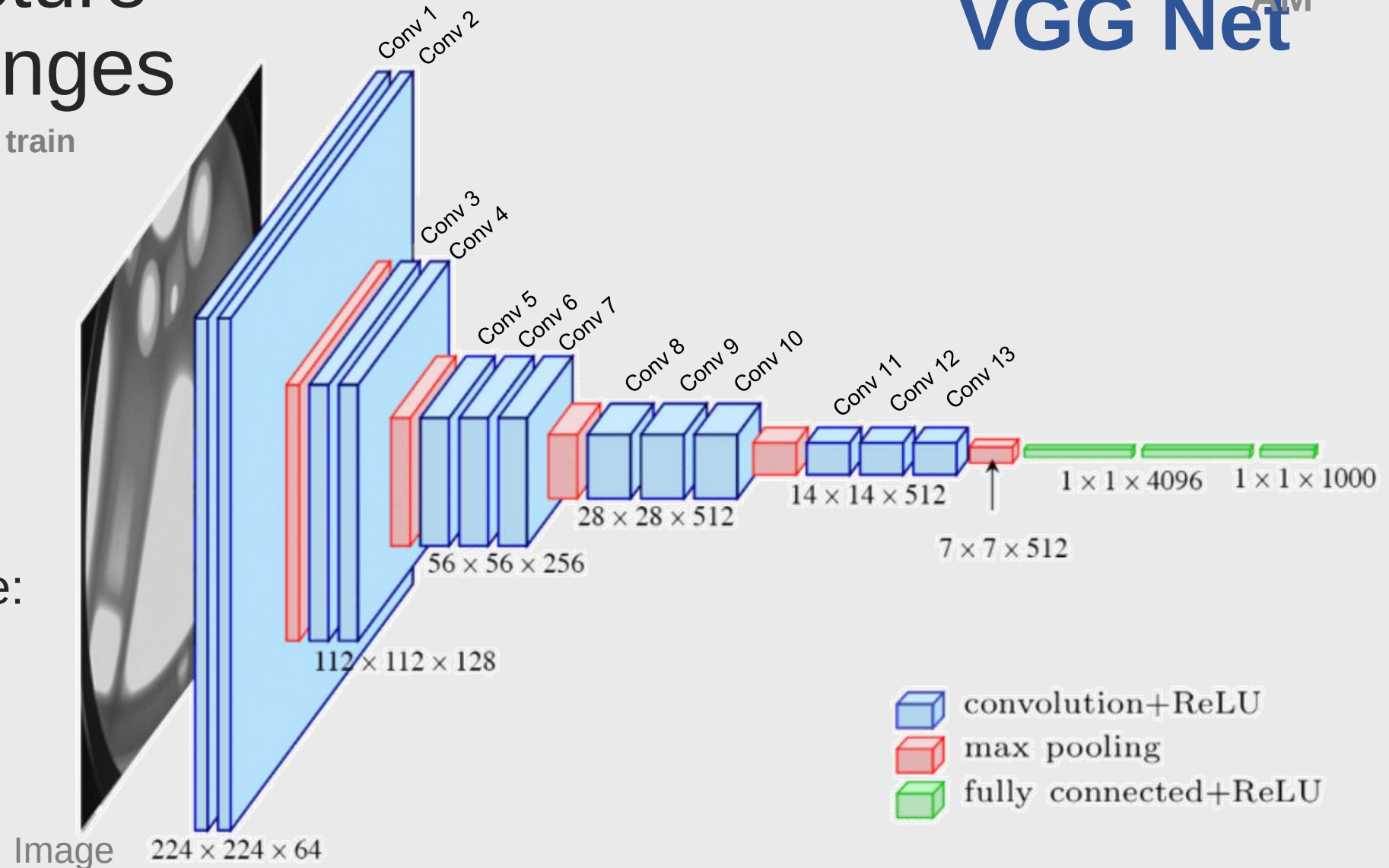


A very good source:
<http://cs231n.github.io/convolutional-networks/>

CNN architecture design challenges

Deeper the network, harder to train

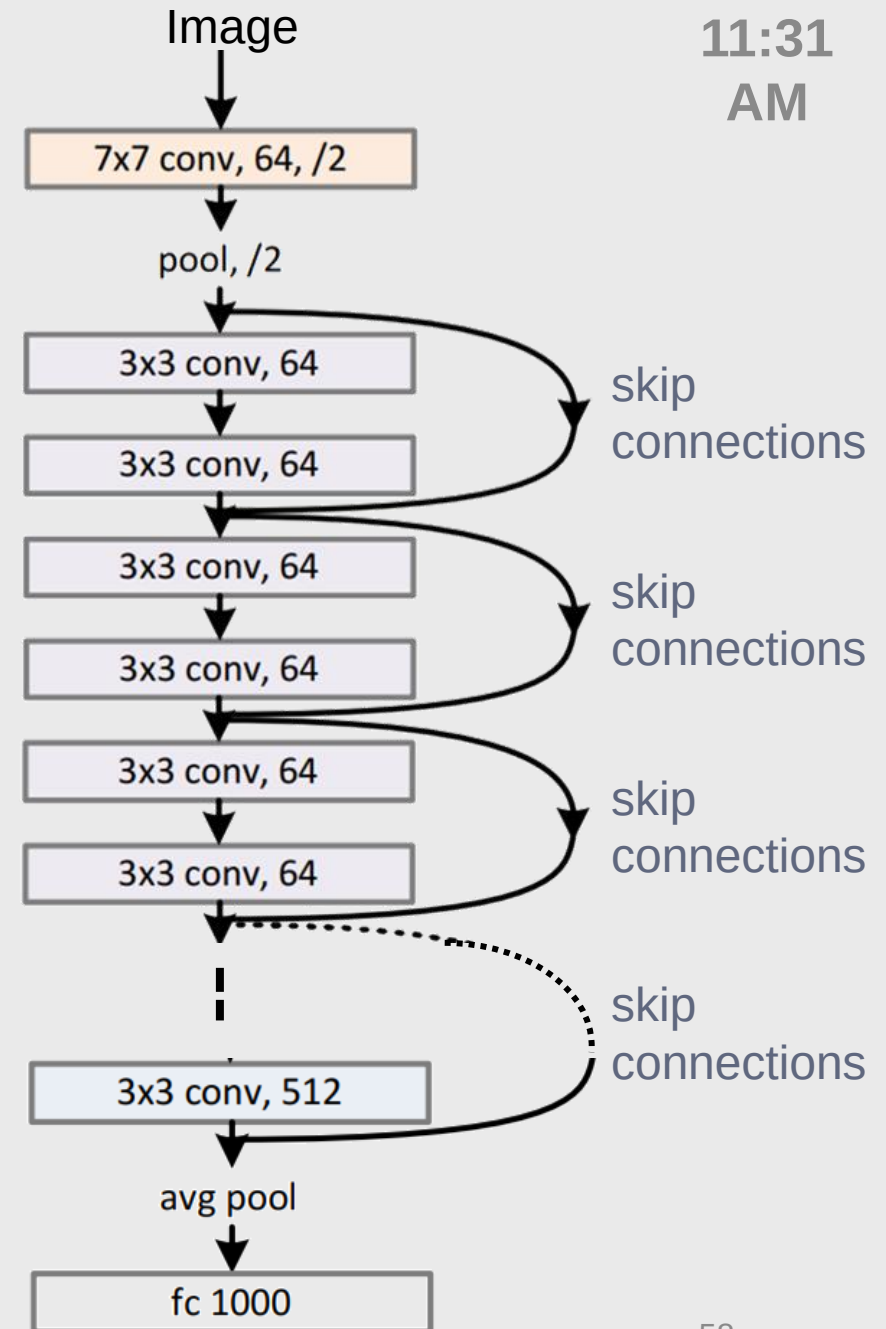
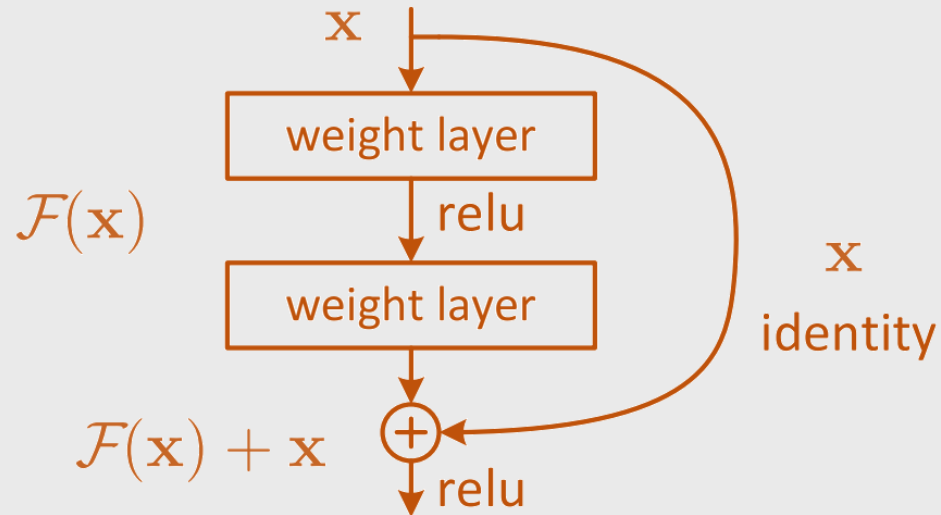
- Visual Geometry Group (VGG) Network (VGG Net)
- VGG 16 - an example architecture: 13 Convolution layers 3 fc layer.



Residual Network (ResNet)

It solves **vanishing gradient problem**. Skip connections allow you to take the activation value from an earlier layer and pass it to a much deeper layer in a network. This allows for smoother gradient flow, ensuring important features are preserved in the training process.

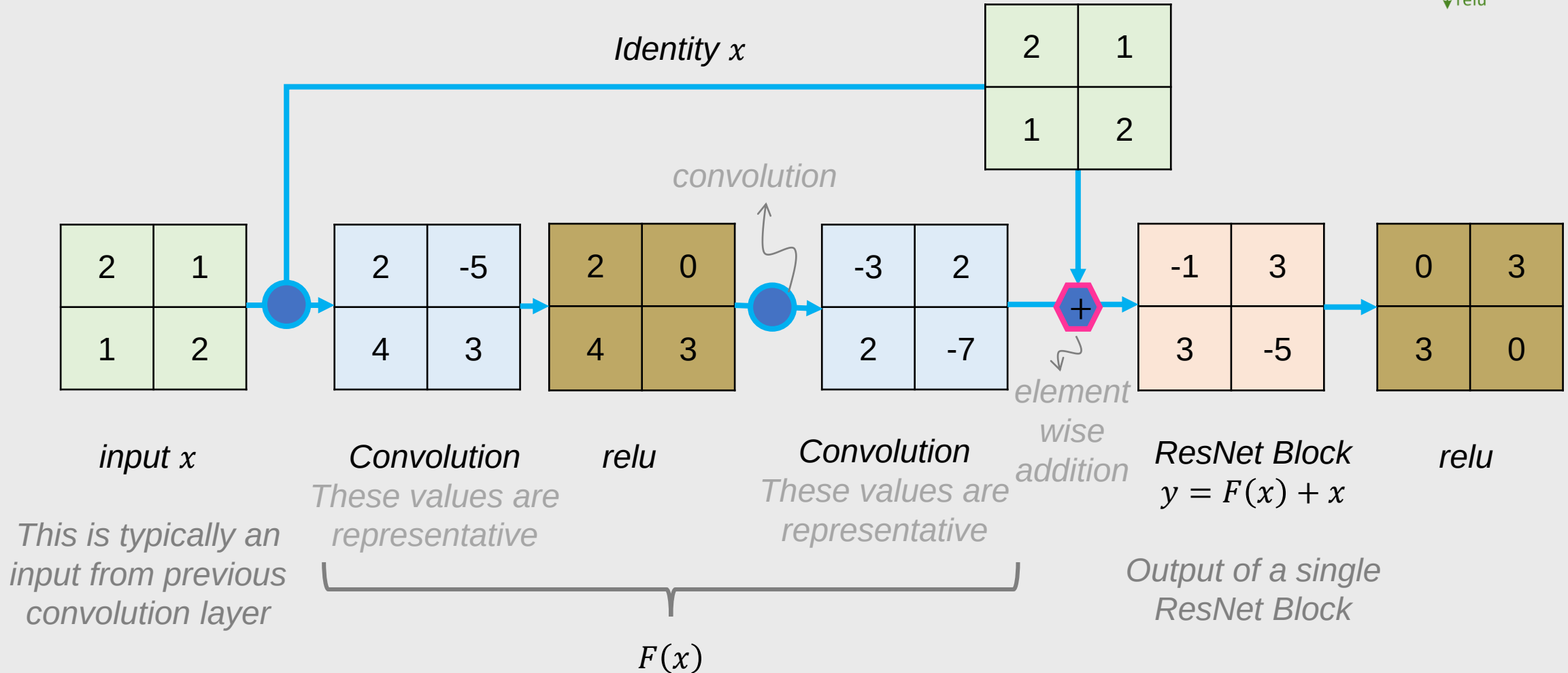
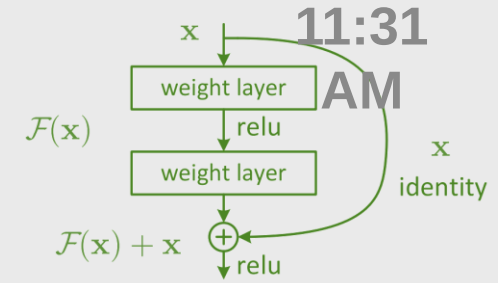
ResNet Block



He et al (2016) Deep Residual Learning for Image Recognition, CVPR

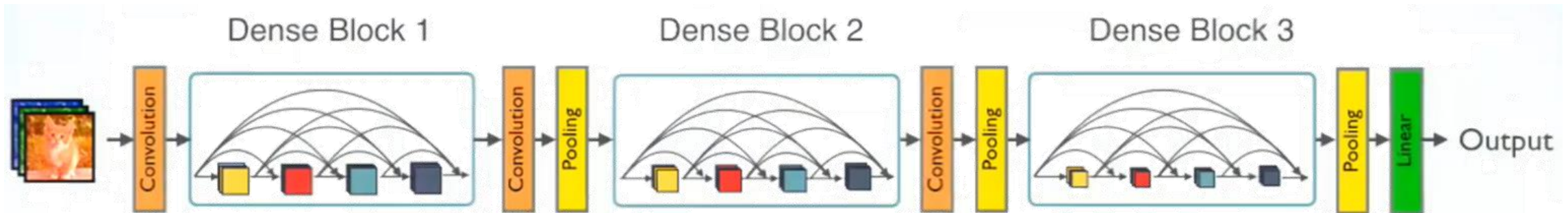
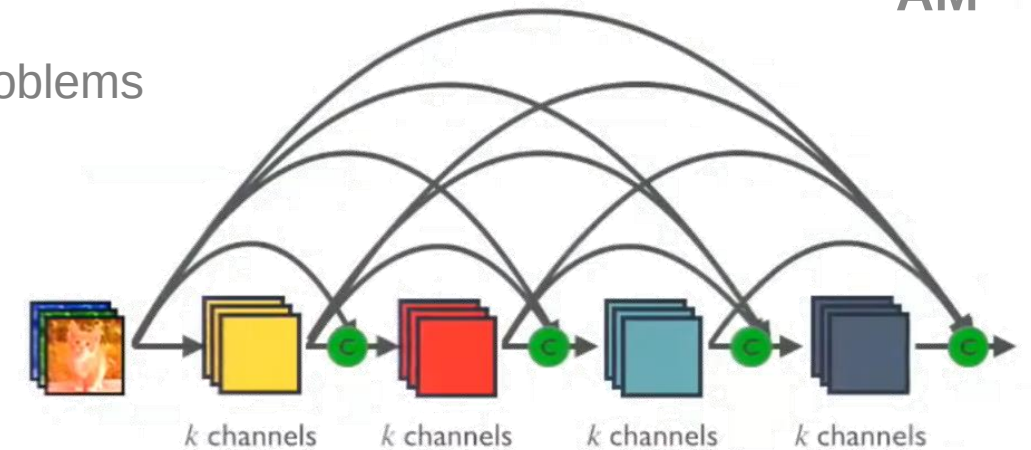
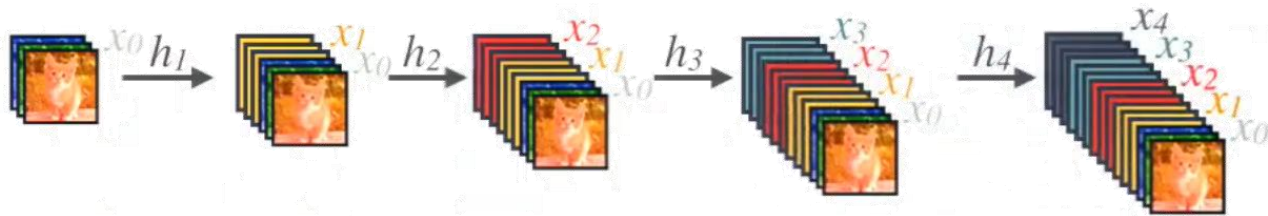
Simple Example of ResNet Block

element-wise addition to pass the output to the next layer or block



Dense Net

Dense Connection to solve vanishing or exploding gradient problems



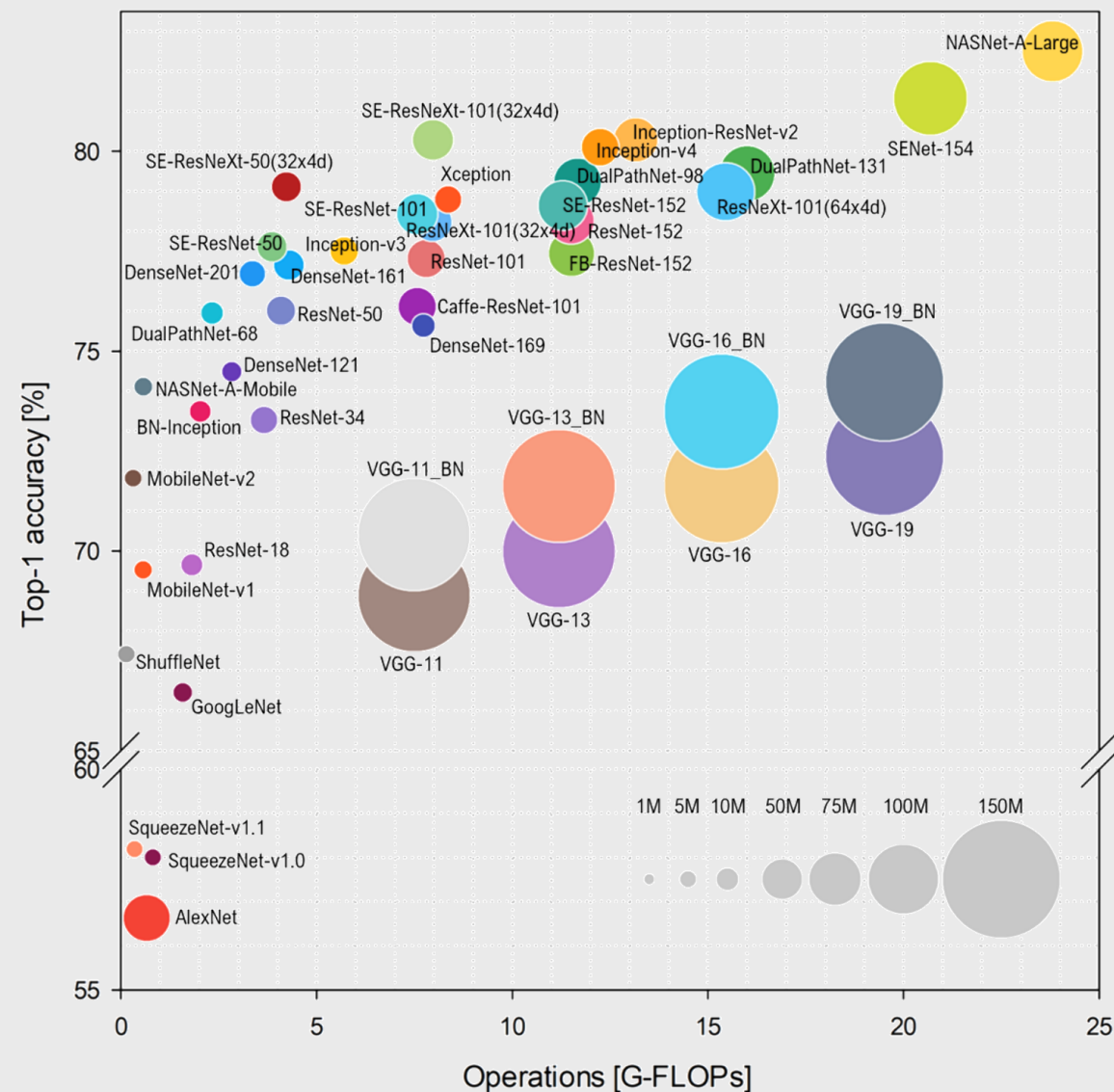
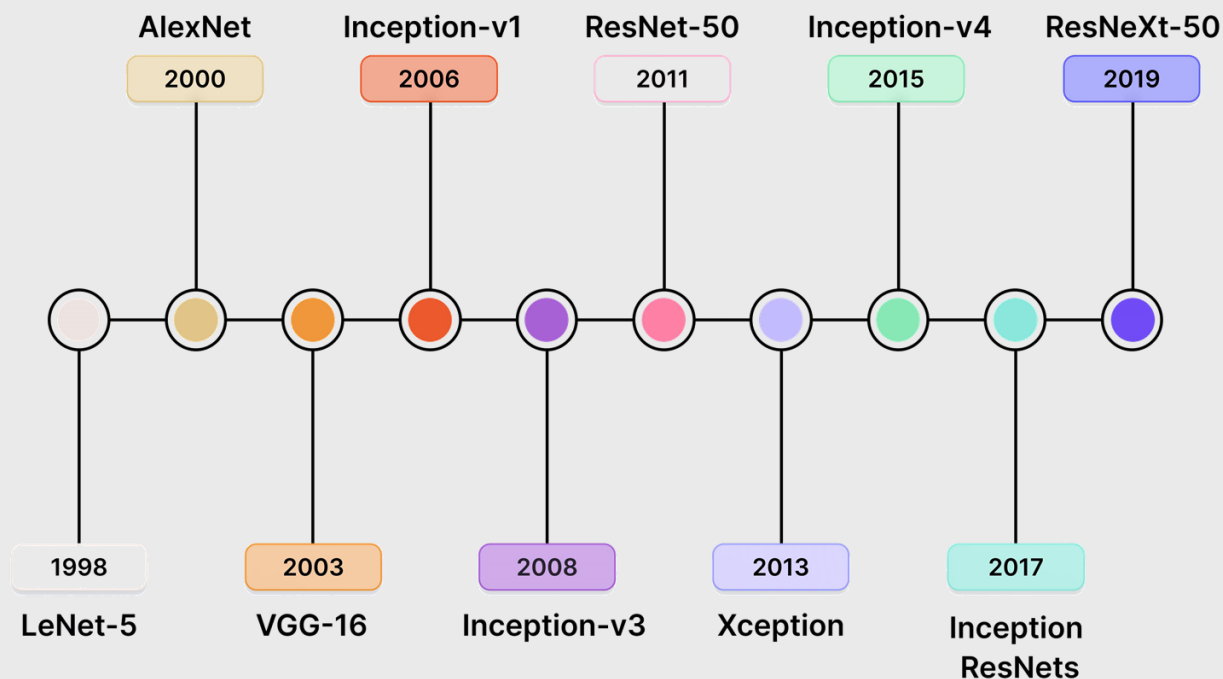
ResNet performs an element-wise addition to pass the output to the next layer or block. **DenseNet** connects all layers directly to each other. It does this through concatenation (combine features by concatenating). Such connection provide stronger gradient flow, feature diversity form using simple features to complex featurers, as opposed to feature corelation in ResNet.

Image source: <https://www.youtube.com/watch?v=-W6y8xnd--U>

Huang et al (2017) Densely Connected Convolutional Networks

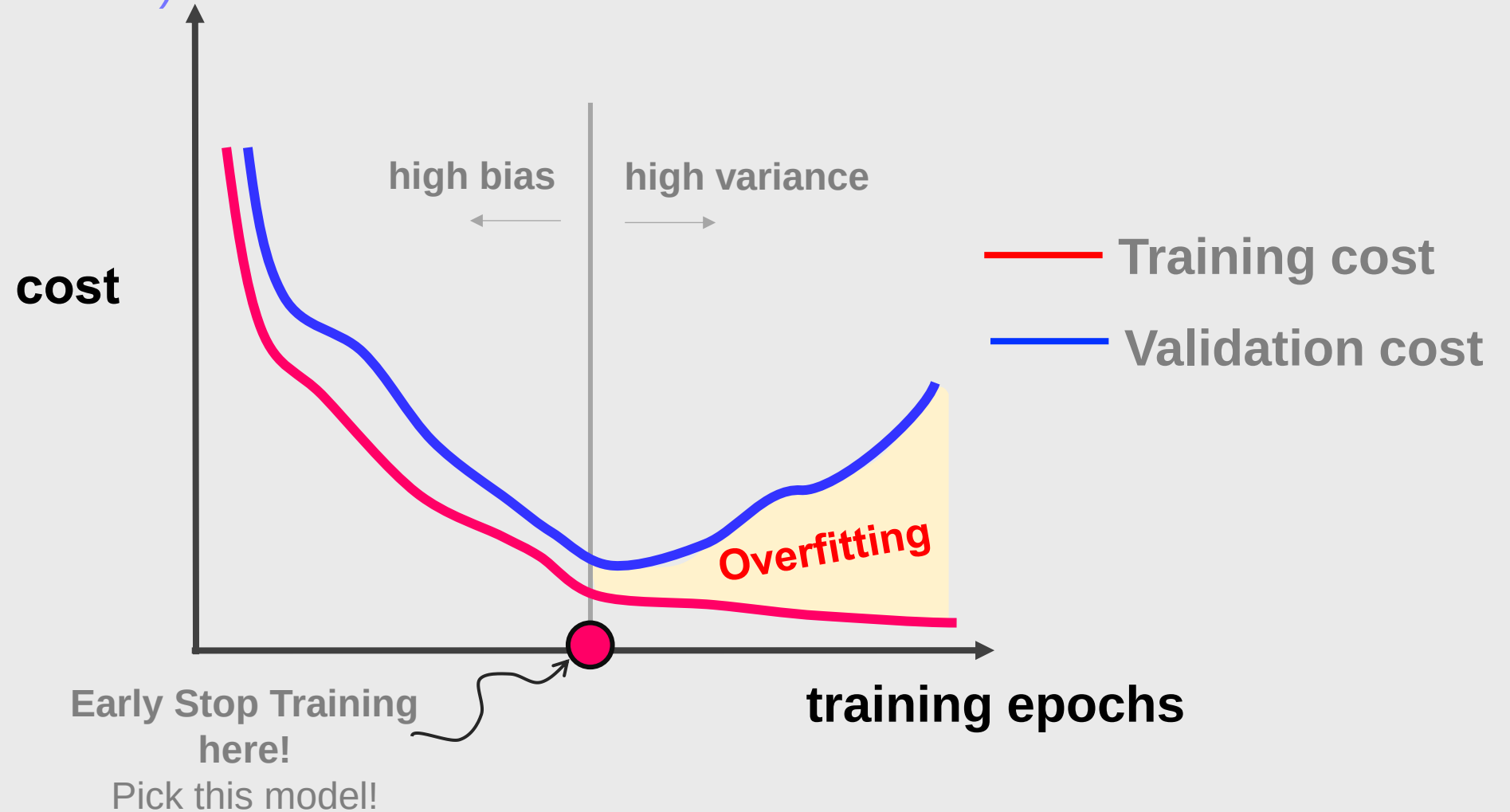
Evolution of CNN Architectures

11:31
AM



Overfitting Issues

(from previous lecturers)



Regularization

11:31
AM

$$E = \frac{1}{n} \sum_i^n (y - f(\mathbf{x}))^2 + \lambda \sum_i^n w_i^2$$

minimize regularizer
term, minima is

$$\lambda \sum_i^n w_i^2 = 0$$

$$\lambda \sum_i^n w_i^2 > 0$$

regularizer term

$$\lambda \sum_i^n w_i^2$$

error term

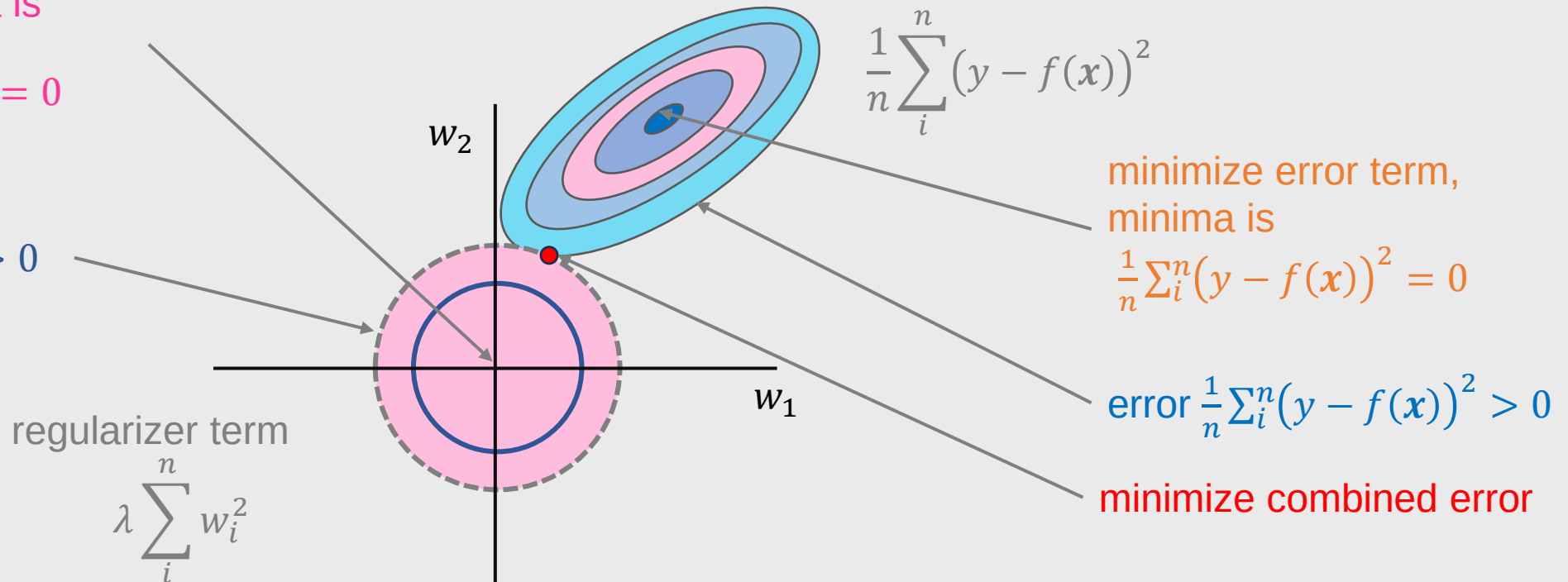
$$\frac{1}{n} \sum_i^n (y - f(\mathbf{x}))^2$$

minimize error term,
minima is

$$\frac{1}{n} \sum_i^n (y - f(\mathbf{x}))^2 = 0$$

error $\frac{1}{n} \sum_i^n (y - f(\mathbf{x}))^2 > 0$

minimize combined error



L_2 -norm regularization

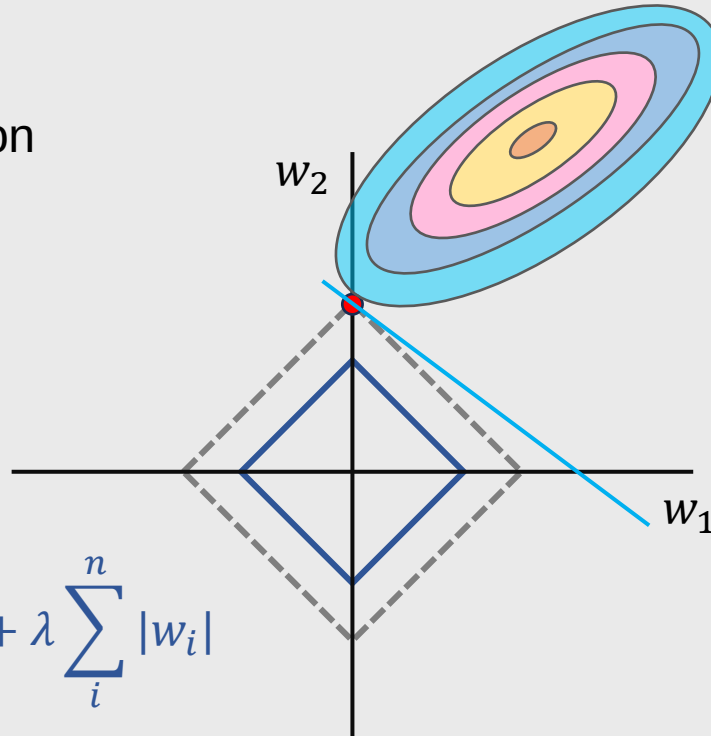
Ridge regularization

Regularization to avoid overfitting

11:31
AM

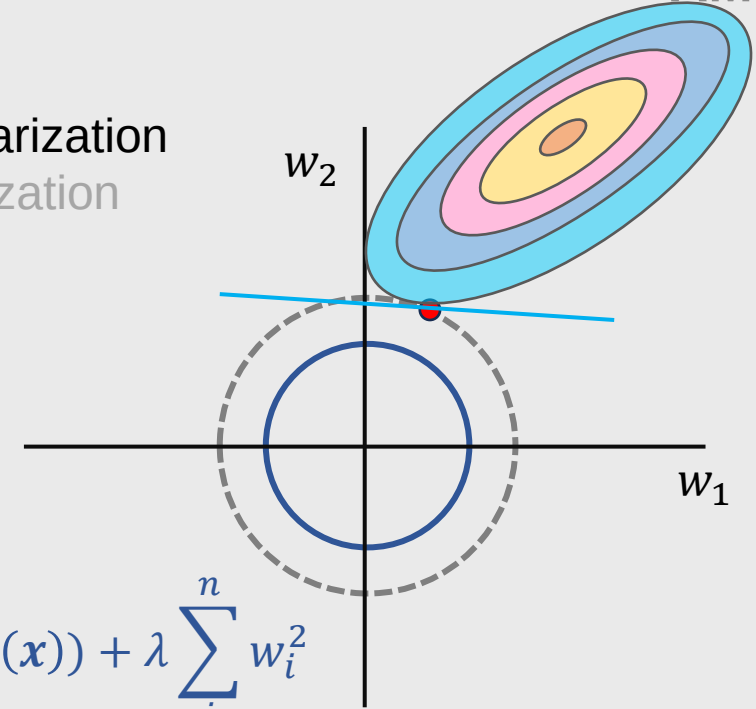
L_1 -norm regularization
Lasso regularization

$$E = \frac{1}{n} \sum_i^n (y - f(\mathbf{x})) + \lambda \sum_i^n |w_i|$$



L_2 -norm regularization
Ridge regularization

$$E = \frac{1}{n} \sum_i^n (y - f(\mathbf{x})) + \lambda \sum_i^n w_i^2$$



- Both Lasso and Ridge regularization help DNNs and CNNs avoid overfitting.
- As weights can get zero in Lasso regularization introduce sparsity in the networks and help feature selection because some weights goes to zero (although not necessary in CNN). Thus, eliminating effect of some input features, while in ridge regression weights can only get close to zeros and not exactly (see images).
- L2 penalizes large errors much more heavily than small errors (compared to L1), thus on optimization of network, it is safe to assume that all the errors are roughly of the same order of magnitude

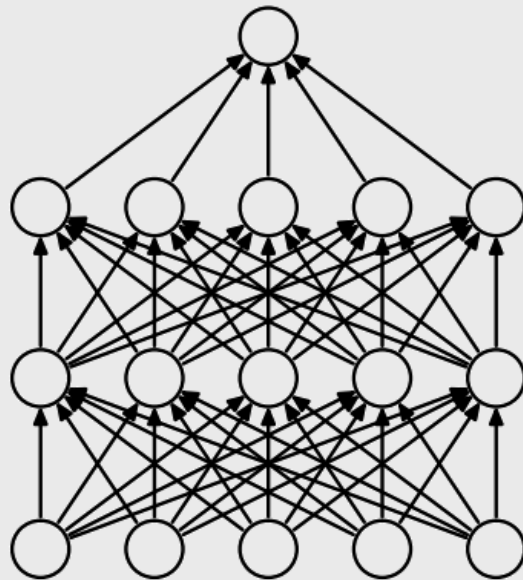
Regularization: Dropouts Layer

Regularisation of DNNs and CNNs

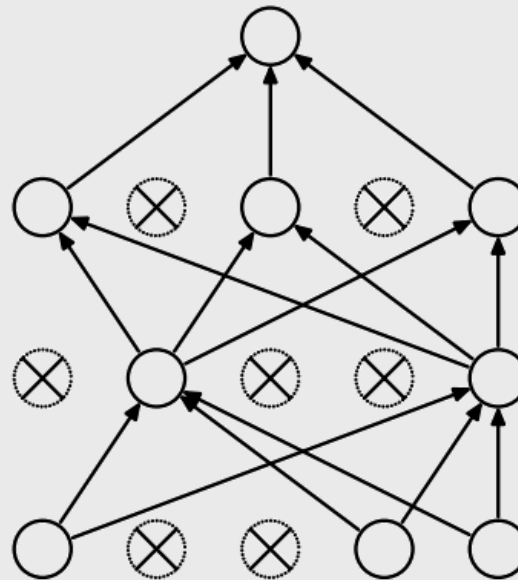
!!!

Slows down training in Convolutional Nets

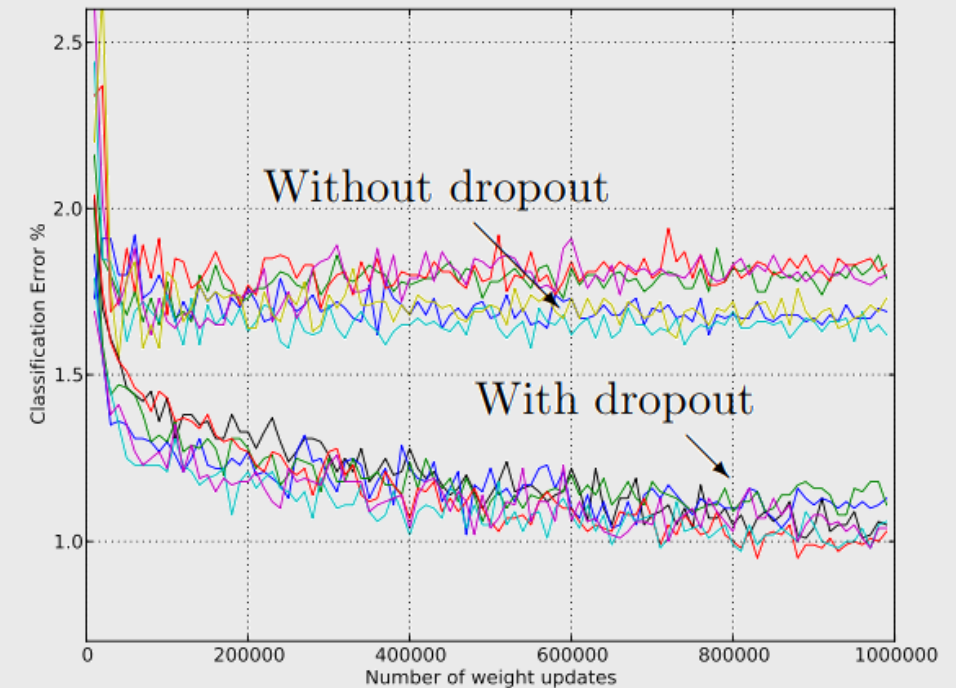
- It drop nodes with some probability
- It regularise Deep Neural Nets and Convolutional Neural Nets



(a) Standard Neural Net



(b) After applying dropout.



Sec 7.12, Goodfellow, Deep Learning, MIT Press

Srivastava et al (2014) Dropout: A Simple Way to Prevent Neural Networks from Overfitting, JMLR
Dr Varun Ojha, Newcastle University

Batch Normalization Layer

Regularisation of Convolutional Neural Networks

- It eliminates the need of dropout
- Accelerates ConvNet training
- It reduces sensitivity to network weight initialisation
- standardizing the activations to have a zero mean and unit variance

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

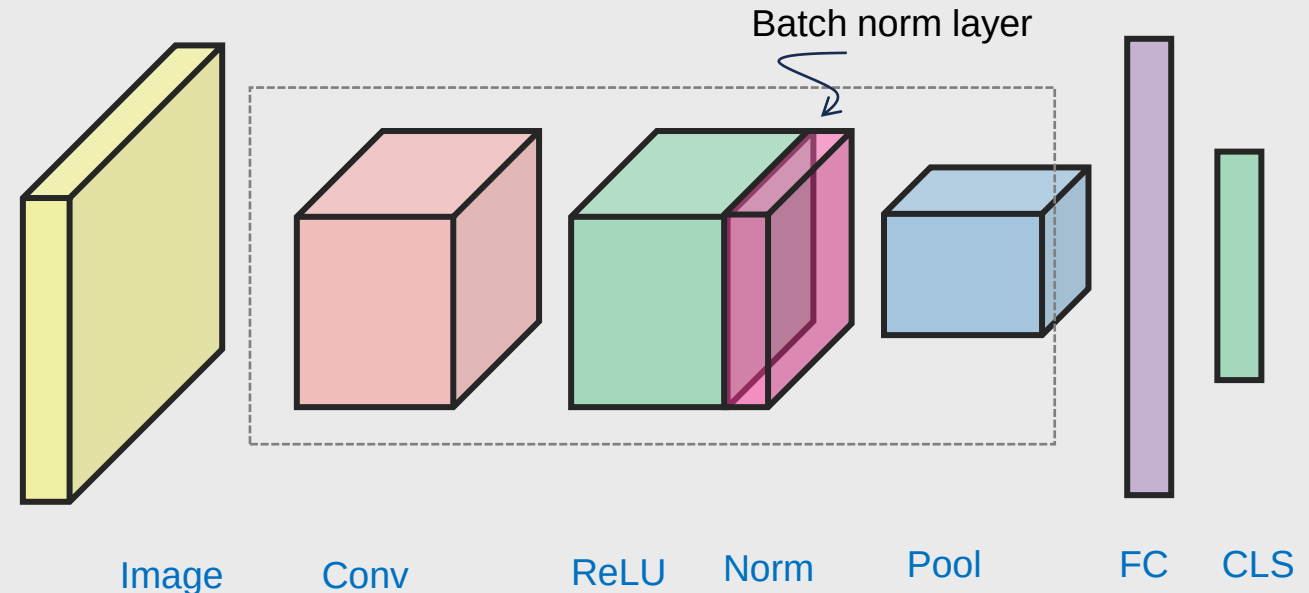
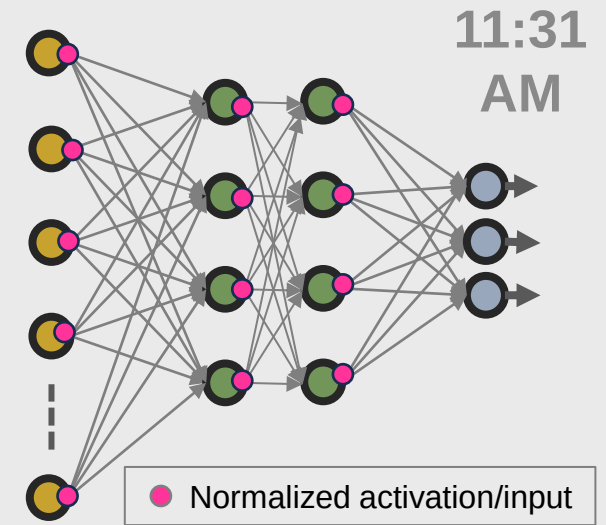
$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$y_i = \gamma \hat{x}_i + \beta = \text{BN}_{\gamma, \beta}(x_i)$$

Ioffe and Szegedy (2017) Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift ICML.

Dr Varun Ojha, Newcastle University

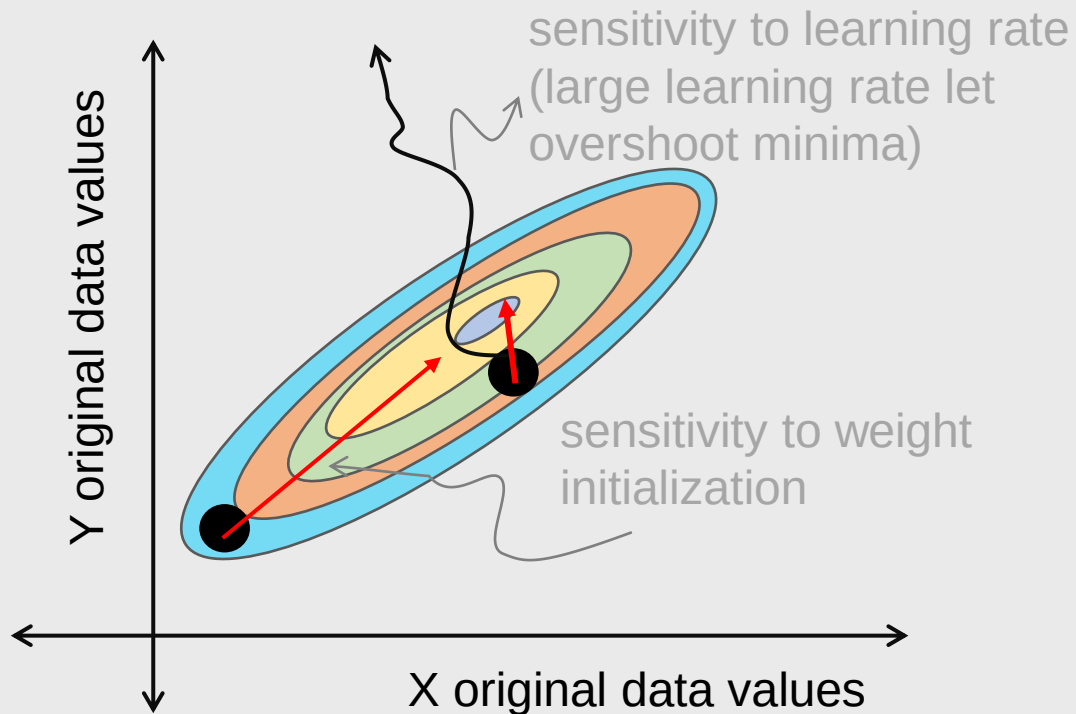


Batch Normalization Layer

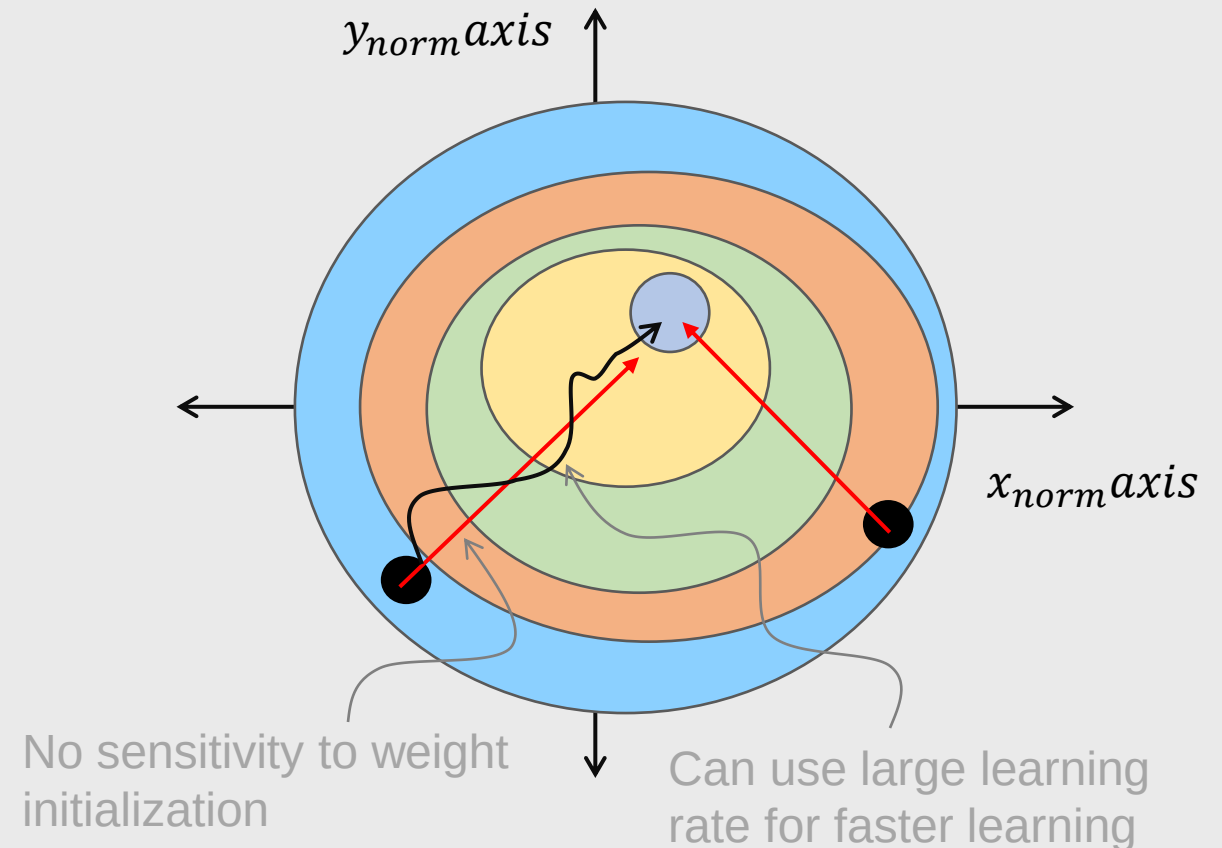
11:31
AM

Regularisation of Convolutional Neural Networks

Optimization/ cost function
space on unnormalized inputs



Optimization/ cost function
space on unnormalized inputs



Part 3

Convolutional Neural Network Applications

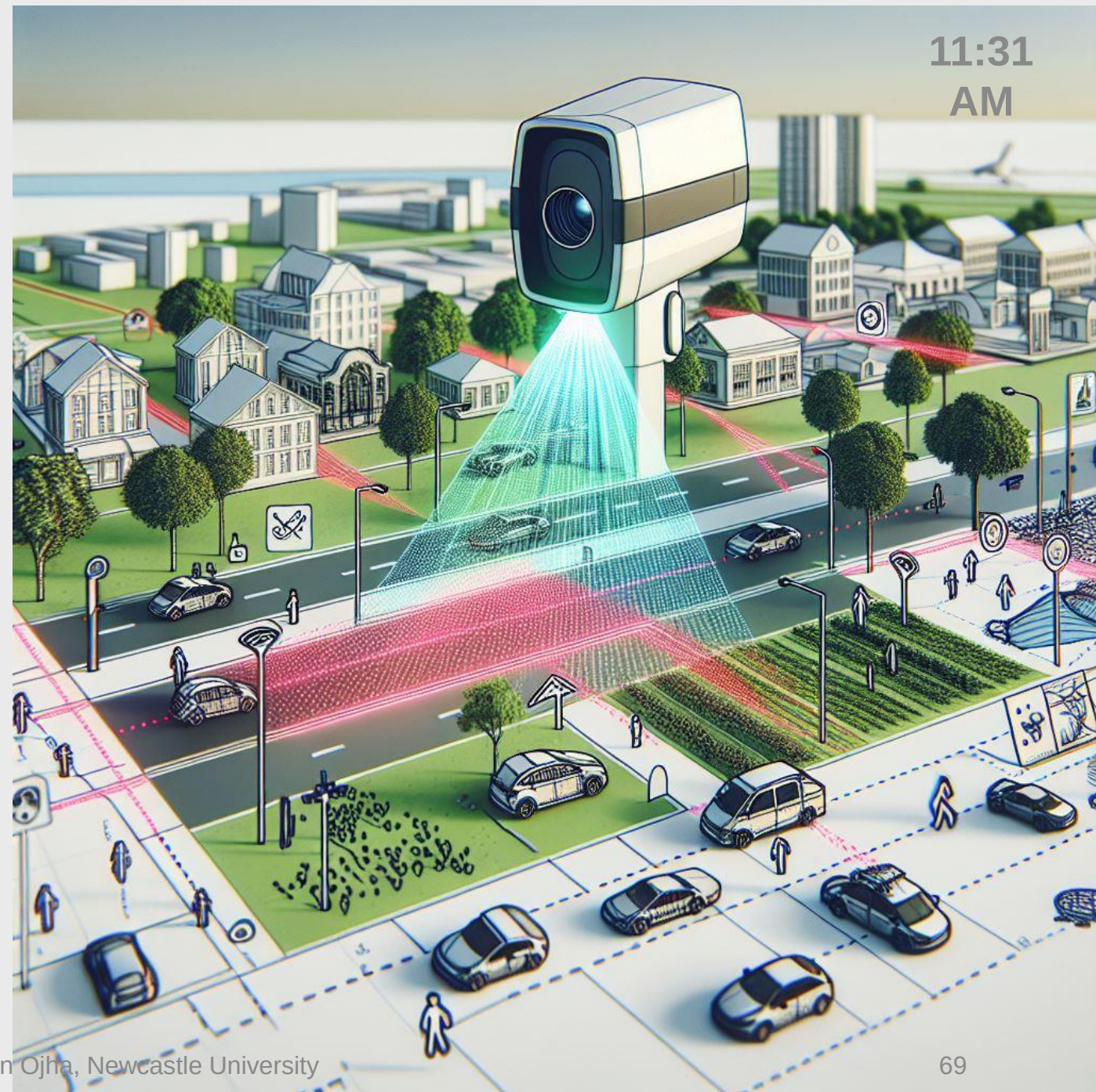


Image Classification Example

Live demo <http://cs231n.stanford.edu/>

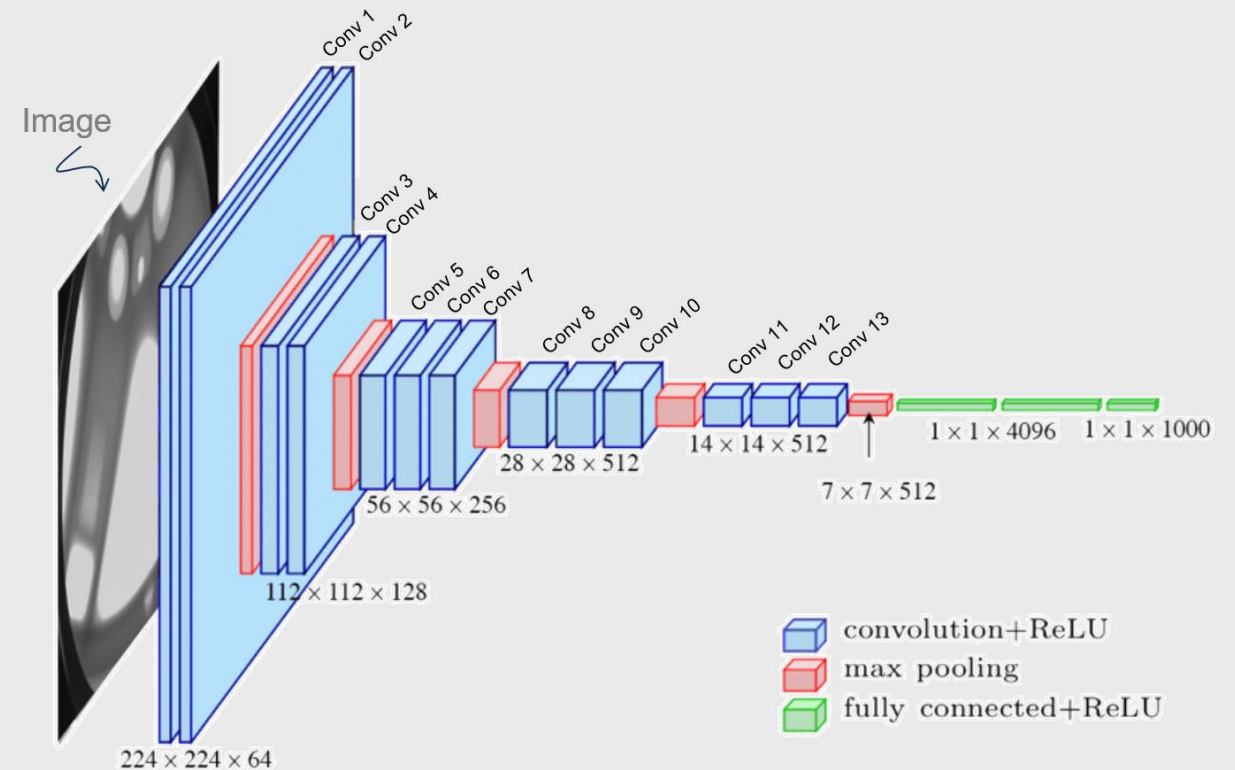


A very good source:
<http://cs231n.github.io/convolutional-networks/>

Image Classification Models

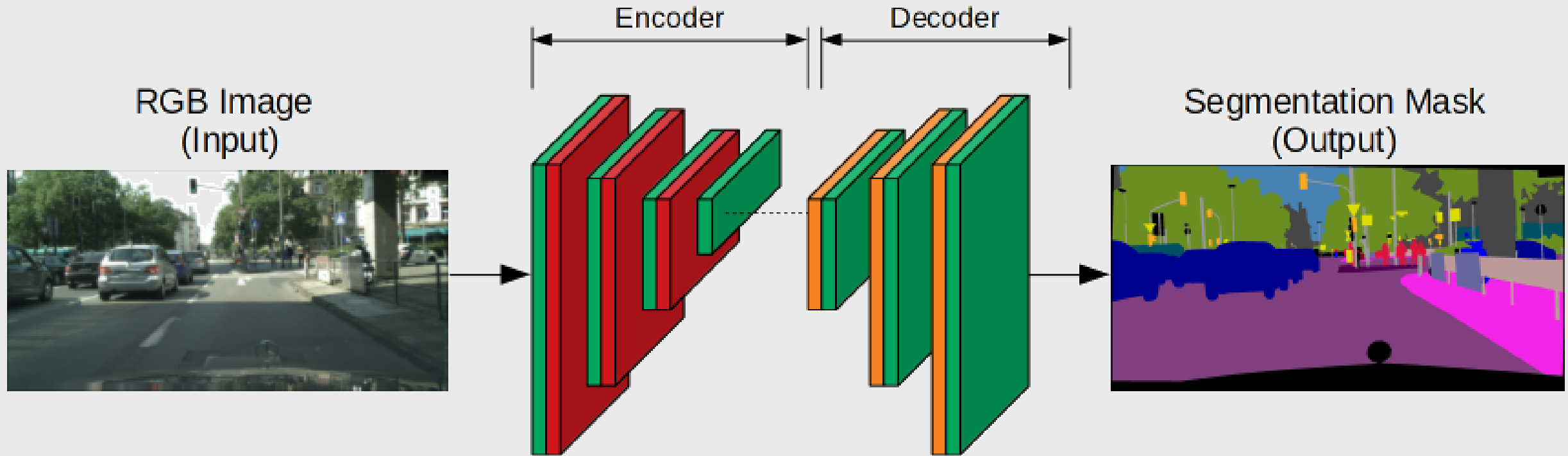
- Residual Networks (**ResNet**)
- Visual Geometry Group (VGG) Network (**VGG Net**)
- **DenseNet**
- **InceptionNet**
- Other pre-trained Image Classification Nets on pytorch library:

<https://pytorch.org/vision/main/models.html>



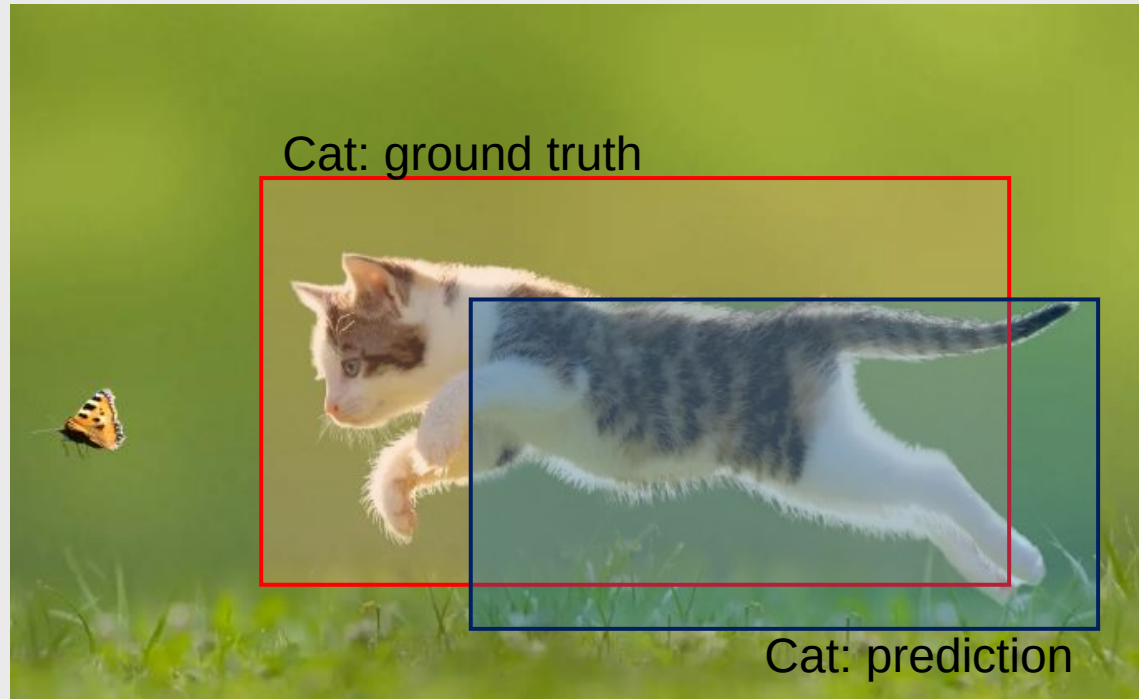
VGG Net 16

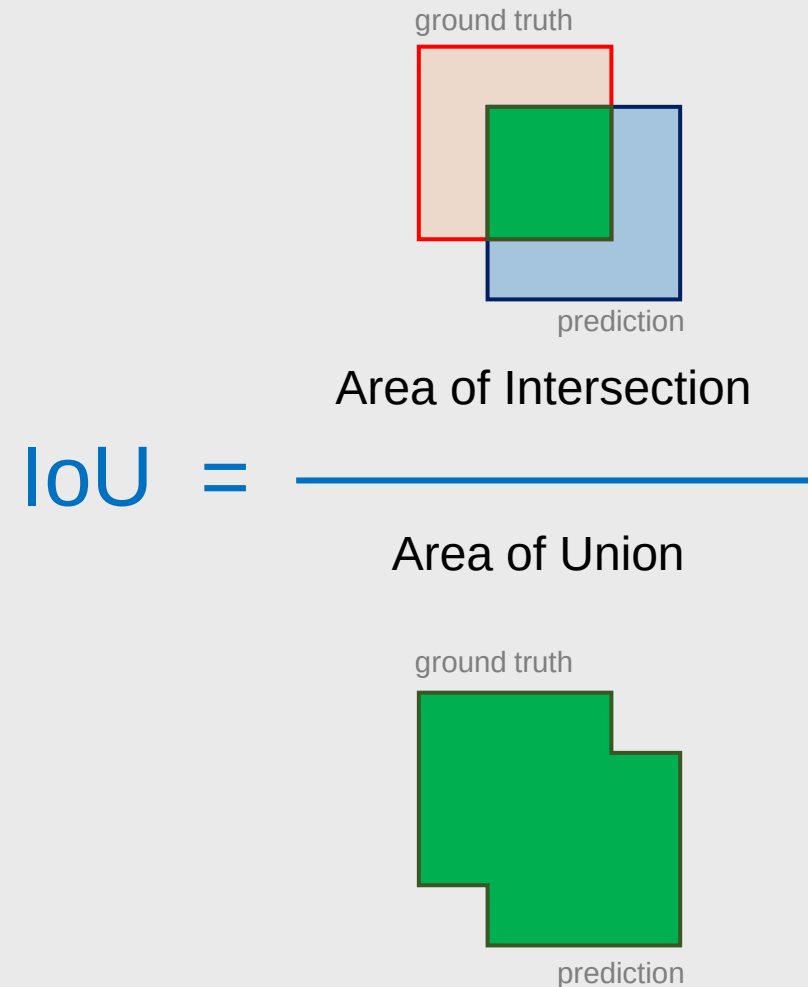
Image Segmentation (Concept)



Intersection over Union (IoU)

IoU measure the performance of image segmentation and object detection algorithms performance





ground truth

prediction

Area of Intersection

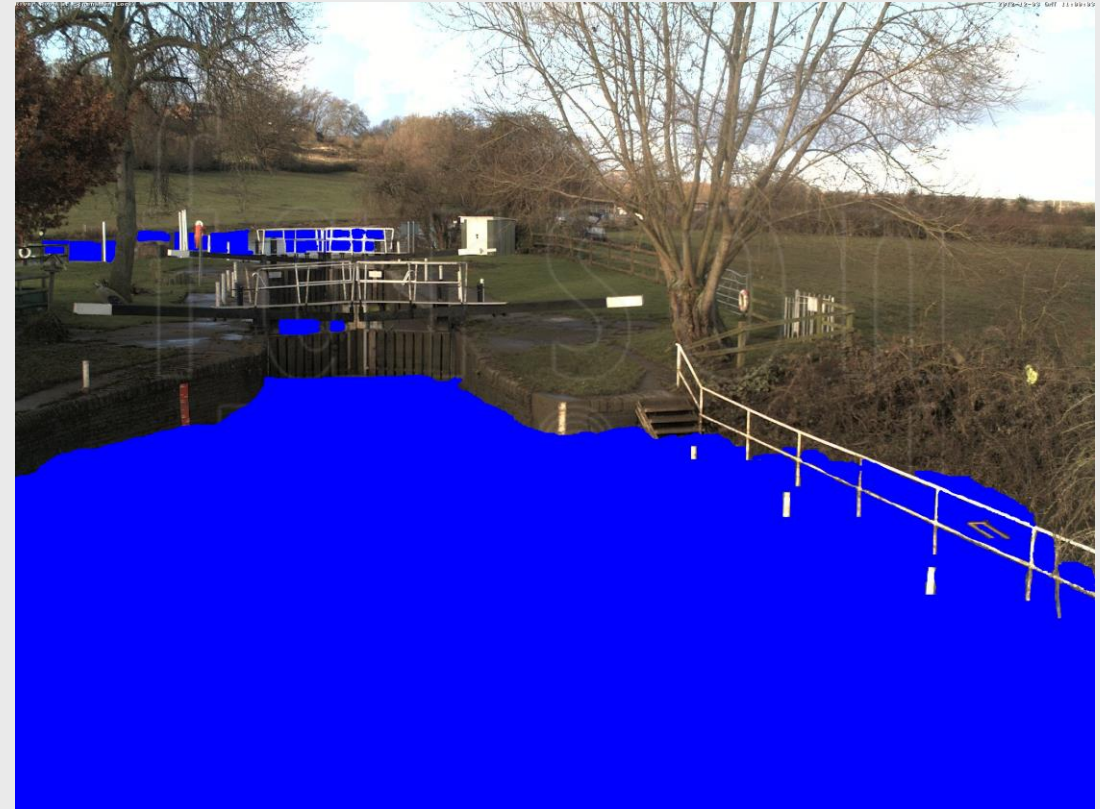
$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

ground truth

prediction

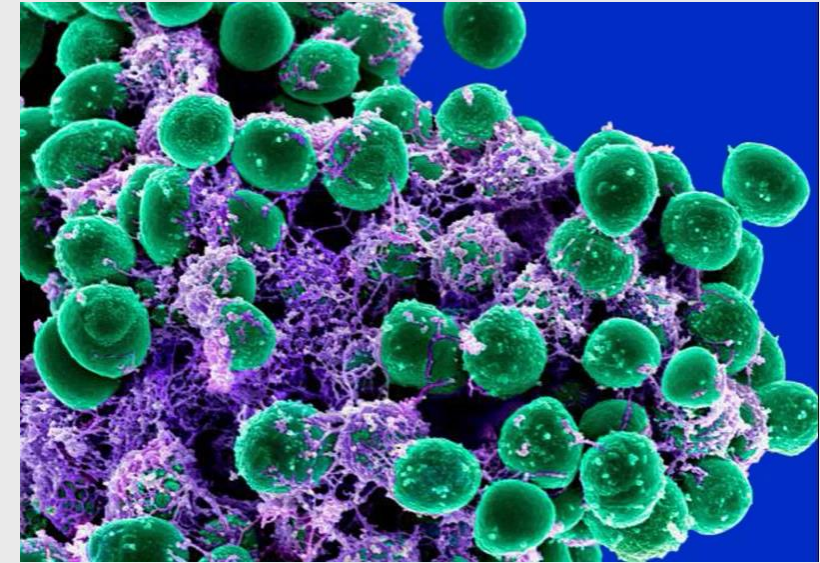
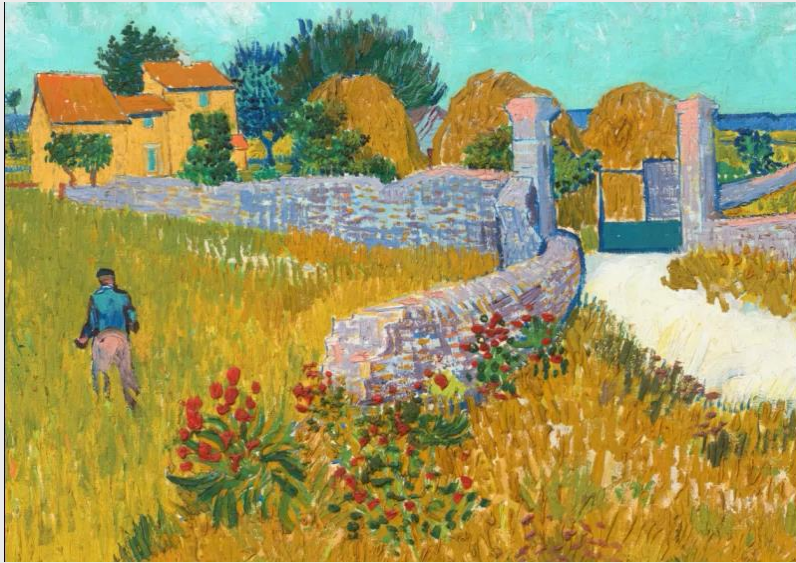
Image Segmentation used for water level estimation

11:31
AM



Pixel-wise water segmentation of RGB images for river water-level monitoring or flood monitoring

Image Segmentation (Concept)



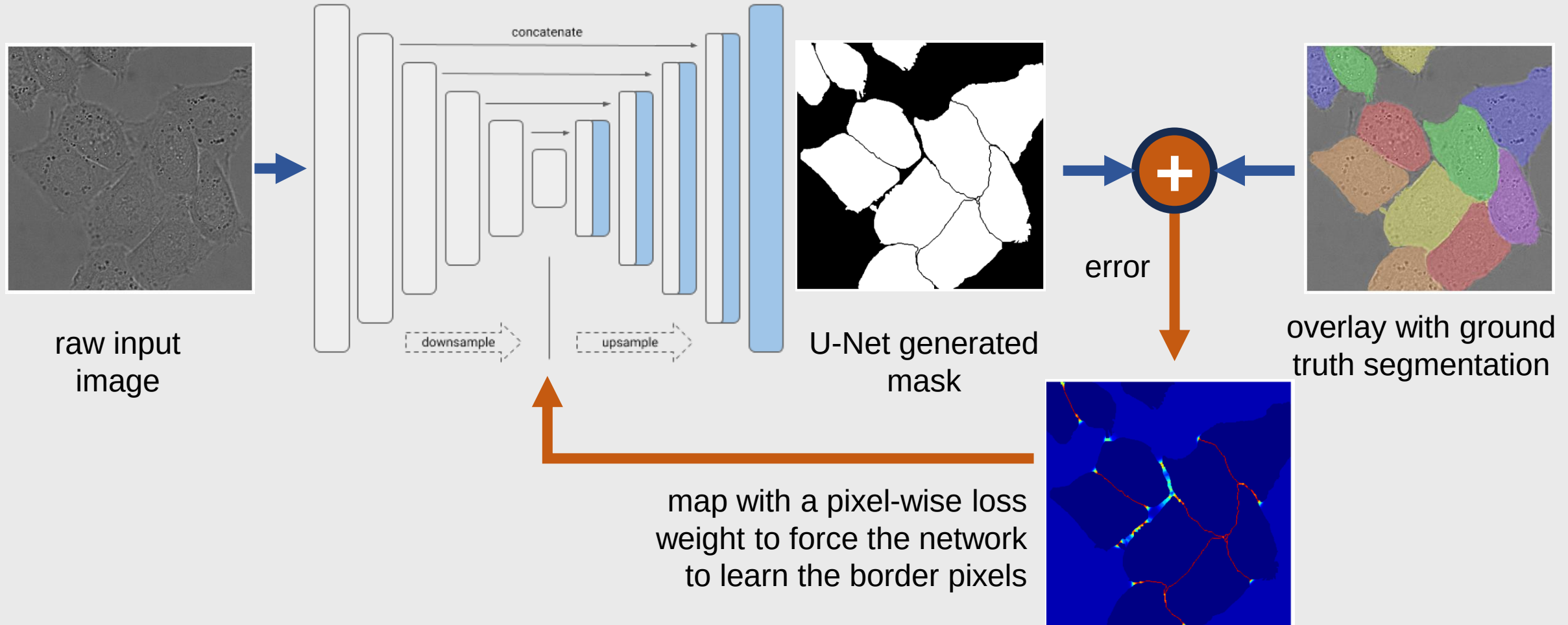
Segment Anything Model from Meta (2023)

<https://segment-anything.com/>

UNet

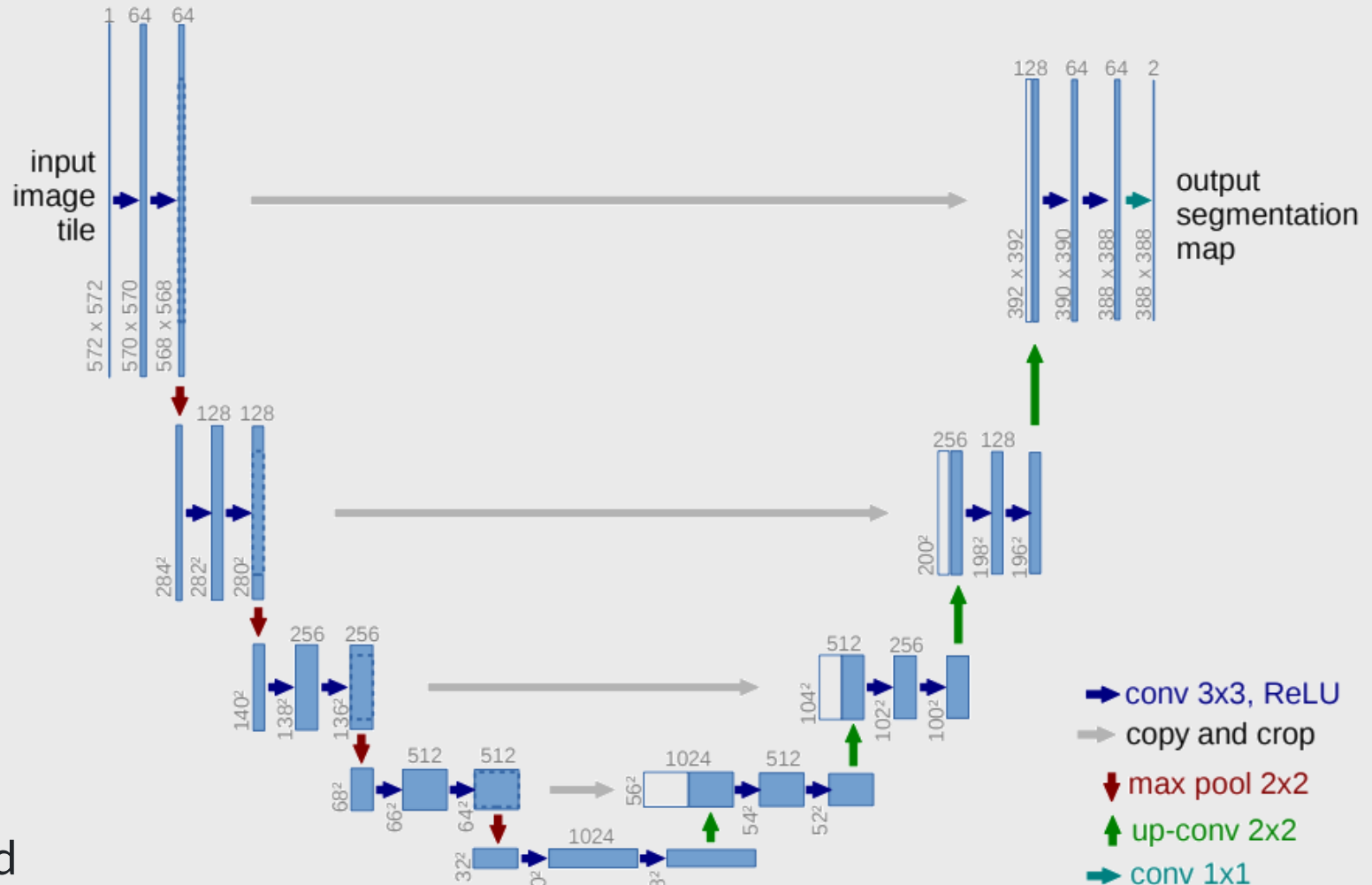
11:31
AM

U-Net is a typical CNN architecture for semantic segmentation. It has a contracting path (down sampling) and an expansive path (up sampling).



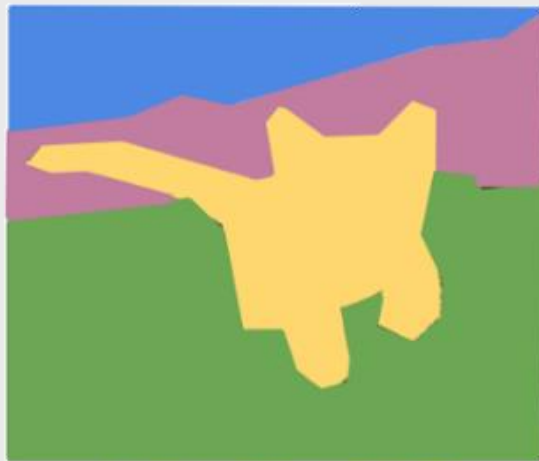
U-Net

- **Contracting path** is a repeated application of **two 3x3 convolutions** and a **ReLU** and a **2x2 Max Pooling** operation with stride 2
- **Expansive path** is an up sampling of the feature map followed by a **2x2 Convolution** (“up-convolution”) that halves the number of feature channels, a concatenation with the correspondingly cropped feature map from the contracting path, and **two 3x3 convolutions**, each followed by a **ReLU**.



Object Detection

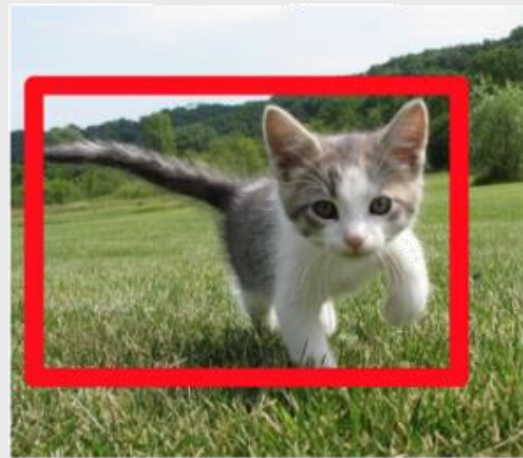
**Semantic
Segmentation**



GRASS, CAT,
TREE, SKY

No objects, just pixels

**Classification
+ Localization**



CAT

Single Object

**Object
Detection**



DOG, DOG, CAT

Multiple Object

**Instance
Segmentation**

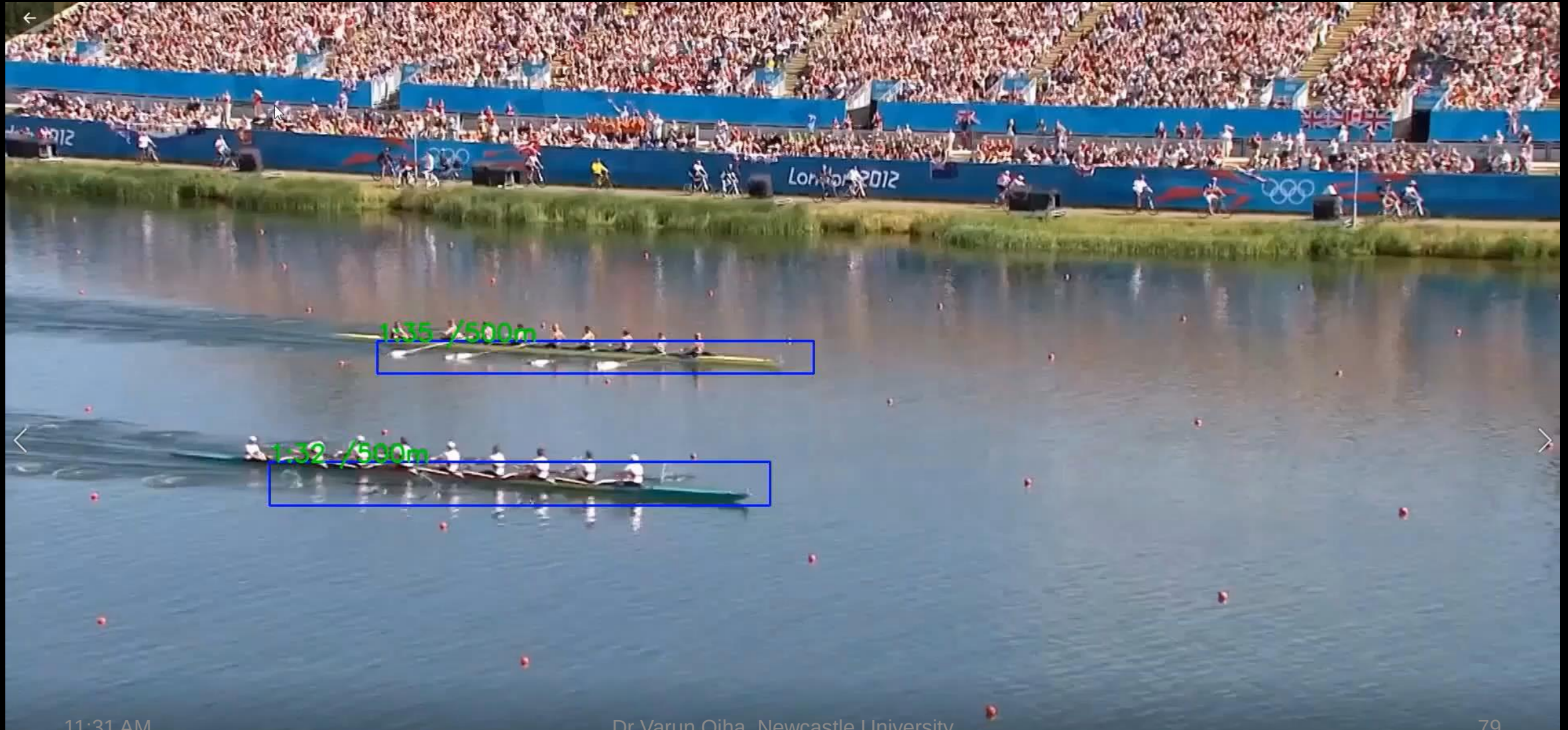


DOG, DOG, CAT

This image is CC0 public domain

Application: Boat (Object) Detection and boat speed measurement

One of my Undergraduate student's project



Application: Instance Segmentation and Object Detection: Plastic Pollution Detection



Object Detection

(plastic pollution detection - One of my student's project)

11:31 AM

Input Video



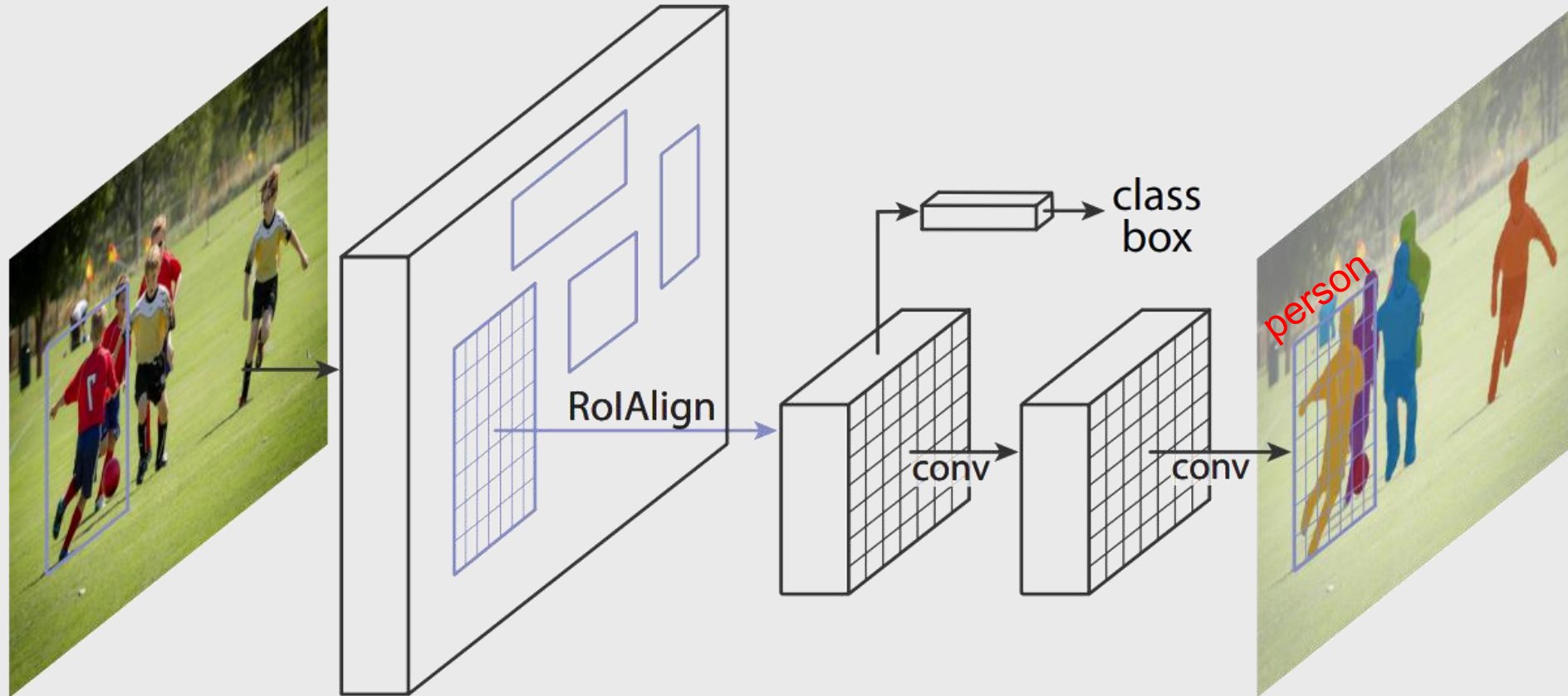
Output Video



Published work: Jaikumar P et al. (2020) ISDA, <https://centaur.reading.ac.uk/98569/>

Mask RCNN

11:31
AM

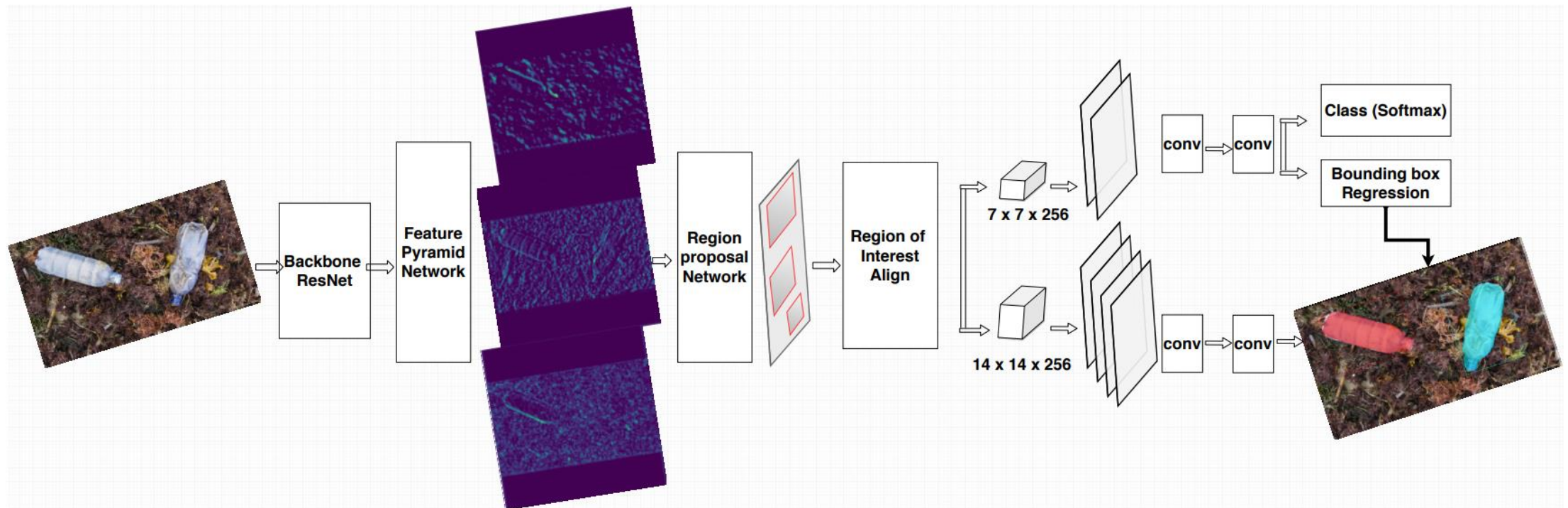


Input Image

Output

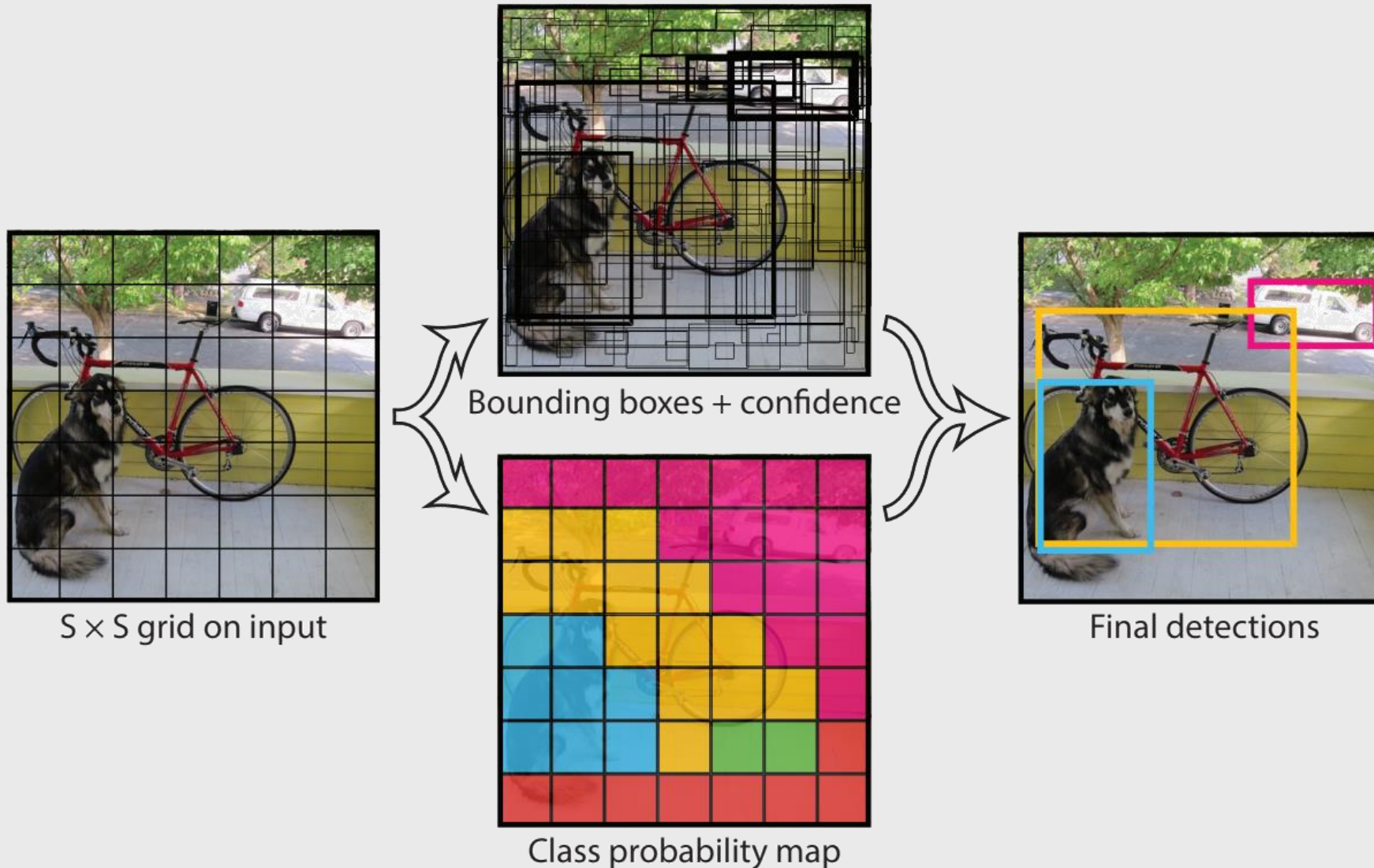
- A bounding box
- A class label (e.g., person)
- A Segmentation mask

Mask-RCNN



You Look Only Once (YOLO) Model

11:31
AM



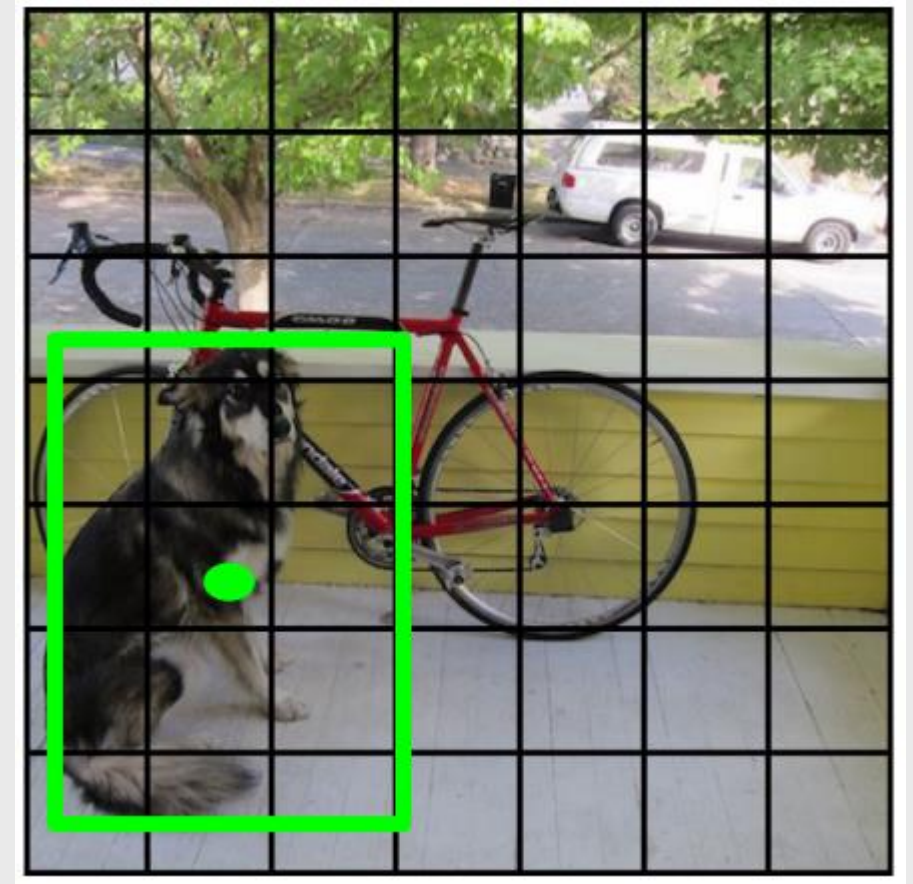
Object detection in YOLO models are as a regression problem. It divides the image into an $S \times S$ grid and for each grid cell it predicts B bounding boxes, confidence for those boxes, and C class probabilities.

You Look Only Once (YOLO) Model

11:31
AM



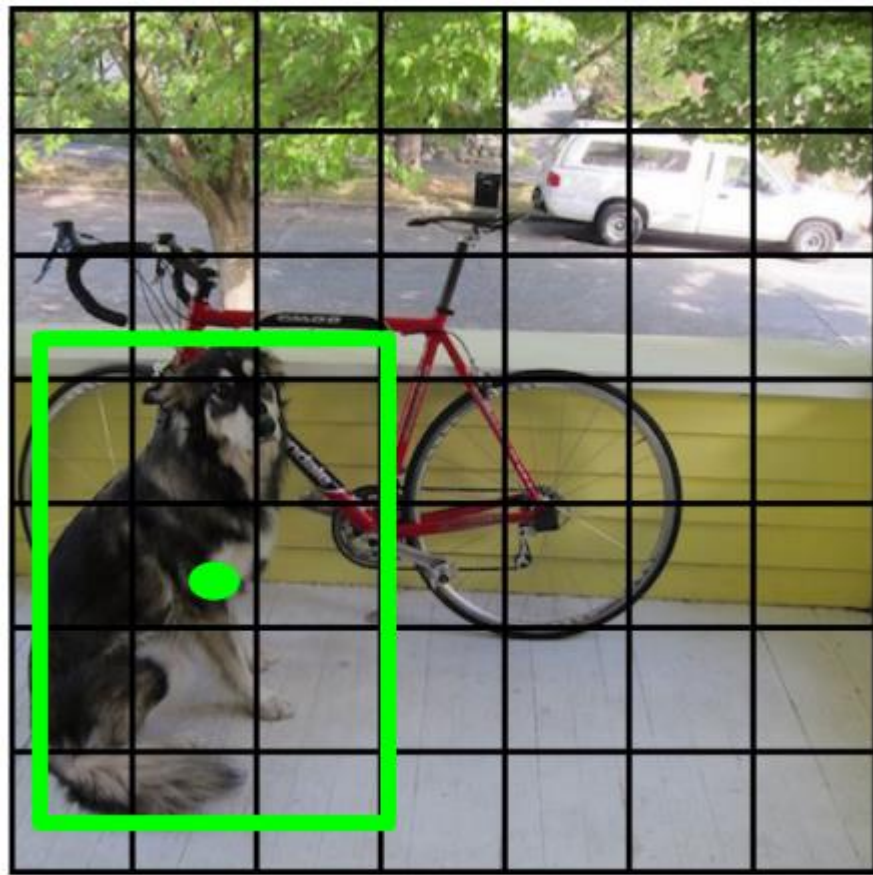
Original input



Ground Truth

You Look Only Once (YOLO) Model

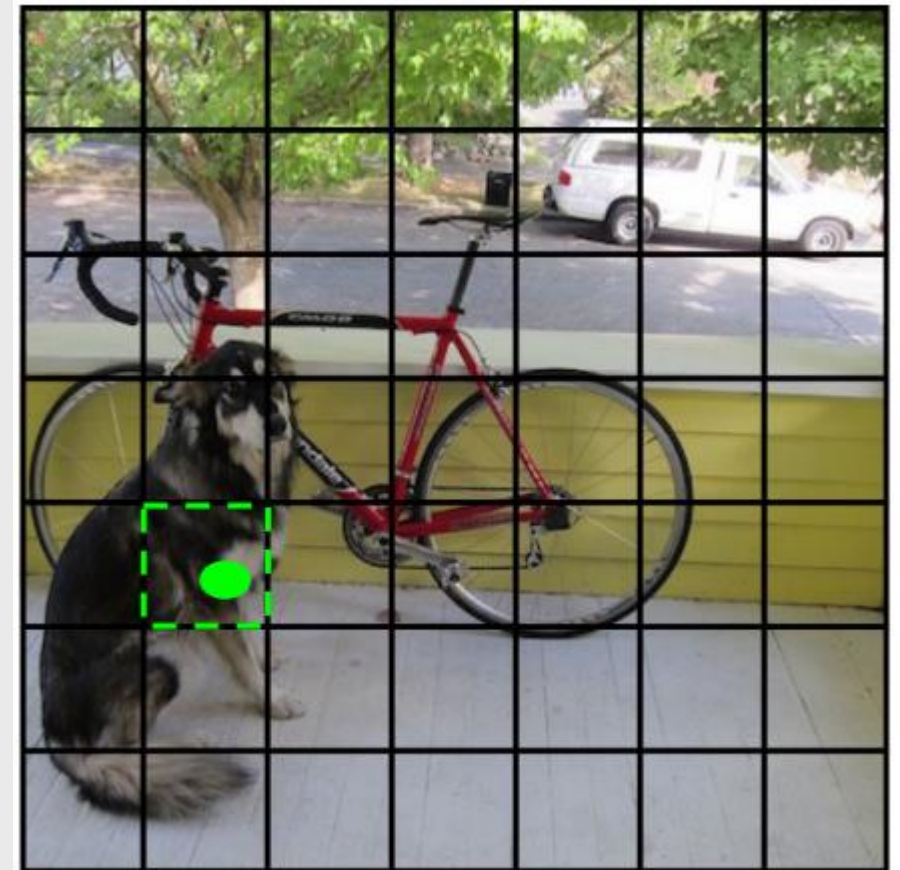
11:31
AM



Ground Truth



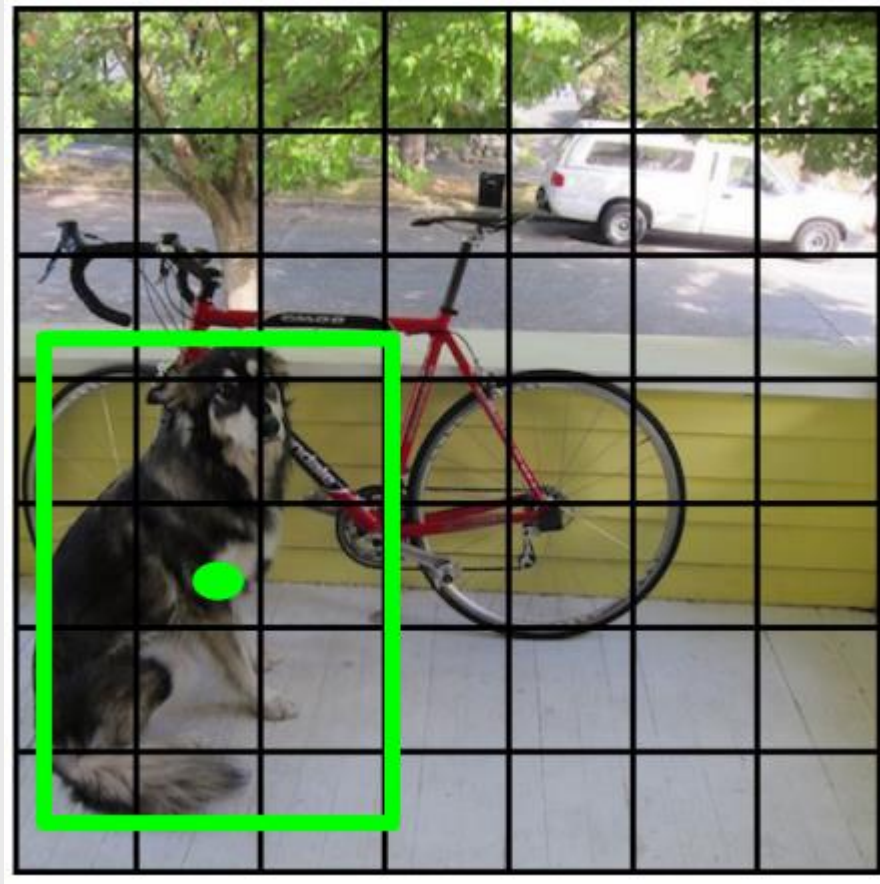
Match
predicted
cell with
ground
through



Cell Prediction

You Look Only Once (YOLO) Model

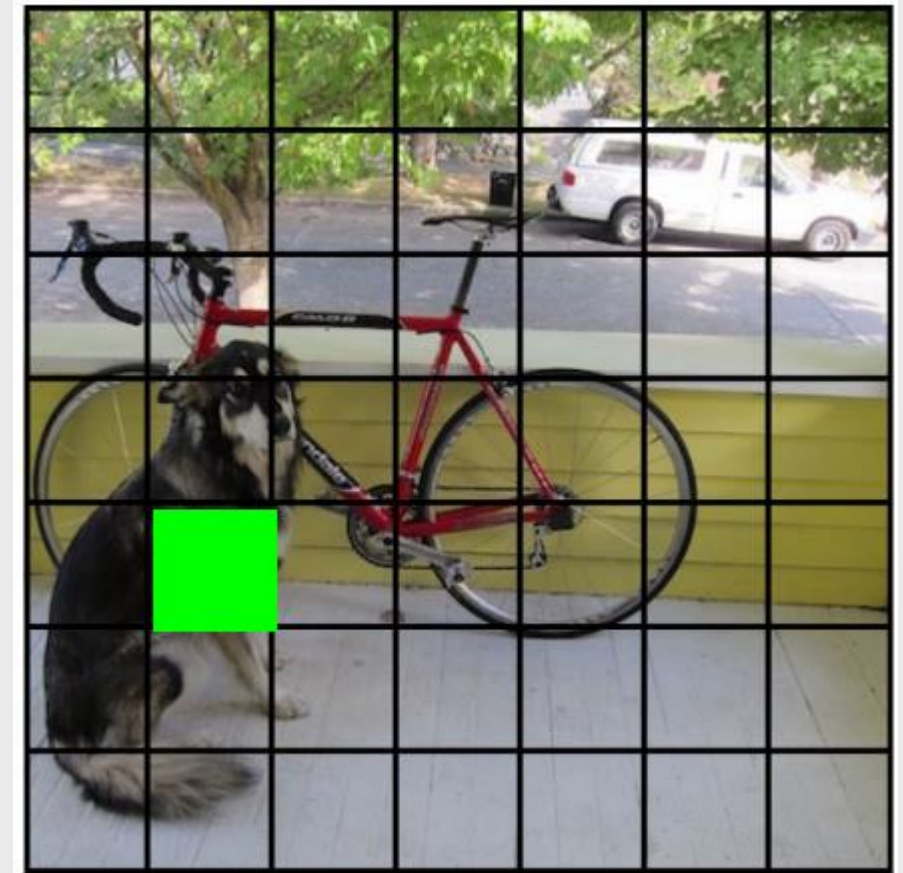
11:31
AM



Ground Truth



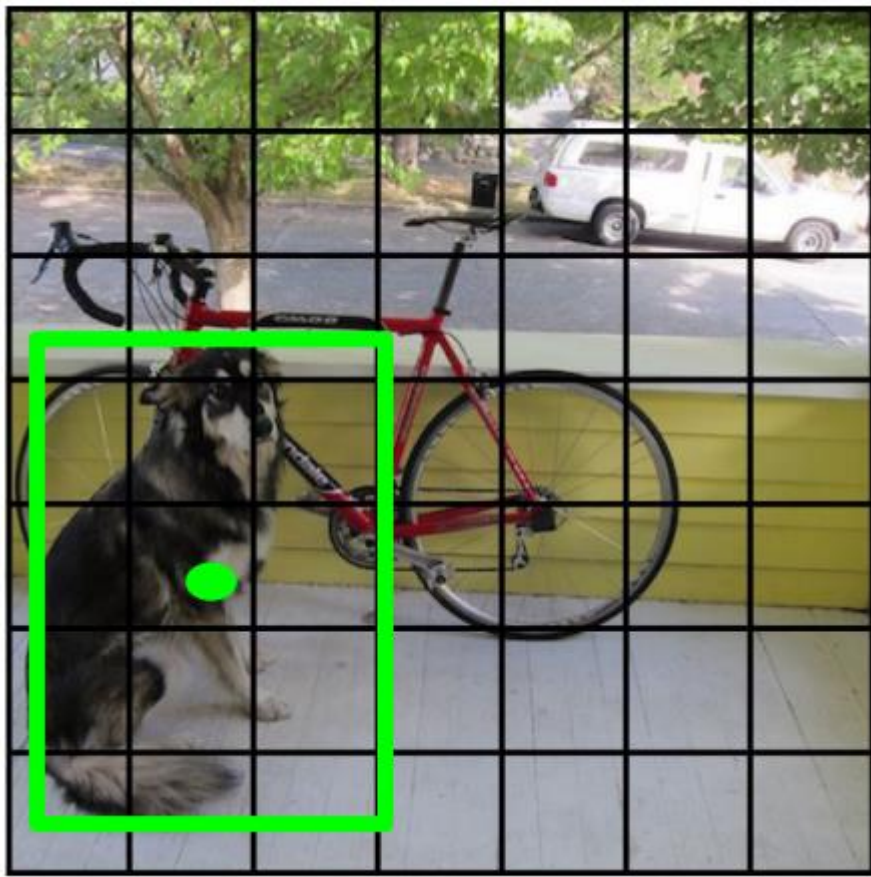
Check the
class
prediction
(class
probability,
i.e.,
SoftMax)



**Class
Prediction**

You Look Only Once (YOLO) Model

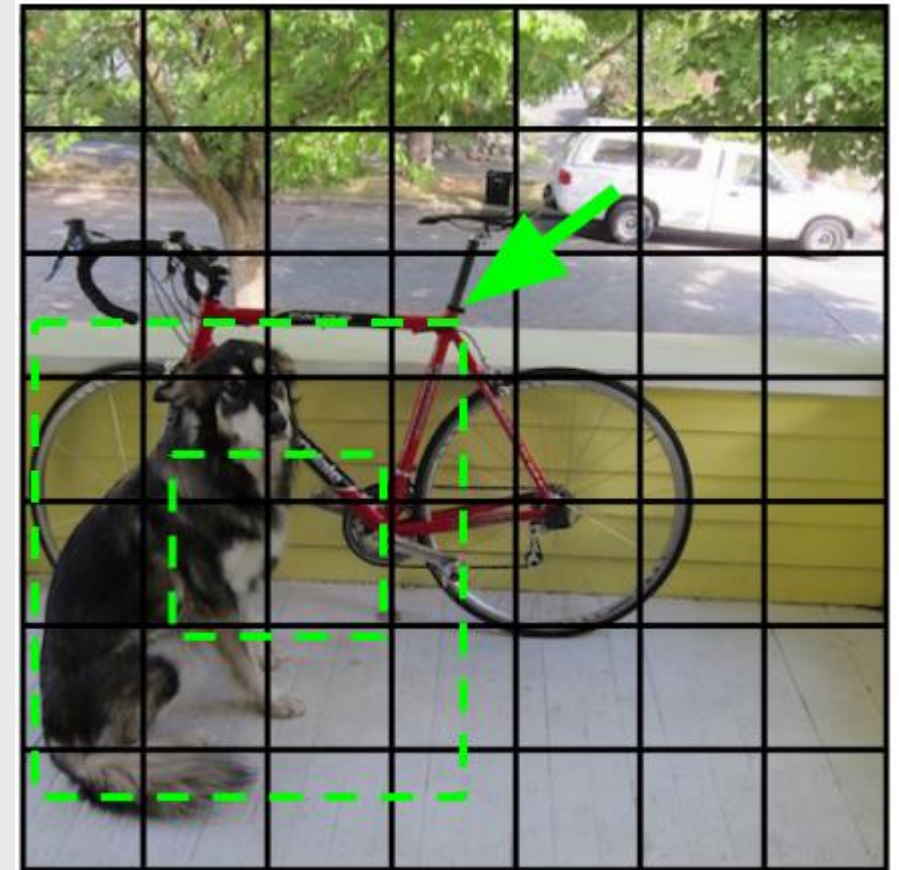
11:31
AM



Ground Truth



Check the
box
prediction
and its
confidence
(i.e., IoU)

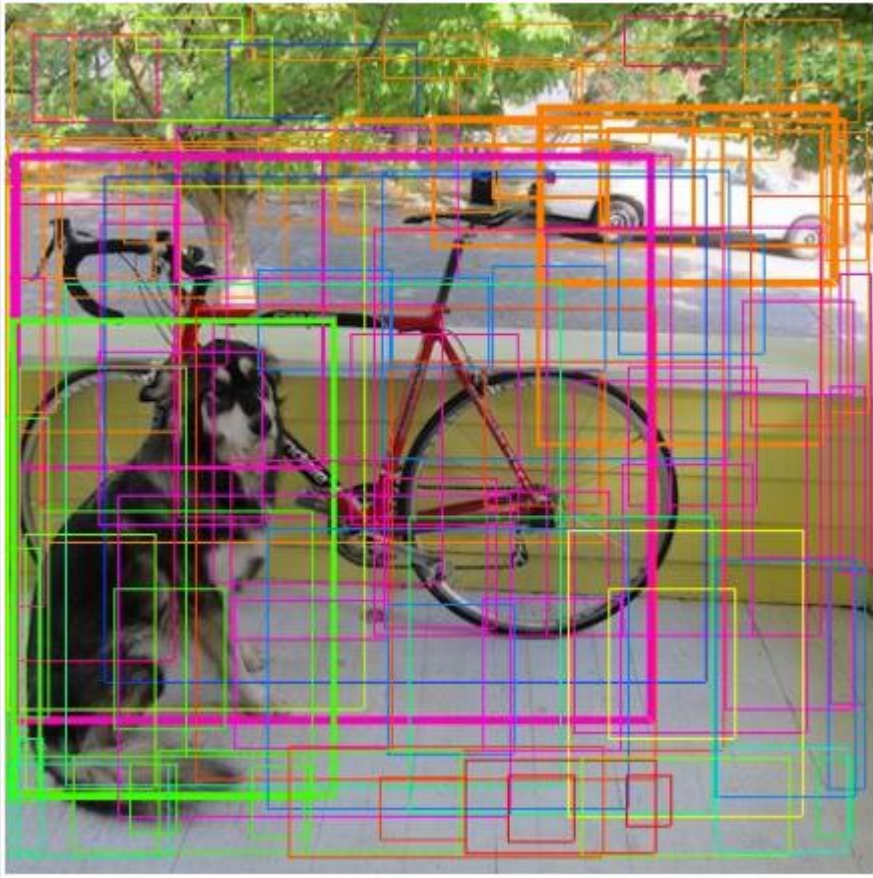


**Box
Prediction**

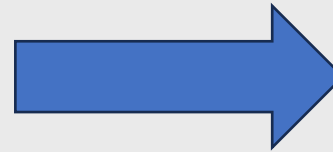
You Look Only Once (YOLO) Model

11:31
AM

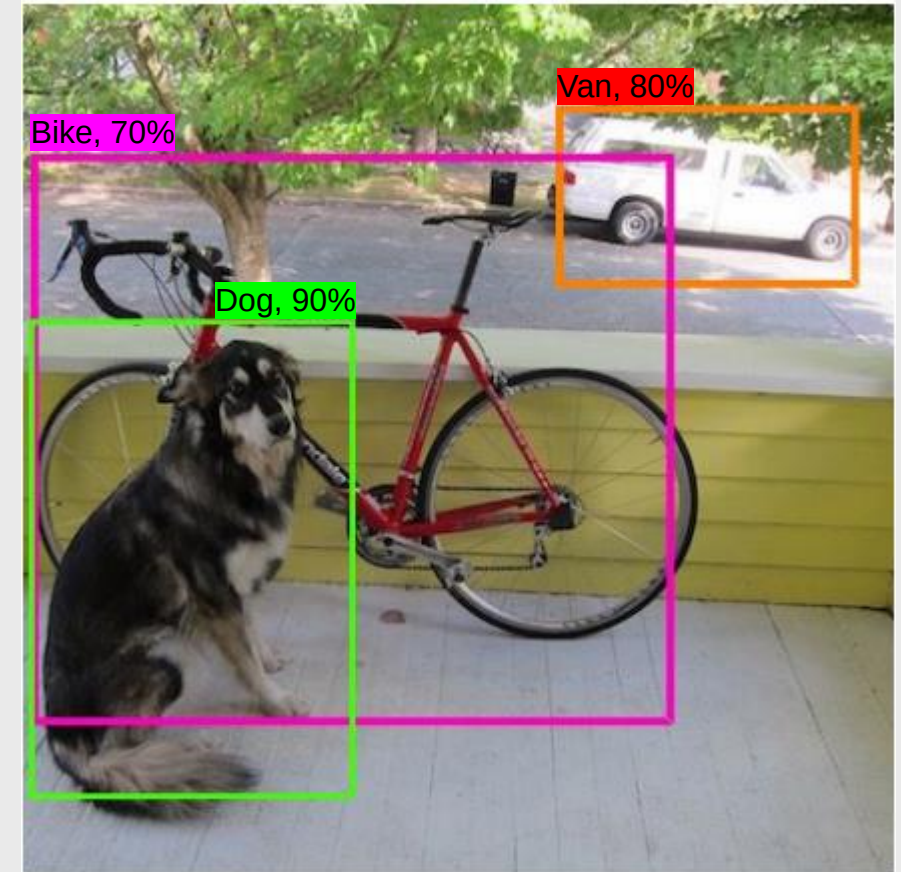
Non-maximal suppression (NMS) $Box = \operatorname{argmax}(C(P_1), C(P_2), \dots, C(P_n))$



**Box Prediction with
confidence scores**



**Perform NMS,
i.e., keep only
boxes with
maximal
confidence
score**



Final Detection

Training Loss of YOLO models

$$L_{YOLO} = L_{\text{class}} + L_{\text{localization}}$$

$$L_{\text{class}} = \sum_{i=0}^{S^2} \mathbb{I}_i^{\text{object}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

$$\mathbb{I}_i^{\text{object}} = \begin{cases} 1 & \text{if object in box } i \\ 0 & \text{otherwise} \end{cases}$$

$$L_{\text{localization}} = L_{\text{confidence}} + L_{\text{coordinate}}$$

Training Loss of YOLO models

$$L_{YOLO} = L_{\text{class}} + L_{\text{localization}}$$

$$L_{\text{localization}} = L_{\text{confidence}} + L_{\text{coordinate}}$$

$$L_{\text{coordinate}} = \lambda_{\text{coordinate}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{\text{object}} l$$

$$l = (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 + (x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2$$

$$L_{\text{confidence}} = \sum_{i=0}^{S^2} \sum_{j=0}^B \left[\mathbb{I}_{ij}^{\text{object}} (c_i - \hat{c}_i)^2 \right] + \lambda_{\text{noObject}} \sum_{i=0}^{S^2} \sum_{j=0}^B \left[\mathbb{I}_{ij}^{\text{object}} (c_i - \hat{c}_i)^2 \right]$$

Where w, h, x, y, s are grid size, width, height, x-axis, and y-axis position of the box B, and the grid size

Generative Models

Which one is
Real,
and which
one is Fake?



1



2

Super Realistic Generative Adversarial Networks

bicubic
(21.59dB/0.6423)



SRResNet
(23.53dB/0.7832)



SRGAN
(21.15dB/0.6868)



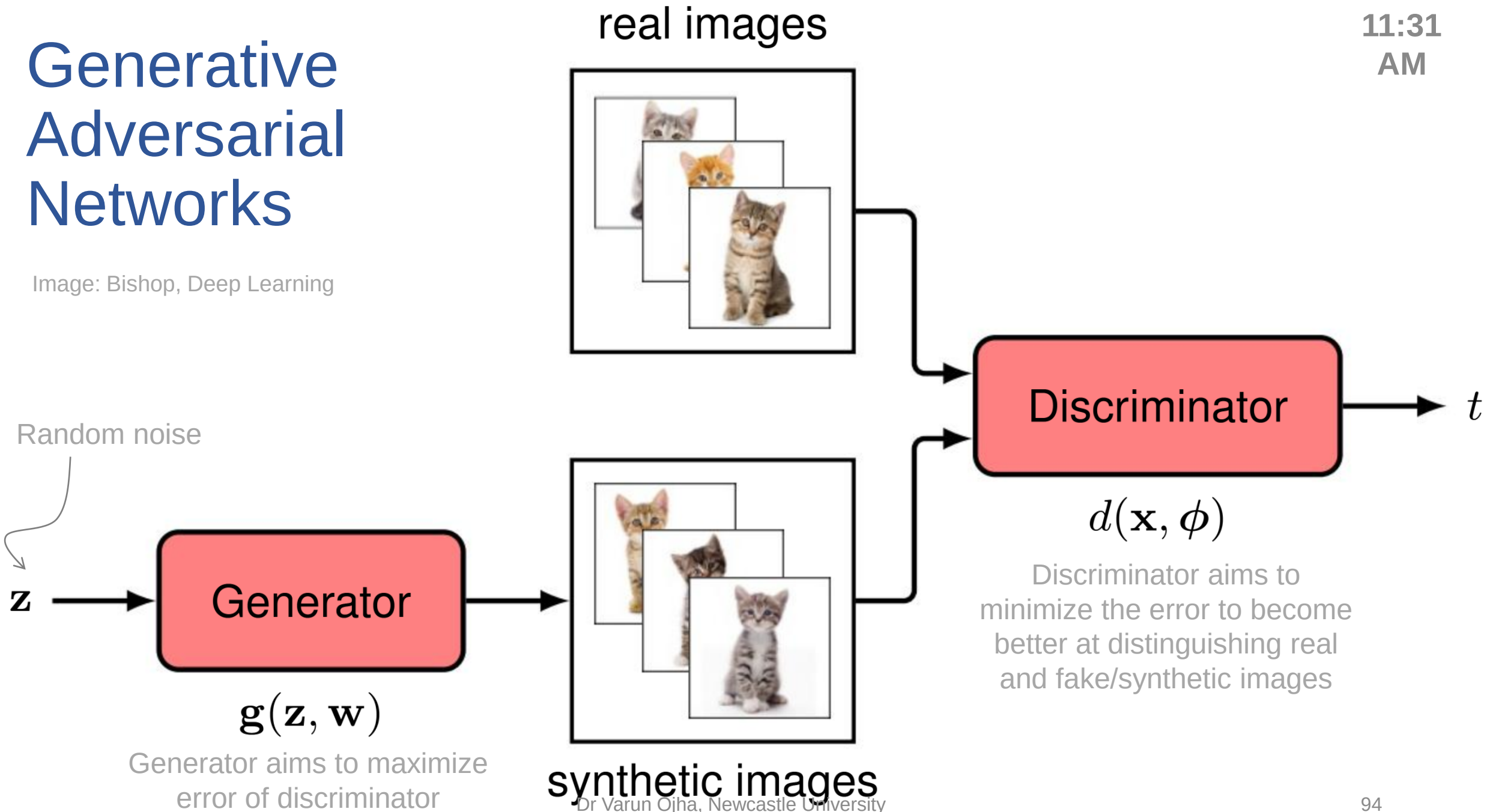
original



Generative Adversarial Networks

Image: Bishop, Deep Learning

11:31
AM



Generative Adversarial Networks

min-max loss

11:31
AM

real image from dataset,
i.e., label '1' so D should
output '1' for real

Synthetic image from
generator, i.e. label '0' so
D should output '0' for fake

$$\min_G \max_D V(D, G) = \mathbb{E}_x [\log(D(x))] + \mathbb{E}_z [\log(1 - D(G(z)))]$$

Generator aims to maximize error of discriminator, i.e., G to minimize:

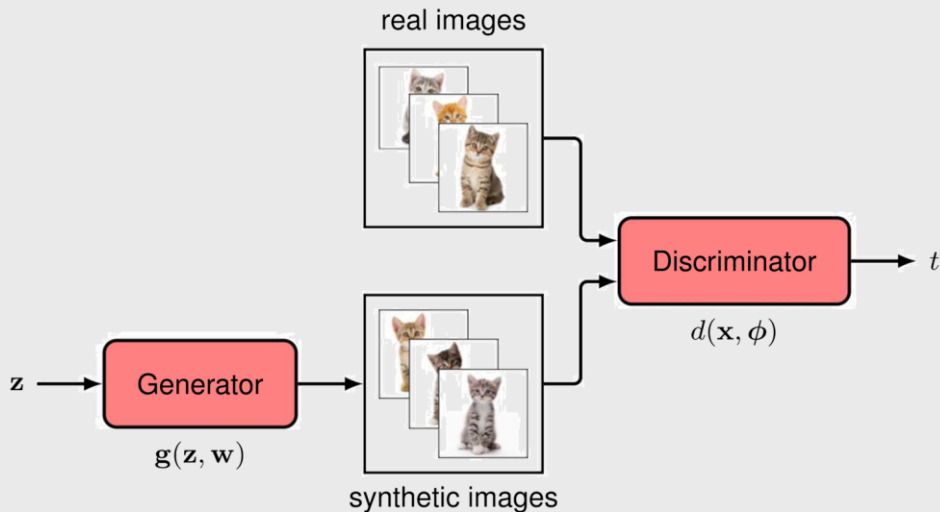
$$\mathbb{E}_z [\log(1 - D(G(z)))]$$

low value means generator create realistic images, i.e., if $D(G(z)) = 1$ means generator fool Discriminator and its wins

Discriminator aims to minimize the error to become better at distinguishing real and fake/synthetic images, i.e., D to maximize the probability of assigning the correct label to both training examples and samples from G by maximizing:

$$\mathbb{E}_x [\log(D(x))] + \mathbb{E}_z [\log(1 - D(G(z)))]$$

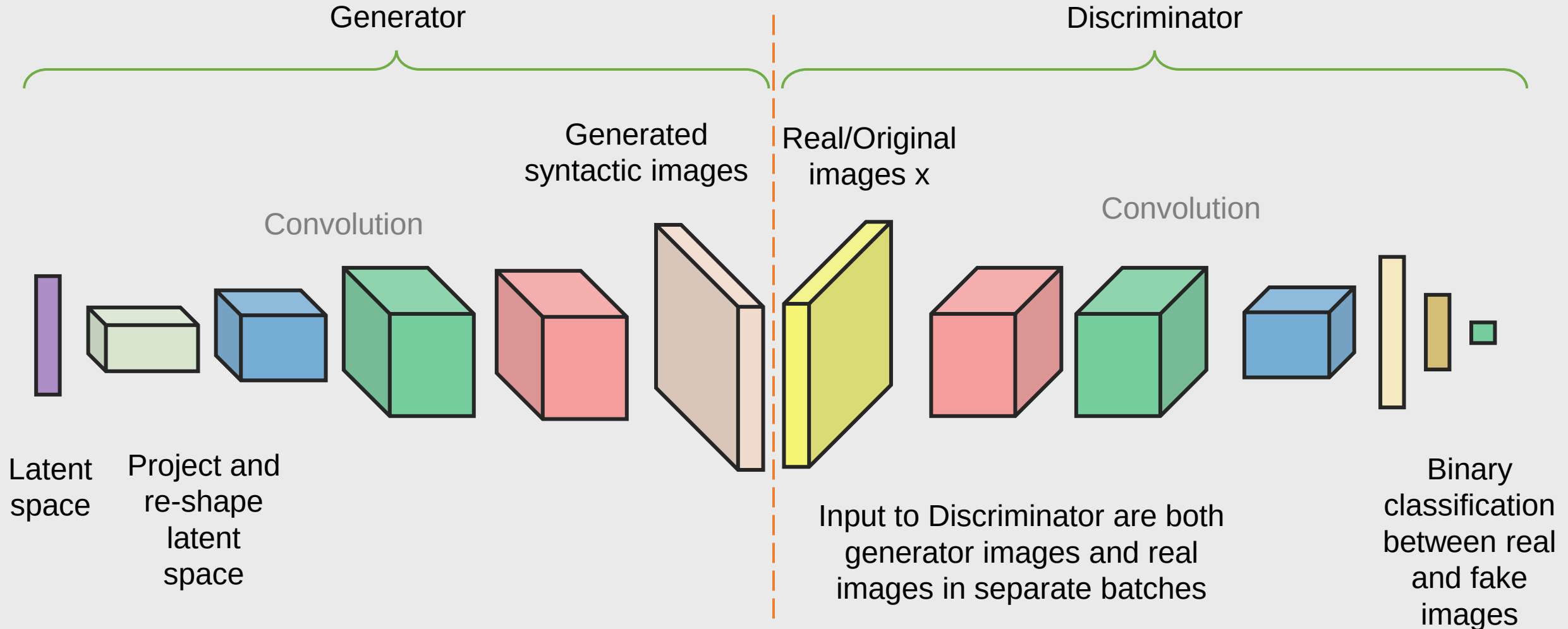
low value means discriminator is able to identify the real vs fake (i.e., if $D(x) = 1$ and $D(G(z)) = 0$ will give $\log 1 + \log 1 = 0$) that will let Discriminator win



Deep Convolutional GAN

11:31
AM

The task of the generator is to produce data which the discriminator predicts as being 'real', meaning that it closely resembles the training dataset.



StyleGAN to Generate China City Scape

My student project



(Jia 2022)

